Overnet presenta:

Claude Code: sviluppo software con AI agentica

di Antonio Piemontese

Ottobre 2026

Indice dei contenuti del corso

01
Introduzione al corso ›

02
Cosa è la AI agentica: altro ›

03
Claude Code a 360° ›

04
Il Vibe Coding ›

05
Le mani sullo strumento ›

06
Varie ›

Per navigare nelle slide (a schermo pieno: maiusc+F5):
Cliccare su › per andare al capitolo
Cliccare poi su per tornare a questo indice

pratica

teoria

Introduzione al corso

Martedì scorso, uno sviluppatore Python di nome Marco ha passato **45 minuti** a discutere con ChatGPT di un bug nella sua app Flask...

Ha incollato il traceback. ChatGPT gli ha dato una correzione.

La correzione ha rotto qualcosa in un file che il modello non conosceva.

Ha incollato anche quell'errore — Il modello si è scusato, ha riscritto la funzione da zero e ha ignorato gli ultimi tre messaggi.

Marco ha provato a spiegare la struttura del progetto. A metà strada il modello ha perso il contesto. Ha aperto un nuovo thread. Ha incollato di nuovo il codice. Ha ottenuto una risposta diversa, in contraddizione con la prima.

Ha chiuso la scheda, ha aperto l'editor e ha corretto il bug da solo. Ci ha messo dodici minuti.

Marco non è un cattivo sviluppatore. Rilascia codice ogni settimana. Scrive in Python da sei anni.
Usa l'AI tutti i giorni.

E ogni giorno gli passa per la testa lo stesso pensiero: *deve esserci un modo migliore di fare tutto questo.*

Ha ragione. C'è.

Di recente abbiamo intervistato **278** sviluppatori Python. La maggior parte di loro somigliava molto a Marco:

«L'AI va bene per risolvere un piccolo compito, ma solo per quello.»

«Portare avanti una lunga sessione, poi sbattere contro i limiti del contesto e dover ricominciare da capo in un nuovo thread.»

«È facilissimo perdere di vista l'architettura di un'applicazione. C'è una frustrazione molto particolare quando qualcosa si rompe e non si ha idea di come risalire il filo del codice.»

Lo schema è sempre lo stesso.

L'AI basata su chat non vede il progetto. Non sa quali file esistono, non può eseguire i test e dimentica tutto quando la finestra di contesto si riempie. Così il lavoro di integrazione ricade su di noi: copiare codice da una finestra all'altra, rispiegare da capo il progetto, rivedere un output di cui non ci si fida fino in fondo.

65% degli sviluppatori del nostro sondaggio è fermo esattamente a questo punto. Usano l'AI per programmare, li aiuta nelle piccole cose, ma crolla davanti a qualsiasi cosa reale.

E sotto a tutto questo, una domanda che la maggior parte degli sviluppatori non pronuncia ad alta voce: *se non riesco a capire come lavorare con questi strumenti, che ne sarà della mia carriera?*

Esiste un modo diverso di lavorare.

Due note di traduzione: abbiamo lasciato invariati i tecnicismi d'uso corrente in italiano (traceback, thread, editor) e reso "context window" con "finestra di contesto", coerentemente con la terminologia usata nei corsi. Se serve, possiamo anche riunire tutte le sezioni tradotte finora in un unico documento impaginato.

Philipp Acsany

Real Python core team member. Uses Claude Code daily to build tools, automate workflows, and ship Python projects.

Invece di copiare codice da una finestra di chat, lavoriamo con un agente dentro il nostro progetto

Con Claude Code, l'AI gira nel nostro terminale. Legge i nostri file, li modifica direttamente, esegue i test, vede gli errori e li corregge. Gestisce git e lavora sull'intera codebase, non su un frammento alla volta.

Due scelte di traduzione da segnalare: "scaffold" è reso con "creare da zero la struttura / creare l'impalcatura" (è il termine tecnico per la generazione automatica dello scheletro iniziale di un progetto), e "skills" è reso con "Skill" maiuscolo perché "portable toolkit of reusable skills" si riferisce quasi certamente alla funzionalità Skill di Claude Code vista nella slide precedente, non a competenze generiche.

CHE COS'È L'AI AGENTICA?

Un sistema AI che persegue un obiettivo, sceglie le azioni da compiere e usa strumenti, adattando il percorso ai risultati con un grado di autonomia definito.

GLI ELEMENTI

MODELLO AI
Spesso un modello linguistico (LLM): interpreta e sceglie i passi.

CONTESTO
Istruzioni, dati e risultati del lavoro già svolto.

STRUMENTI
Consentono di operare su file, codice, web e applicazioni.

Il modello sceglie le azioni; il software le esegue.

IL CICLO DELL'AGENTE

1. PIANIFICA
Sceglie il prossimo passo.

2. AGISCE
Usa gli strumenti disponibili.

3. VERIFICA
Controlla esiti e progressi.

4. ADATTA
Rivede il piano in base agli esiti.

OBIETTIVO

Ripete se serve; conclude o si ferma ai limiti impostati.

UN ESEMPIO CONCRETO

«Analizza questi file e prepara un rapporto.»

Legge i file autorizzati.

Sceglie ed esegue le analisi.

Controlla dati e calcoli.

Prepara il rapporto.

Se mancano informazioni, può chiedere chiarimenti.

IL CONTROLLO UMANO

La persona definisce obiettivi, permessi e limiti, approva le azioni quando previsto e valuta il risultato.

IL PANORAMA DELL'AI AGENTICA

Ottobre 2026 - I Principali Strumenti e Framework per Agenti Autonomi

(aggiornato a ottobre 2026)

MULTI-AGENT FRAMEWORKS
(Coordinamento)

crewai
Framework orchestrazione agenti.

AG AutoGen
Sviluppo di flussi multi-agente di Microsoft

LangGraph
Agenti a stati complessi e ciclici.

AGENTIC CODING & TASK AGENTS
(I Tool più Importanti e Rivoluzionari)

FOCUS PRINCIPALE - OTTOBRE 2026

Claude Code (Anthropic)
L'agente CLI autonomo di Anthropic. Naviga, modifica, testa ed esegue comandi direttamente da terminale nel codebase

OpenAI Codex
L'ambiente agentico e di sviluppo di OpenAI. Risolve task complessi, corregge vulnerabilità e scrive software in autonomia

DATA & MEMORY AGENTS
(Integrazione RAG)

LlamaIndex
Integrazione di dati e RAG avanzato

LangChain
Libreria per catene di prompt e interazione con LLM.

ENTERPRISE SUITES

Semantic Kernel
Integrazione enterprise in .NET e Python.

PydanticAI
Validazione dati e sviluppo agenti basato su Pydantic.

AGENT OPS & EVALUATION
(Monitoraggio)

AgentOps
Monitoraggio, tracciamento e debugging di agenti.

CHE COSA È CLAUDE CODE

Una sintesi pratica per orientarsi rapidamente

In breve

Claude Code è un assistente di coding agentico: legge il contesto del progetto, conversa con l'utente e può eseguire azioni utili per analizzare, scrivere, modificare e verificare il codice.

1. COSA FA

- Legge file e struttura del progetto
- Propone o modifica codice
- Esegue comandi e controlli
- Aiuta nel debug e nel refactoring

2. COME SI USA

1. Apri Claude Code
2. Descrivi l'obiettivo
3. Claude legge, propone, agisce
4. Tu verifichi e iteri

È una collaborazione tra utente e AI.

PAROLE CHIAVE

- prompt
- contesto
- tool
- workflow
- iterazione
- verifica

3. DOVE LAVORA

- Nel terminale / CLI
- Con editor e IDE integrati
- Su file, repository e tool del progetto

4. PERCHÉ È UTILE

- Accelera attività ripetitive
- Riduce il lavoro manuale
- Aiuta a esplorare codice sconosciuto
- Supporta prototipi, fix e miglioramenti

5. COSA NON È

- Non sostituisce il giudizio umano
- Non garantisce che tutto sia corretto
- Non conosce automaticamente tutto il progetto senza leggerlo
- Non elimina la necessità di test e revisione

Idea centrale | Claude Code non è solo una chat: è un ambiente operativo per lavorare sul software in modo conversazionale, progressivo e assistito dall'AI.

Gian Segato
Ricercatore Anthropic

“Se puoi farlo col computer, puoi farlo con Claude Code".
Era chiaro che Claude Code sarebbe diventato una sorta di nuovo sistema operativo. Cioè l'idea di interagire col computer attraverso il linguaggio naturale. Il punto fondamentale è che tutti i lavori d'ufficio oggi hanno un qualche tipo di componente informatica. Lo facciamo dal computer e l’Informatica cos'è? Codice.

L’ecosistema Claude: quattro modi di lavorare

Conversare, sviluppare software, delegare attività, progettare esperienze visive

Claude AI

LA CONVERSAZIONE

Assistente generalista su claude.ai e nelle app.

Per: ragionare, scrivere, studiare e analizzare contenuti.

“Spiega questo documento e riassumi i punti chiave.”

Focus: capire ed elaborare

Claude Code

LO SVILUPPO SOFTWARE

Agente che opera sul progetto e sul repository.

Legge e modifica file, esegue comandi e test, lavora con Git.

Accesso: terminale, IDE, desktop e web.

“Correggi il bug ed esegui i test.”

Focus: realizzare e mantenere software

Claude Cowork

IL LAVORO OPERATIVO

Capacità agentiche per attività oltre la programmazione.

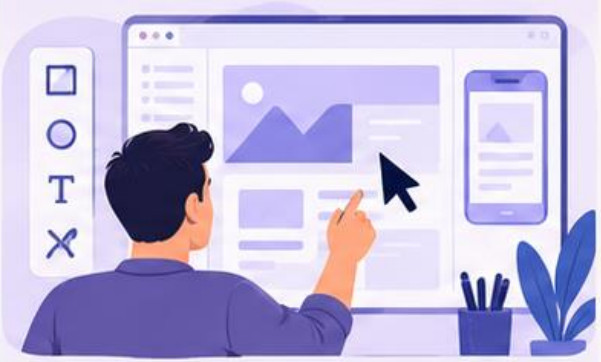
Pianifica ed esegue compiti in più passaggi su file e strumenti autorizzati.

“Analizza questi file e prepara un report.”

Focus: delegare un risultato

Claude Design

LA PROGETTAZIONE VISIVA



Crea e perfeziona design, mockup e prototipi interattivi.

Si lavora tramite conversazione e modifiche sul canvas.

“Progetta il prototipo del portale clienti.”

Focus: esploare e rifinire il design

DISTINTI PER FUNZIONE, SEMPRE PIÙ INTEGRATI

Da settembre 2026 chat e **Cowork stanno confluendo in un’unica esperienza Claude.** Design è disponibile anche nelle conversazioni e in Claude Code.
Le capacità si sovrappongono: la scelta dipende dal risultato e dall’ambiente di lavoro.

Quadro aggiornato al 22 ottobre 2026 • Fonti: documentazione ufficiale Anthropic / Claude

AI IN CLOUD vs AI AGENTICA

Differenze pratiche e quando usare Claude AI o Claude Code

Idea chiave
Claude AI e Claude Code non sono concorrenti diretti:
il primo è una superficie conversazionale general purpose,
il secondo è un ambiente agentico operativo per lavorare sul software.

1. DUE MODI DI USARE L'AI

AI in cloud · Claude AI

- Interazione soprattutto conversazionale
- Ottima per domande, testi, riassunti e brainstorming
- Lavora bene su prompt, file allegati e conoscenza generale
- Di solito non opera direttamente nel tuo progetto locale
- Più adatta a ragionare, spiegare, scrivere e analizzare

Punti forti
- semplicità
- rapidità
- versatilità
- accesso immediato

Dalla sola conversazione all'azione sul progetto

AI agentica · Claude Code

- Interazione conversazionale + azione operativa
- Legge contesto, file e struttura del progetto
- Può usare tool, terminale e workflow guidati
- Propone, modifica, verifica e itera sul codice
- Più adatta a sviluppo software e lavoro su repository locale o GitHub.com

Punti forti
- contesto del progetto
- automazione operativa
- iterazione
- debug e refactoring

2. CARATTERISTICHE A CONFRONTO

Claude AI	Voce	Claude Code
capire, spiegare, creare contenuti	**Obiettivo**	costruire, modificare e migliorare software
prompt e allegati	**Contesto**	prompt + file + struttura + tool
suggerisce	**Azione**	suggerisce e agisce
web / app cloud	**Ambiente**	terminale, editor, IDE, file locali e repository
analisi generale	**Tipo di lavoro**	workflow tecnico e iterativo
revisione delle risposte	**Controllo umano**	revisione di codice, azioni e risultati

3. QUANDO USARE CLAUDE AI

- **Per fare domande e capire un argomento**
- Per riassumere documenti o appunti
- Per scrivere, riscrivere o tradurre testi
- Per brainstorming e idee iniziali
- **Per valutare strategie prima di passare all'implementazione**

4. QUANDO USARE CLAUDE CODE

- **Per avviare un progetto software**
- Per esplorare codice esistente
- Per scaffolding, workflow e organizzazione
- **Per iterare, fare debug** e refactoring
- Per eseguire controlli e verifiche
- Per lavorare su repository locale o GitHub.com
- Per lavorare direttamente sui file locali
- Per evitare il **continuo** copia-incolla tra chatbot e IDE

5. PERCHÉ USARE CLAUDE CODE ANZICHÉ CLAUDE AI

1. Perché non si limita a rispondere: lavora sul progetto
2. Perché usa il contesto reale del codice, non solo il prompt
3. Perché supporta un ciclo completo: leggere → proporre → agire → verificare
4. Perché è più adatto a task tecnici multi-step
5. Perché riduce il lavoro manuale nello sviluppo software
6. Perché lavora direttamente sui file locali
7. Perché riduce il continuo copia-incolla tra chatbot e IDE

Nota: Claude Code non sostituisce il giudizio del docente o dello sviluppatore. Anche nell'AI agentica restano essenziali verifica, test e supervisione umana.

In una frase | Usa **Claude AI** quando ti serve soprattutto **pensare** con l'AI. Usa **Claude Code** quando ti serve far lavorare l'AI sul software, sui **file locali e sul repository**.

Claude Code: dove rende di più

Uno strumento agentico: legge il codebase, modifica file, esegue comandi e si integra con gli strumenti di sviluppo.
Non si limita a suggerire: agisce.

TASK SU CUI È COSTRUITO

Lavoro multi-file su codebase esistenti
Riceve un obiettivo in linguaggio naturale, pianifica, modifica più file e verifica il risultato.

Debugging a ciclo chiuso
Parte da errore o sintomo, trova la causa nel repo, corregge e lancia i test.

Lavoro tedioso e ripetitivo
Test mancanti, fix di lint, merge conflicts, aggiornamento dipendenze, release notes.

Workflow Git
Staging, commit message, branch, pull request; utile anche in CI per review e issue triage.

Esplorazione di codice sconosciuto
Spiega architettura e flussi di un repository reale.

Automazione da riga di comando
Componibile in pipeline Unix, script e CI.

DOVE RENDE MENO

Contesto fuori dal repo
Se non gli indichiamo esplicitamente dove guardare, può perdere informazioni importanti.

Conoscenza generale o brainstorming puro
Per domande secche o idee generiche, la chat è spesso più economica e adatta.

Requisiti ambigui
Senza piano e criteri di qualità espliciti, tende a proporre quick fix e accumulare debito tecnico.

Sessioni lunghe e sporche
Il degrado del contesto è il rischio principale; aiutano /clear frequente e un CLAUDE.md asciutto.

REGOLA PRATICA

Se il task ha un criterio di verifica eseguibile — test che passano, **lint pulito, script che gira, diff che si applica** — **Claude Code è nel suo territorio migliore**, perché può iterare da solo fino al **verde.**

Se il criterio è soggettivo o esterno al progetto, spesso conviene restare in chat.

Riferimento ufficiale: code.claude.com/docs/en/overview

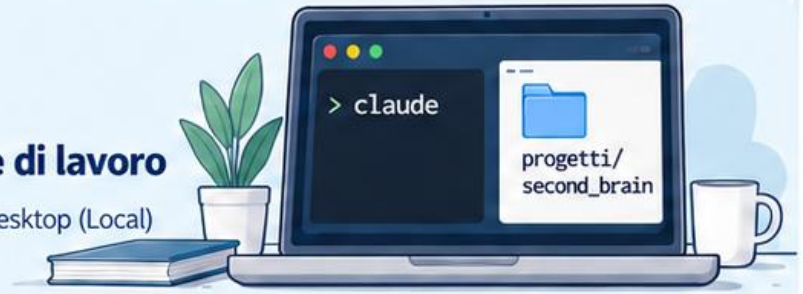
Claude Code: locale e cloud

Due ambienti complementari. La scelta dipende da dove devono lavorare codice e strumenti.

LOCALE

Controllo sull'ambiente di lavoro

Terminale · VS Code / JetBrains · Desktop (Local)



Esecuzione	Sul proprio PC, nella cartella del progetto.
Risorse	File, conda, notebook e GPU locali, secondo i permessi.
Configurazioni	Utente, progetto, locale e policy aziendali.
Integrazioni	Hook, MCP e plugin configurati sul PC.
Requisiti	Installazione locale. Installer nativo senza Node.js.
Ideale per	Sviluppo iterativo, debug, test e integrazione con l'IDE.

CLOUD

Delega su un ambiente remoto

Claude Code sul web · **claude.ai/code**

Esecuzione	VM isolata di Anthropic con clone del repository.
Risorse	File e tool della VM; il PC resta separato.
Configurazioni	Settings del repo committati, ambiente cloud e policy server.
Integrazioni	Hook e MCP del repo; plugin non installati automaticamente.
Requisiti	Percorso web: browser, piano abilitato e repo GitHub collegato.
Ideale per	Task delimitati e paralleli, anche a PC spento. Risultati da rivedere e possibile PR.

IL REPOSITORY PORTA LE REGOLE CONDIVISE

Commit e push su GitHub rendono disponibili istruzioni e impostazioni condivise alle sessioni cloud che clonano quel branch.

`CLAUDE.md`

`.claude/settings.json`

Restano sul PC:
`~/.claude/settings.json`
e `.claude/settings.local.json`

PER IL CORSO

Locale come base di lavoro · Cloud per attività delegabili

Locale indica dove girano strumenti e codice; il modello Claude è remoto.

CLI e browser sono interfacce: il terminale può avviare il cloud; Remote Control può controllare una sessione locale.

Cloud: scenario Anthropic con un repository.

Fonti: documentazione ufficiale Claude Code.

CLAUDE CODE: LOCALE E CLOUD

Dove lavorano agente, codice e strumenti

Inferenza del modello remota in entrambi

L'interfaccia non determina il luogo di esecuzione.

CLI · ESECUZIONE LOCALE

AMBIENTE
Agente, file, shell, build e test sulla macchina di sviluppo.

RISORSE E CONFIGURAZIONE
Working tree locale, strumenti installati, impostazioni utente e progetto, MCP e plugin configurati.

QUANDO CONVIENE
Debug interattivo, integrazione con l'IDE e dipendenze locali.

L'host deve restare attivo e connesso.

SESSIONE CLOUD · HOSTED

AMBIENTE
Agente, checkout, build e test in una VM remota Linux.
Docker supportato.

RISORSE E CONFIGURAZIONE
Dipendenze, rete e accessi da configurare. Settings del repo, ambiente cloud e policy server; plugin non installati automaticamente.

QUANDO CONVIENE
Compiti delimitati, test riproducibili e attività indipendenti in parallelo.

Può proseguire con il portatile spento.

PASSAGGIO TRA LOCALE E CLOUD

Commit e push su GitHub → Sessione cloud → Revisione e integrazione

Possibile anche un bundle locale. Nessuna sincronizzazione continua.

Il cloud può essere avviato dalla CLI: `claude --cloud`

REMOTE CONTROL

Browser o app → agente locale.
Stessi file e strumenti; host attivo e connesso.

SELF-HOSTED

Team / Enterprise · beta

Sessioni cloud su runner aziendali.
Esecuzione interna; contenuti inviati ad Anthropic.

Fonte: documentazione ufficiale Claude Code · code.claude.com/docs · 21/10/2026

Claude Code su Windows

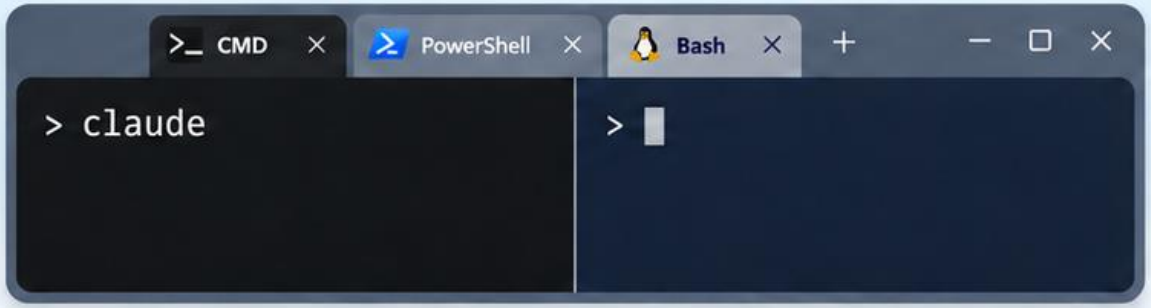
La shell esegue i comandi. Windows Terminal organizza le sessioni.

L'APP CHE OSPITA LE SHELL

Windows Terminal

App Terminale di Windows

Scaricabile dal Microsoft Store



Più schede
Più sessioni in una finestra

Riquadri affiancati
Sessioni visibili insieme

Profili per le shell
Un menu per ambienti diversi

Personalizzazione
Font, colori e scorciatoie

Profili per shell installate e configurate sul PC.

DA DOVE AVVIARE CLAUDE CODE

CMD
Prompt dei comandi

La shell classica di Windows.

PowerShell
Windows PowerShell
PowerShell

Comandi e automazione

Shell con scripting avanzato.

Non confondere con il vecchio Windows PowerShell

Bash
Git Bash oppure Linux / WSL

Git Bash lavora su Windows. Con WSL, installare Claude Code nell'ambiente Linux.

Anaconda Prompt
CMD con conda già attivo

Può avviare Claude Code. Opzionale: utile per progetti Python.

PER INIZIARE

1. **Aprire una shell** →
2. **Entrare nella cartella del progetto** →
3. `claude` — Avvia Claude Code

Requisito: Claude Code installato nell'ambiente scelto e comando `claude` disponibile nel PATH.

Fonti: Claude Code Docs · Microsoft Learn · Anaconda Docs

IDEE
DOMANDE
PROGETTI
RISULTATI
TUTTO INSIEME

Novità del 16 settembre

Claude unifica Chat e Cowork in un'unica esperienza

Un'unica conversazione. Più possibilità.

Anthropic ha annunciato che non è più necessario scegliere prima dove iniziare: la stessa conversazione può gestire sia domande rapide sia lavori più complessi.

1 Prima

L'utente doveva scegliere tra:

- **Chat** → per domande rapide
- **Cowork** → per attività più articolate

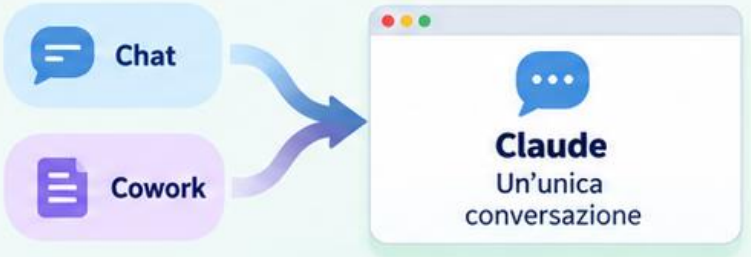
Chat
Domande rapide

oppure

Cowork
Attività più articolate

2 Ora

Chat e Cowork diventano un'unica esperienza.
Non c'è più una modalità da scegliere prima di iniziare.



3 Come funziona

Si chiede ciò che serve in un'unica conversazione.

Claude decide quali capacità usare per svolgere il compito.

Può rispondere a una domanda oppure portare avanti un lavoro più ampio.

4 Cosa può fare

- **Ricerca e sintesi**
- **Report e fogli di calcolo**
- **Presentazioni**
- **Lavori che continuano anche nel cloud**

Più attività possono essere gestite nella stessa conversazione.

5 Nuovi output

Arrivano anche:

- **Claude Docs**
- **Claude Slides**
- **Claude Design**

nella conversazione.

I risultati possono essere modificati, condivisi ed esportati (inclusi PowerPoint e PDF).

6 Importante

- Il rilascio è graduale
- Prima per i piani Pro e Max
- Su web, desktop e mobile
- Team e Free seguiranno dopo
- Chat, task, progetti e impostazioni restano disponibili

Per impostazione predefinita, Claude chiede conferma prima di eseguire azioni: il controllo resta all'utente.

In sintesi: dal 16 settembre Claude non separa più Chat e Cowork.
Una sola conversazione può rispondere, creare documenti e portare avanti attività complesse.

STESSE IDEE.
MAGGIORI POSSIBILITÀ.

PROGRAMMA DEL CORSO

Claude Code in 2 giornate

Panoramica sintetica del percorso: giovedì e venerdì

Obiettivo

Impostare le basi pratiche per lavorare con Claude Code, creare un primo progetto Python e imparare a raffinarlo progressivamente.

GIOVEDÌ · PRIMA GIORNATA

Dalle basi al primo progetto

1. Introduzione a Claude Code
2. Requisiti di installazione dei vari strumenti
   - VM/lab di godeploy
   - Claude Code
   - Terminale Windows
   - Visual Studio Code
   - Miniconda o python.org
   - GitHub Desktop
   - Git for Windows
   - uv
3. Creazione dell'ambiente virtuale (conda o venv)
4. Alcuni comandi iniziali particolari di Claude Code `(/...)`
5. Scaffolding: infrastruttura del progetto
6. Creazione del primo progetto Python
7. Costruzione di un workflow iniziale

Focus: avvio, setup e struttura

Setup

↓

Primo progetto

↓

Raffinamento

VENERDÌ · SECONDA GIORNATA

Raffinamento e concetti avanzati

1. Raffinamento del progetto creato il primo giorno
2. Iterazione sul progetto
3. Debug
4. Skill e skill agentiche
5. Buzzwords: alcuni termini di moda
6. Agenti e sub-agenti
7. Server MCP

C'è tempo?

8. Mappa finale dei termini

Focus: miglioramento, agenti e terminologia

Nota finale

Non ci sarà il tempo per vedere e implementare tutto. Alla fine della seconda giornata forniremo comunque una mappa dei termini per orientarsi tra i concetti principali.

Un corso di **3 giorni** proposto in **2 giorni** per ridurre impegno e costi

COME L'AI HA CONTRIBUITO A QUESTE SLIDE

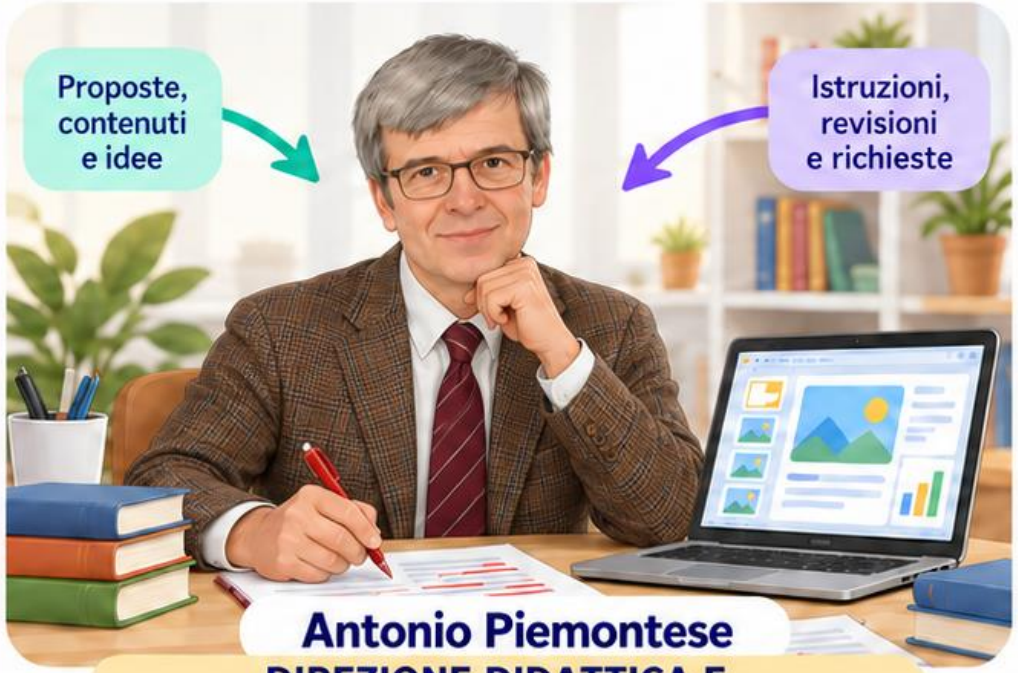
ChatGPT e Claude AI hanno affiancato il docente nella preparazione dei materiali didattici.

IL SUPPORTO DELL'AI

ChatGPT | Claude AI

- Stesura e sintesi dei testi
- Riformulazioni e traduzioni
- Elaborazione di esempi e schemi
- Preparazione dei contenuti visivi

SOTTO LA DIREZIONE DEL DOCENTE



Antonio Piemontese
DIREZIONE DIDATTICA E RESPONSABILITÀ DEI CONTENUTI

LE SCELTE DEL DOCENTE

- Ha definito obiettivi e impostazione.
- Ha guidato il lavoro con istruzioni.
- Ha richiesto modifiche e adattamenti.
- Ha selezionato i contenuti delle slide.

DICHIARAZIONE DI TRASPARENZA

Queste slide sono state preparate con il supporto di **ChatGPT e Claude AI, sotto la direzione del docente** Antonio Piemontese, che mantiene la responsabilità dei contenuti.

RIFERIMENTO ALL'AI ACT
Regolamento (UE) 2024/1689 — art. 50

Gli obblighi di trasparenza riguardano casi specifici. Per i testi pubblicati su temi di interesse pubblico è prevista un'esenzione in presenza di revisione umana o controllo editoriale, insieme alla responsabilità editoriale della pubblicazione.

Fonte normativa: Commissione europea — AI Act Service Desk, art. 50

Gli strumenti AI del corso

Esecuzione sul progetto e assistenza durante le attività

Claude Code

circa **70%**

ESECUZIONE SUL PROGETTO LOCALE

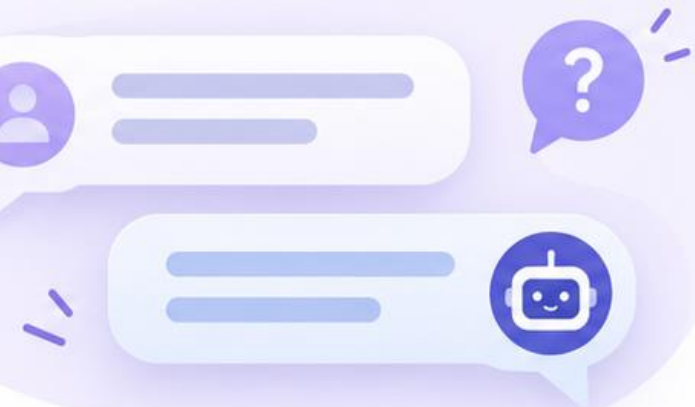
Claude Code viene eseguito sul PC.
Lavora sui file ed esegue le attività del progetto.

Copilot Chat

circa **30%**

ASSISTENTE AI

Offre spiegazioni e chiarimenti.
Aiuta a ragionare durante le attività.



Ripartizione indicativa dell'utilizzo durante il corso

CLAUDE CODE: PIANI DI ACCESSO

Sintesi del Markdown · Ottobre 2026 · Prezzi web in USD

PIANI INDIVIDUALI

Free
0 $

Claude Code: **NO**

Uso base di Claude.
La finestra d'uso si azzera ogni 5 ore.

Pro
20 $/mese

fatturazione mensile
oppure 200 $/anno anticipati
≈ 17 $/mese con annuale

Claude Code: **SÌ**

Almeno **5× Free** per sessione
+ limiti settimanali
Punto d'ingresso individuale

Max 5x
100 $/mese

solo fatturazione mensile

Claude Code: **SÌ**

5× Pro per sessione
100 $/mese di crediti API inclusi
Uso intensivo dalla CLI

Max 20x
200 $/mese

solo fatturazione mensile

Claude Code: **SÌ**

20× Pro per sessione
200 $/mese di crediti API inclusi
Uso intensivo dalla CLI

ORGANIZZAZIONI E ALTERNATIVA API

Team
2–150 posti

Claude Code: **SÌ**

Standard e Premium mescolabili

PREZZI PER POSTO AL MESE

Standard:	**20 $** con annuale · **25 $** con mensile
Premium:	**100 $** con annuale · **125 $** con mensile

Standard > Pro · Premium = 5× Standard
Fino a 500 $/mese di crediti API condivisi
SSO e amministrazione centralizzata

Enterprise
Claude Code: **SÌ**

20 $/posto/mese
+ consumo a tariffe API

Fatturazione annuale · minimo 20 posti
Tetti di spesa per utente e organizzazione
Governance avanzata: RBAC, SCIM e audit log

Console/API
Claude Code: **SÌ**

Pagamento per token

Nessun canone mensile

CLI · VS Code · JetBrains
Alternativa all'abbonamento

I prezzi sono quelli riportati nell'allegato e sono soggetti a modifica.
Sugli store mobili possono variare.

Fonte: "Piani di abbonamento Claude (ottobre 2026)" · @Antonio

Piani di abbonamento Claude (ottobre 2026)

19 ott 2026 ·

I prezzi sono pubblicati in dollari USA (la pagina ufficiale non riporta prezzi in euro), valgono per la sottoscrizione via web — sugli store mobili possono variare — e sono dichiarati soggetti a modifica a discrezione di Anthropic.

Tabella riassuntiva

Piano	Prezzo	Limiti d'uso	Claude Code
Free	0 $	Uso base, finestra che si azzera ogni 5 ore	No
Pro	17 $/mese con fatturazione annuale (200 $ anticipati) · 20 $/mese mensile	Almeno 5× il Free per sessione, più limiti settimanali	Sì
Max 5x	100 $/mese (solo mensile)	5× il Pro per sessione	Sì
Max 20x	200 $/mese (solo mensile)	20× il Pro per sessione	Sì
Team	Posto Standard: 20 $/posto/mese annuale (25 $ mensile) · Posto Premium: 100 $ annuale (125 $ mensile)	Standard: superiore al Pro · Premium: 5× lo Standard	Sì
Enterprise	20 $/posto/mese (annuale, minimo 20 posti) + consumo fatturato a tariffe API	Scala col consumo; gli admin impostano tetti di spesa per utente e organizzazione	Sì

Team è pensato per gruppi da 2 a 150 posti, con posti Standard e Premium mescolabili.

Piani individuali: offerta tecnica incrementale

Free — Modelli Sonnet e Haiku; fino a 5 Projects; memoria, connettori, ricerca web, creazione file ed esecuzione codice, Artifacts (Design, Slides, Docs), voice mode, skills; contesto fino a 1M di token (varia per modello, come su tutti i piani). Niente Claude Code né Research.

Pro (aggiunge al Free) — Opus incluso; Fable accessibile tramite *crediti d'uso* (consumo extra fatturato a tariffe API quando si esauriscono i limiti); **Claude Code** su tutte le superfici; Research; pianificazione e handoff dei task; Claude in Chrome, Claude per Microsoft 365/Outlook, Claude Science, Dashboards (beta). Nessun controllo amministrativo; training sui dati disattivabile (opt-out).

Max 5x / 20x (aggiunge al Pro) — Limiti 5× o 20×; **Fable incluso fino al 50% dei limiti settimanali**; crediti API inclusi: 100 $/mese (5x) o 200 $/mese (20x); accesso prioritario nelle ore di punta e accesso anticipato alle funzioni nuove. È il piano tipico per chi usa Claude Code in modo intensivo dalla CLI.

Piani per organizzazioni: Team ed Enterprise

Team (2–150 posti, Standard e Premium mescolabili) — Tutto quanto sopra più: condivisione di Projects, enterprise search nei connettori, Motion (beta), fino a 500 $/mese di crediti API *condivisi* dal team; Fable al 50% dei limiti settimanali sui posti Premium. Lato amministrazione: SSO, verifica del dominio, fatturazione centralizzata, controlli admin sui connettori, analytics d'uso, skills a livello di organizzazione; **nessun training sui dati per impostazione predefinita**. Non include: SCIM, audit log, Compliance API, retention personalizzata, offerta HIPAA. Pagamento con carta, mensile o annuale (niente bonifico/fattura).

Enterprise — Modello nuovo: quota posto bassa (20 $) ma **consumo fatturato a tariffe API**, con tetti di spesa impostabili dagli admin. Tutti i modelli inclusa Fable; tutto il Team più Claude Security (beta) e il pacchetto completo di governance: SSO, permessi granulari (RBAC), SCIM, audit log, Compliance API, retention personalizzata, controlli di rete e IP allowlisting, offerta HIPAA-ready, inference hooks. Fatturazione annuale (carta o ACH in self-serve; con il sales anche fattura con termini di pagamento).

Note pratiche per Claude Code

- Per usare Claude Code individualmente il **Pro** è il punto d'ingresso; **Max** è il piano tipico per l'uso intensivo dalla CLI.
- In alternativa all'abbonamento, Claude Code (CLI, VS Code, JetBrains) funziona anche con un account **Console/API a consumo**: si paga per token, senza canone mensile.

Fonti

Verificate il 19 ottobre 2026:

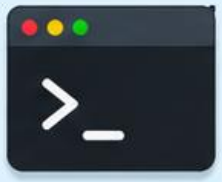
- Pricing — claude.com
- Che cos'è il piano Max — Supporto Claude
- Che cos'è il piano Pro — Supporto Claude

CLAUDE CODE: 6 MODALITÀ DI ACCESSO

Le “superfici” descritte nel Markdown • Ottobre 2026

STESSO MOTORE | Configurazione del repository: CLAUDE.md • impostazioni • server MCP

raccomandato!



1. Terminale (CLI)

Avvio nel progetto: `claude`
Funzioni complete, script, pipe e SSH.
Print mode: `-p`

Richiede pratica; diff testuali.
Windows: Git for Windows per Bash.

2. VS Code / Cursor

Estensione con diff inline e revisione dei piani.
File e codice nel contesto dell’editor.

Legata al progetto aperto; poca automazione.

3. JetBrains

IntelliJ IDEA • PyCharm • WebStorm

Diff interattivi e selezione come contesto.

Richiede la CLI installata separatamente.

4. App desktop

macOS • Windows • Ubuntu/Debian beta

File locali, sessioni parallele e pianificate.
Anche sessioni cloud.

Abbonamento a pagamento.
CLI separata non necessaria.

5. Web

`claude.ai/code`

Zero installazione; task cloud in parallelo.
Prosegue anche a computer spento.

Repository collegati: niente accesso diretto a file e strumenti locali.

6. App mobile

iOS • Android

- Sessioni web → cloud
- Dispatch → task al desktop
- Remote Control → continua una sessione locale
- Avvio e supervisione ovunque.

Schermo piccolo per i diff.

ACCOUNT E PROVIDER | In genere: abbonamento Claude o account Console.
CLI, VS Code e JetBrains supportano anche provider terzi.

INTEGRAZIONI: PIPELINE E STRUMENTI

GitHub Actions / GitLab CI-CD
Review e triage delle issue

GitHub Code Review
Revisione automatica delle pull request

Slack
@Claude: da bug report a pull request

Chrome
Debug di web app nel browser

Channels
Eventi da Telegram, Discord, iMessage e webhook

Agent SDK
Agenti personalizzati

NELLA PRATICA | **CLI + editor:** lavoro quotidiano e didattica • **Desktop:** via grafica • **Web e mobile:** task lunghi e lavoro da fuori | **Fonte:** “Modalità di accesso a Claude Code” • @Antonio • 19 ottobre 2026

Evoluzione del coding con l'AI

Dalla programmazione tradizionale al vibe coding

1 Programmazione manuale o tradizionale

pre-AI

Il codice viene scritto a mano dal programmatore.

Ruolo umano
Scrive tutto il codice.

Ruolo dell'AI
Assente.

2 Completamento automatico del codice

dal 2021

L'IDE suggerisce o completa porzioni di codice basandosi su ciò che è stato digitato.

Ruolo umano
Scrive il codice e accetta o modifica i suggerimenti.

Ruolo dell'AI
Suggerisce completamenti locali.

3 Programmazione a chat
chat coding

dal 2023

Si chiede all'AI di generare o modificare codice in una conversazione turno per turno.

Ruolo umano
Formula richieste e integra le risposte.

Ruolo dell'AI
Genera codice e propone modifiche.

4 Programmazione agentica
agentic coding

inizio 2025

L'agente AI modifica file, usa strumenti, esegue test e lavora in modo più autonomo.

Ruolo umano
Assegna obiettivi e supervisiona.

Ruolo dell'AI
Implementa, testa e perfeziona.

5 Vibe coding

oggi / emergente

Il programmatore delega molto della scrittura del codice all'AI, ma resta attivo: supervisiona, valuta e critica i risultati.

Ruolo umano
Definisce obiettivi, supervisiona e valida.

Ruolo dell'AI
Scrive gran parte del codice e propone soluzioni.

! Il **Vibe Coding** non è una nuova tecnologia separata, ma un nuovo modo di lavorare con gli strumenti AI, soprattutto quelli agentici

Come cresce la delega all'AI

In sintesi: l'evoluzione non elimina il programmatore; cambia il suo ruolo, da autore diretto del codice a supervisore sempre più strategico.

I SEI COMANDI ESSENZIALI

Dall’avvio del progetto alla ripresa del lavoro: una selezione indicata dal file.

1 `/init`

Prepara il progetto

Crea il file guida CLAUDE.md del progetto.

2 `/model`

Sceglie il modello

Cambia il modello e lo salva come predefinito per le nuove sessioni.

3 `/permissions`

Regola l’autonomia

Gestisce le regole allow, ask e deny per gli strumenti.

4 `/compact`

Libera contesto

Riassume la conversazione per ridurre il contesto occupato.

5 `/rewind`

Torna indietro

Riporta conversazione e/o codice a un checkpoint precedente.

6 `/resume`

Riprende il lavoro

Riprende una conversazione precedente per nome, ID o dal selettore.

FLUSSO DA RICORDARE

Progetto → Modello → Permessi → Contesto → Ripristino → Ripresa

CONTESTO, SESSIONE E PROGETTO

Comandi per orientarsi, verificare lo stato e organizzare il lavoro.

/ SESSIONE E STATO

Continuare, verificare e consultare

`/clear [nome]`	Avvia una nuova conversazione con contesto vuoto.
`/context`	Mostra l'uso del contesto e suggerimenti di ottimizzazione.
`/usage`	Costi, limiti del piano e statistiche. Alias: /cost, /stats.
`/help`	Mostra la guida e i comandi disponibili.
`/status`	Versione, modello, account e connettività della sessione.
`/export [file]`	Esporta la conversazione come testo, su file o negli appunti.

/ PROGETTO E CONFIGURAZIONE

Preparare e controllare il lavoro

`/memory`	Modifica CLAUDE.md e gestisce la memoria automatica.
`/add-dir <percorso>`	Aggiunge una directory di lavoro accessibile nella sessione.
`/plan [descrizione]`	Propone un piano di lavoro prima di modificare i file.
`/sandbox`	Isolamento dei comandi: non disponibile su Windows nativo.
`/effort [livello]`	Imposta il livello di ragionamento, incluso auto.
`/config [chiave=valore]`	Gestisce impostazioni; accetta anche chiave=valore.

i Uso: digitare / nella sessione; un comando slash è riconosciuto all'inizio del messaggio.

ESTENSIONI, CODICE E DIAGNOSTICA

Gli strumenti slash per ampliare Claude Code e controllare le modifiche.

/ ESTENSIONI E AGENTI

MCP, plugin, skill, hook, subagent

`/mcp`	Gestisce connessioni, stato e autenticazione dei server MCP.
`/plugin`	Scoperta, installazione e gestione dei plugin.
`/skills`	Elenca le skill, con filtri e costo in token.
`/skill-doctor`	Mostra costo di contesto e uso effettivo delle skill.
`/hooks`	Visualizza gli hook legati agli eventi del ciclo di vita.
`/agents`	Promemoria per creare o gestire i subagent.

/ CODICE E CONTROLLO

Revisioni, attività e diagnostica

`/diff`	Mostra le modifiche nel progetto, anche quelle di Claude.
`/code-review`	Cerca bug nel diff, PR, branch o percorso. Alias: /review.
`/security-review`	Cerca vulnerabilità nelle modifiche del branch.
`/tasks`	Gestisce attività e subagent in background.
`/doctor`	Diagnostica installazione, impostazioni e CLAUDE.md.
`/release-notes`	Changelog interattivo, versione per versione.

i Nel menu / compaiono anche skill personali (/nome-skill) e skill dei plugin (/plugin:nome).

Cosa è l’AI agentica: altro

Le skill degli agenti, spiegate chiaramente

contiene errori, è riferita a OpenAI

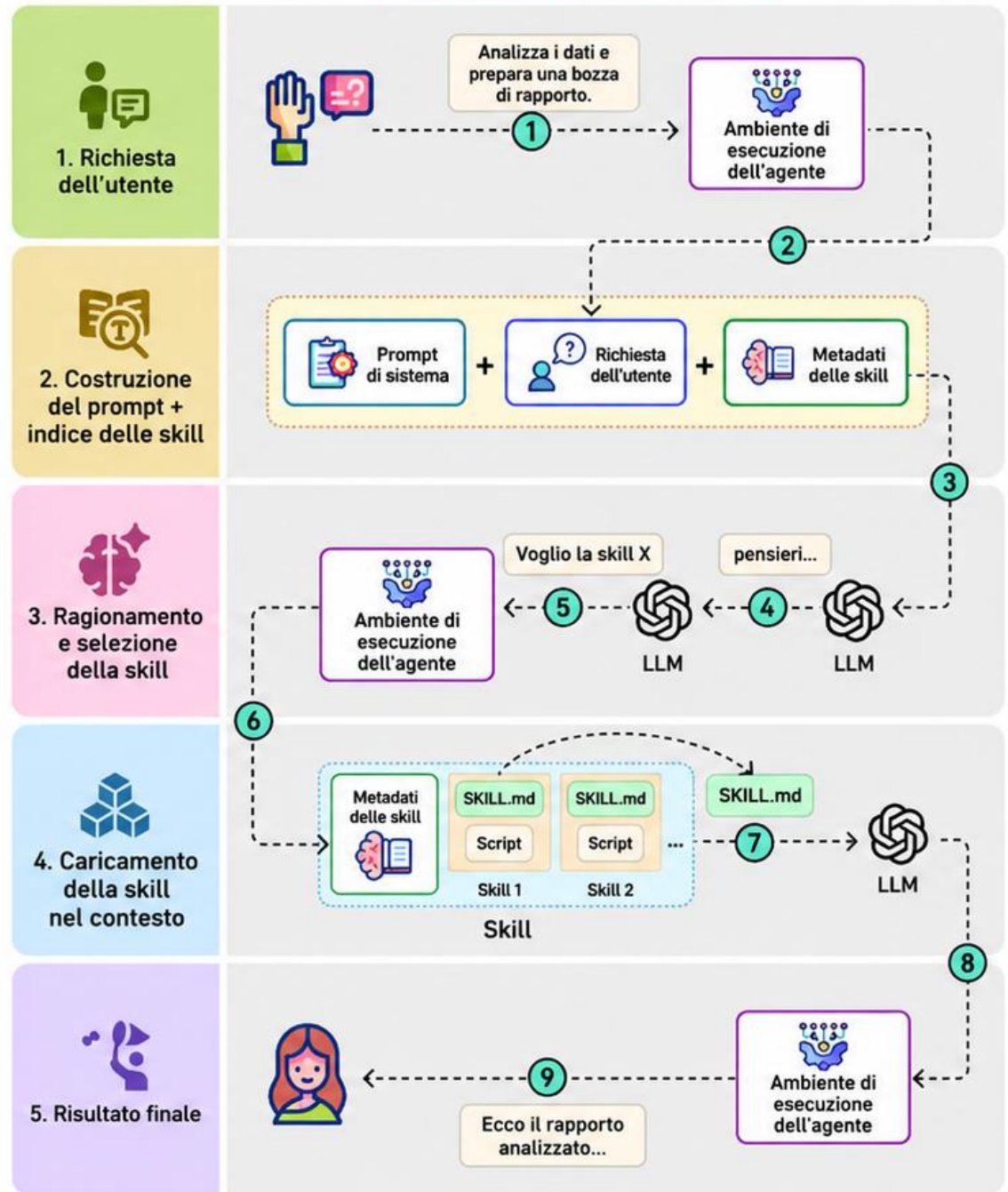
Perché servono le skill degli agenti? I prompt lunghi peggiorano le prestazioni degli agenti. Al posto di un unico prompt enorme, gli agenti mantengono un piccolo catalogo di skill: procedure riutilizzabili con istruzioni chiare, caricate solo quando necessario.

Ecco come funziona il flusso di lavoro delle skill degli agenti:

1. Richiesta dell'utente: un utente invia una richiesta come «Analizza i dati e prepara una bozza di rapporto».
2. Costruzione del prompt + indice delle skill: l'ambiente di esecuzione dell'agente combina la richiesta con i metadati delle skill: un elenco essenziale delle skill disponibili e delle loro brevi descrizioni.
3. Ragionamento e selezione della skill: l'LLM elabora il prompt, ragiona e decide: «Voglio la skill X».
4. Caricamento della skill nel contesto: l'ambiente di esecuzione dell'agente riceve dall'LLM la richiesta di una specifica skill. Quindi carica il file SKILL.md e lo aggiunge al contesto attivo dell'LLM.
5. Risultato finale: l'LLM segue SKILL.md, esegue gli script e genera il rapporto finale.

Le skill degli agenti vengono caricate dinamicamente solo quando necessario, mantenendo ridotto il contesto e coerente il comportamento dell'LLM.

La parola a voi: quali skill trovereste più utili negli agenti?



AI AGENTICA CON CLAUDE CODE · MODULO SKILLS

Agent Skills → Claude Code

Lo stesso flusso in cinque passi: a sinistra il pattern generico delle Agent Skills, a destra il meccanismo con cui lo re
Code.

Mapping della slide precedente su Claude Code (fatta da Claude AI)

ANATOMIA DI SKILL.MD

```
---
name: riassumi-modifiche
description: quando usarla
---
# istruzioni per Claude…
```

PATTERN GENERICO · LE AGENT SKILLS COME CONCETTO		IN CLAUDE CODE · IL MECCANISMO REALE
Richiesta dell'utente Una richiesta arriva all'agente. Per esempio: «Analizza i dati e prepara il report».	01	**Il prompt entra nella sessione** La richiesta viene scritta nella sessione di Claude Code (terminale, VS Code o app desktop). Nel contesto ci sono già il system prompt e le istruzioni di progetto di CLAUDE.md. `claude` `CLAUDE.md`
Prompt + indice delle skill Il runtime — l'infrastruttura che orchestra l'agente — unisce la richiesta a un indice leggero delle skill: solo nome e breve descrizione di ciascuna.	02	**In contesto va solo il «listing»** All'avvio della sessione entrano in contesto soltanto name e description di ogni skill, letti dal frontmatter: l'intestazione YAML in cima a SKILL.md. Il corpo del file resta fuori. `name:` `description:` `frontmatter`
Ragionamento e selezione Il modello valuta la richiesta rispetto all'indice e decide quale skill gli serve: «voglio la Skill X».	03	**Claude sceglie — o si invoca a mano** Claude confronta la richiesta con le description e invoca la skill pertinente; in alternativa la si richiama con /nome-skill. Due campi del frontmatter stabiliscono chi può invocarla. `/nome-skill` `disable-model-invocation` `user-invocable`
Caricamento della skill Il runtime riceve la scelta e carica il file di istruzioni della skill — SKILL.md — nel contesto attivo del modello.	04	**SKILL.md entra in conversazione** Il corpo di SKILL.md si carica solo all'invocazione e resta in contesto anche nei turni successivi. I file di supporto — riferimenti, esempi, script — vengono letti solo se servono davvero. `SKILL.md` `reference.md` `scripts/`
Esecuzione e output Il modello segue le istruzioni della skill, esegue gli script previsti e produce il risultato finale.	05	**Claude esegue con i suoi tool** I passi vengono svolti con i tool della sessione (Bash, Read, Edit…); il campo allowed-tools può pre-approvare i permessi per il turno dell'invocazione. Il risultato torna all'utente. `allowed-tools` `Bash` `Edit`

PROGRESSIVE DISCLOSURE
«rivelazione progressiva»: ogni livello entra in contesto solo quando serve davvero.

LIVELLO 1 · SEMPRE IN CONTESTO
Metadati: name + description

LIVELLO 2 · ALL'INVOCAZIONE
Corpo di SKILL.md

LIVELLO 3 · SOLO SE SERVONO
File di supporto: esempi, script

RISULTATO
Contesto corto → prestazioni migliori e comportamento coerente.

Infografica originale per il corso · terminologia dalla documentazione ufficiale di Claude Code

code.claude.com/docs/en/skills

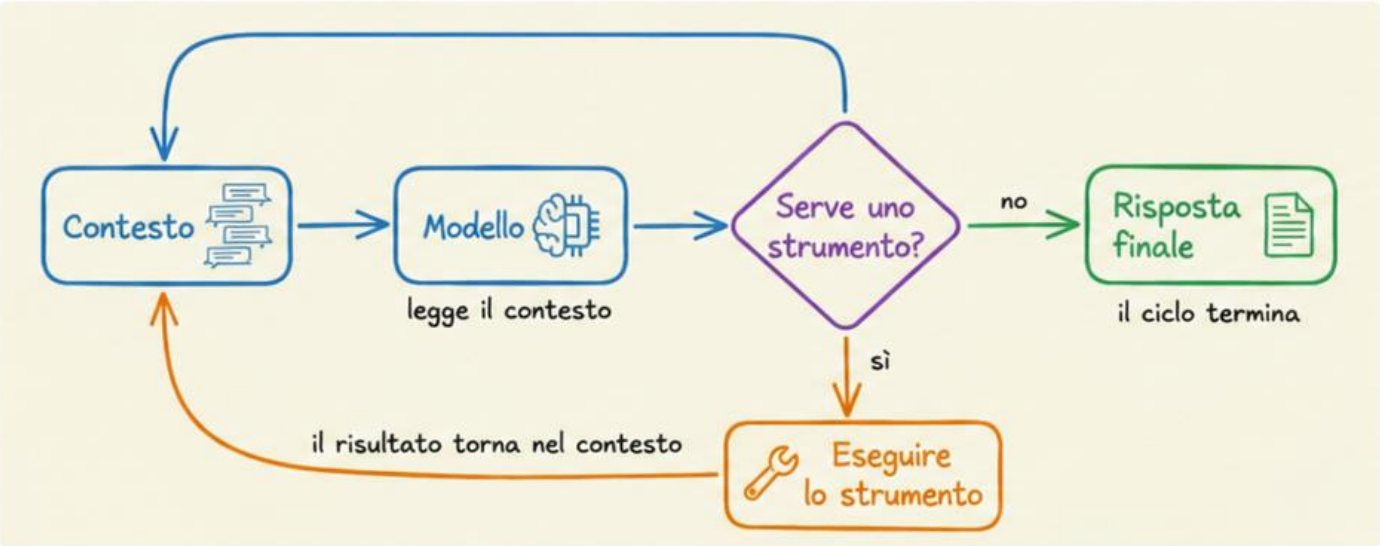
LLM

Ingegneria del prompt, del contesto, dell'harness e del ciclo

Daily Dose of Data Science

3.7.2026

In sostanza, un agente è un ciclo while:



- Il modello viene eseguito
- Richiede chiamate a strumenti
- I risultati degli strumenti tornano nel contesto
- Il modello viene eseguito di nuovo finché non richiede più strumenti

ReAct ha descritto questa forma di ciclo già nel 2022–23, e quasi ogni agente/framework ne esegue un'implementazione simile (**qui abbiamo implementato ReAct da zero in puro Python**).

Ma attorno a questo intero ciclo si articolano quattro livelli di ingegneria:

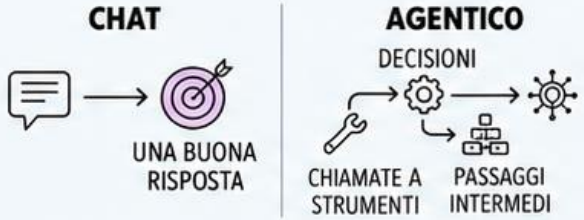
- Ingegneria del prompt
- Ingegneria del contesto
- Ingegneria dell'harness
- Ingegneria del ciclo

MACHINE LEARNING MASTERY — Making developers awesome at machine learning

maggio 2026

PROMPT ENGINEERING PER L'AI AGENTICA

CHAT VS. PROMPTING AGENTICO



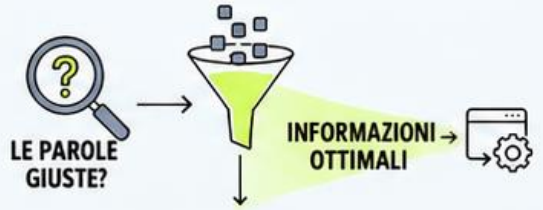
GUIDA UN SISTEMA ATTRAVERSO **MOLTI PASSAGGI**

L'AMBIGUITÀ AL PRIMO PASSO PORTA ALLA DERIVA, SENZA UN ERRORE VISIBILE.

DETERIORAMENTO DEL CONTESTO

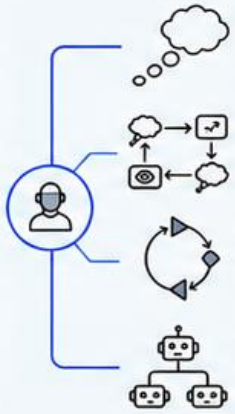
I TOKEN SI ACCUMULANO → PEGGIORANO IL RECUPERO DELLE INFORMAZIONI E LA PRECISIONE DEL RAGIONAMENTO.
OGNI OUTPUT METTE SOTTO PRESSIONE IL BUDGET DI ATTENZIONE.

INGEGNERIA DEL CONTESTO



DALLE «PAROLE GIUSTE?» ALLE «INFORMAZIONI OTTIMALI DI CUI IL MODELLO HA BISOGNO IN OGNI MOMENTO».

ARCHITETTURE DI RAGIONAMENTO



CATENA DI PENSIERO (CoT)
ESPLICITARE I PASSAGGI DI RAGIONAMENTO PRIMA DELL'OUTPUT.

ReAct
(PENSIERO → AZIONE → OSSERVAZIONE → PENSIERO)
BASA IL RAGIONAMENTO SUGLI OUTPUT REALI DEGLI STRUMENTI.

REFLEXION
L'AGENTE VALUTA L'OUTPUT RISPETTO ALL'OBIETTIVO DOPO OGNI PASSAGGIO PRINCIPALE.

SISTEMI MULTIAGENTE
SCHEMA ORCHESTRATORE–ESECUTORI: OGNI ESECUTORE RICEVE SOLO IL CONTESTO NECESSARIO.

ERRORI COMUNI

In questo articolo si vedrà come il prompt engineering cambi radicalmente quando viene applicato ai sistemi di AI agentica, e quali principi e schemi permettano agli agenti di comportarsi in modo affidabile su larga scala.

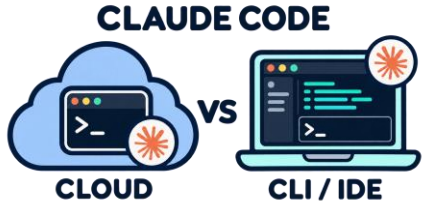
Gli argomenti trattati includono:

- Perchè formulare prompt per gli agenti è diverso dal farlo per i chatbot e che cosa significa, in pratica, ingegneria del contesto.
- I quattro componenti necessari a ogni prompt per agenti, tra cui *prompt* di sistema, strumenti, esempi e gestione dello stato del contesto.
- Le architetture di ragionamento che rendono gli agenti più affidabili, dalla catena di pensiero a ReAct e *Reflexion*.

Prompt engineering per l'AI agentica

di **Shittu Olumide** il 19 maggio 2026 in **Intelligenza artificiale** 1

Claude Code a 360°



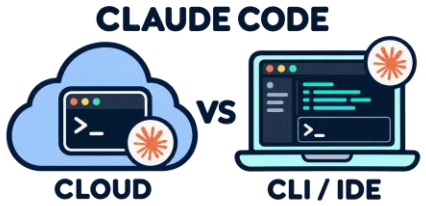
ChatGPT

Claude Code comprende uno strato software attorno al modello, chiamato anche *agentic harness*: gestisce il contesto, rende disponibili gli strumenti e coordina il ciclo di lettura, modifica ed esecuzione delle verifiche. Il modello decide quali azioni proporre; gli strumenti le eseguono nell'ambiente in cui opera Claude Code.

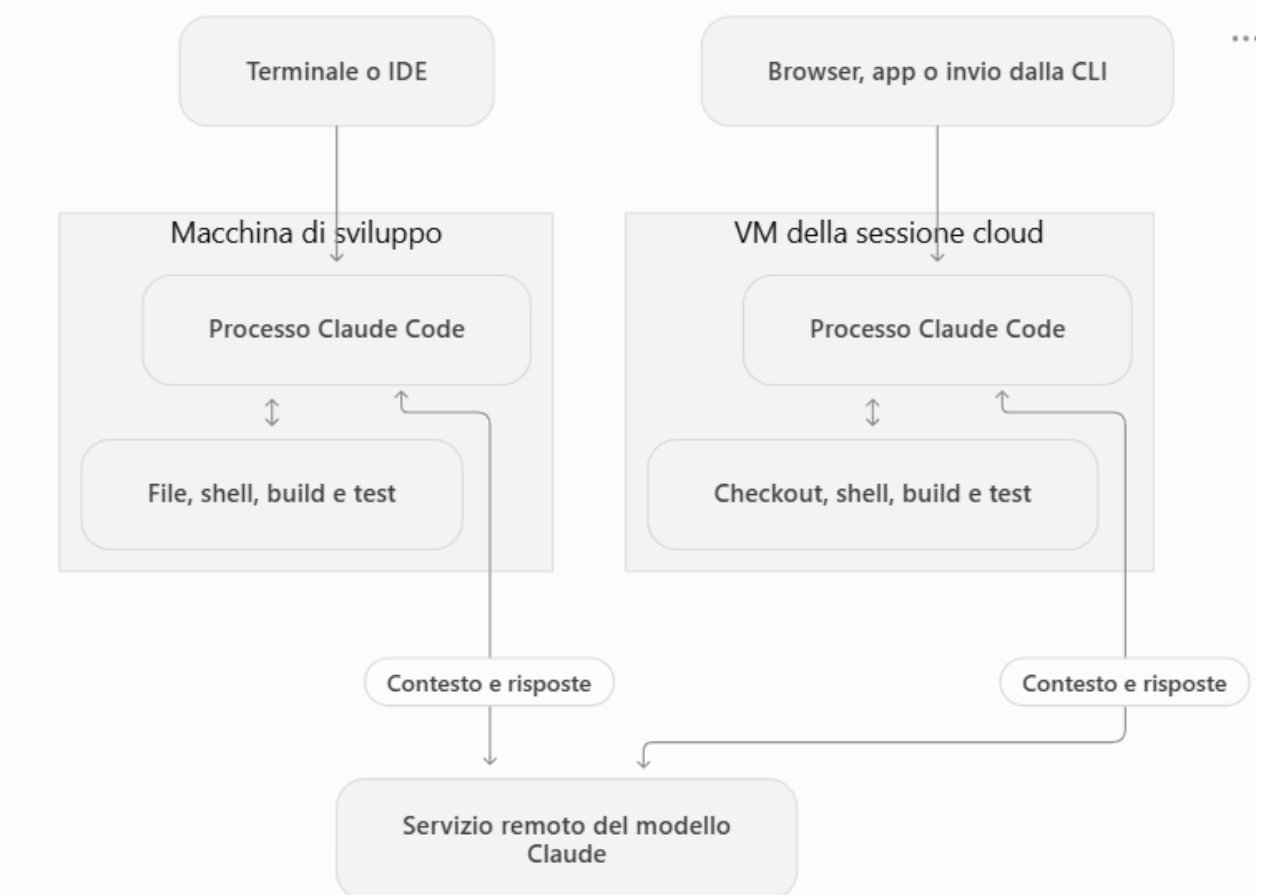
La distinzione utile è questa:

Livello	CLI con esecuzione locale	Sessione cloud ospitata da Anthropic
Interfaccia	Terminale o integrazione con l'IDE	Browser, app o avvio dal terminale
Processo agente	Sulla macchina in cui è avviato Claude Code	Nell'ambiente remoto della sessione
Ambiente di sviluppo	File, shell e strumenti della macchina di sviluppo	Checkout e strumenti dell'ambiente remoto
Inferenza del modello	Servizio remoto configurato	Servizio remoto utilizzato dalla piattaforma

Questa separazione è documentata nei flussi di esecuzione e di trattamento dei dati di Claude Code.

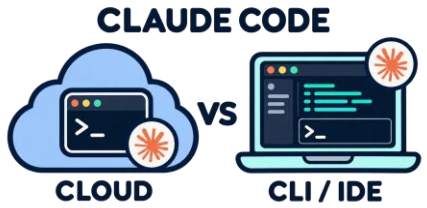


Questa separazione è documentata nei flussi di esecuzione e di trattamento dei dati di Claude Code.



Per esempio, quando viene richiesto di correggere un test, il ciclo può essere: leggere i file pertinenti, inviare il contesto al modello, ricevere una proposta di modifica, applicarla, eseguire il test e usare il risultato per il passo successivo. **Il luogo in cui vengono letti i file ed eseguito il test cambia; il principio del ciclo agentico resta lo stesso.**

Una conseguenza pratica: una workstation più potente può accelerare compilazioni, test e servizi locali. Non accelera direttamente l'inferenza eseguita sui server del fornitore del modello.



Due architetture ulteriori: Remote Control e cloud self-hosted

Remote Control: interfaccia remota, esecuzione sulla macchina di sviluppo

Remote Control permette di guidare dal browser o dal telefono una sessione che continua a usare filesystem, strumenti e configurazione della macchina originaria. Quella macchina e il processo Claude Code devono restare disponibili. La connessione usa richieste HTTPS in uscita; il servizio Anthropic inoltra i messaggi e conserva la trascrizione necessaria alla sincronizzazione.

Il caso d'uso è diverso da quello della VM cloud: **continuare a lavorare sullo stesso ambiente di sviluppo da un'altra interfaccia**.

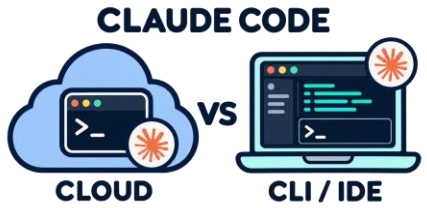
Per esempio, se un'attività dipende da servizi già predisposti su una macchina aziendale, Remote Control consente di mantenere quel contesto senza ricostruirlo altrove.

Self-hosted: sessioni cloud su infrastruttura dell'organizzazione

Anthropic documenta anche ambienti self-hosted, attualmente in beta pubblica per Team ed Enterprise. Il servizio centrale accoda gli incarichi; un runner dell'organizzazione li preleva, prepara il repository e avvia Claude Code sulla propria infrastruttura. Questo permette di eseguire il lavoro dentro la rete aziendale.

È un'architettura interessante quando servono toolchain personalizzate o accessi a servizi interni, mantenendo il modello operativo delle sessioni cloud.

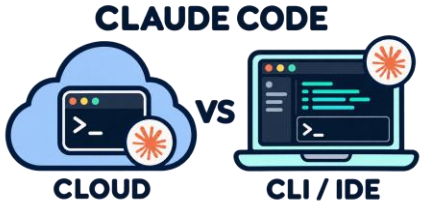
Self-hosted, in questo contesto, riguarda l'esecuzione del lavoro di sviluppo. Non significa ospitare autonomamente il modello Claude: la documentazione specifica che il contenuto delle sessioni raggiunge comunque Anthropic, anche con alcuni instradamenti dell'inferenza verso altri provider.



Quale scegliere nello sviluppo software

Le seguenti sono le mie indicazioni operative, ricavate dalle differenze architetturali descritte:

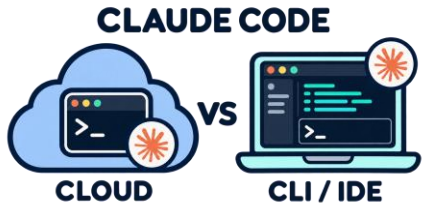
Situazione	Scelta che valuterei per prima	Ragione
Debug frequente insieme allo sviluppatore	**CLI locale / IDE**	Ciclo rapido sullo stesso ambiente e sullo stato corrente del progetto
Problema legato a browser, dati o servizi locali	**Esecuzione locale**	Il contesto necessario è già vicino al processo agente
Correzione delimitata con test riproducibili	**Cloud**	L'incarico può essere eseguito in un ambiente separato
Più attività indipendenti sul repository	**Cloud o CLI con worktree**	Il criterio decisivo è la separabilità del lavoro e la gestione degli ambienti
Attività che richiede una specifica piattaforma nativa	**Ambiente con quella piattaforma**	La validazione deve avvenire dove esistono le dipendenze necessarie
Continuare dal telefono un lavoro sulla stessa macchina	**Remote Control**	Mantiene il contesto operativo originario
Delega remota con accesso a servizi aziendali interni	**Self-hosted, se disponibile e appropriato**	Porta l'esecuzione dentro l'infrastruttura controllata dall'organizzazione



Per un progetto reale può risultare efficace un **flusso misto**: analisi e riproduzione del problema nell'ambiente di sviluppo, delega di un intervento delimitato, revisione del risultato e validazione finale nel contesto di integrazione.

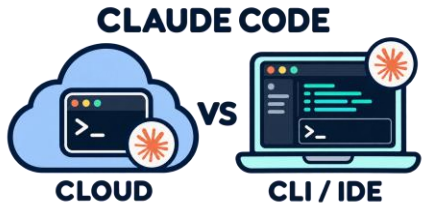
Claude Code supporta anche il rientro di una sessione cloud nel terminale tramite `--teleport`, recuperando branch e conversazione. Il lavoro successivo diventa una continuazione locale distinta e non viene sincronizzato automaticamente nella precedente sessione cloud. code.claude.com

> **Nell'esecuzione locale, Claude Code utilizza l'ambiente di sviluppo della macchina che lo ospita. Nell'esecuzione cloud, utilizza un ambiente remoto predisposto per l'incarico. La scelta incide soprattutto su accesso al codice, strumenti, rete, riproducibilità dei test e modalità di collaborazione.**



Cosa consiglio per i tuoi 4 casi d'uso (e perché)

- **Scrivere codice Python e sviluppare nuove app → Usa la CLI Locale.**
  - **Perché:** In locale Claude opera direttamente sul tuo vero ambiente di lavoro (`.venv` , `uv` , `pytest` , database e container Docker locali) e, soprattutto, può utilizzare il plugin `pyright-lsp` . Con il server LSP attivo in locale, ogni volta che Claude modifica un file `.py` riceve istantaneamente gli errori di tipo o gli import mancanti senza dover eseguire l'interprete, riducendo drasticamente errori, tempo e consumo di token. Se devi allontanarti dalla scrivania, usa `claude --remote-control` per controllare la sessione locale dallo smartphone senza perdere l'ambiente locale.
- **Rifattorizzare basi di codice → Usa la CLI Locale per l'architettura, il Cloud per i lavori meccanici.**
  - **Perché:** Nelle sessioni Cloud, Claude Code **non avvia i language server dei plugin**, quindi lo strumento `LSP` è disattivato. Se devi rinominare simboli, modificare firme di funzioni o ristrutturare classi su molti file, la CLI locale con `pyright-lsp` è indispensabile per trovare i riferimenti esatti invece di affidarsi a semplici ricerche testuali (`grep`). Al contrario, se il refactoring è meccanico e ripetitivo (es. aggiornare dipendenze, correggere lint o scrivere test su 10 moduli indipendenti), lancia più sessioni `claude --cloud` in parallelo: ogni comando crea una VM autonoma che lavora anche a PC spento e che puoi poi riportare in locale con `claude --teleport` .
- **Analizzare codice Python di altri → Usa il Cloud se *Untrusted*, la CLI Locale se *Trusted*.**
  - **Perché (Codice *Untrusted*):** La sandbox integrata nella CLI (`/sandbox` , ovvero il *Sandboxed Bash Tool*) isola **solo** i comandi shell, ma lascia gli strumenti di lettura/scrittura file, i server MCP e gli hook liberi di agire direttamente sul tuo computer. Analizzare un repository sconosciuto in locale ti espone a *prompt injection* o esecuzione di codice malevolo (es. tramite `setup.py` o `pytest`). Le **Cloud Sessions** girano invece in una VM isolata gestita da Anthropic a zero configurazione, dove i tuoi token GitHub restano protetti fuori dal container. *(Se il repo supera il limite di 100 MB del bundle Cloud, analizzalo in locale ma solo dentro una VM completa, un Dev Container o tramite* `@anthropic-ai/sandbox-runtime` *).*
  - **Perché (Codice *Trusted*/Aziendale):** Se il codice è sicuro (es. di un altro team della tua azienda), usa la **CLI Locale in Plan Mode (** `claude --permission-mode plan` **).** Avrai una sessione bloccata in sola lettura ma potenziata da `pyright-lsp` per navigare rapidamente definizioni e dipendenze senza rischiare modifiche accidentali



Gemini Deep Research

Le 5 Best Practice fondamentali (Ottobre 2026)

1. **Installa `pyright-lsp` nella CLI locale:** Installa il binario (`pip install pyright`) e aggiungi il plugin (`/plugin install pyright-lsp@claude-plugins-official`). Aggiunge un costo fisso irrisorio (~146 token a turno) ma dà a Claude la navigazione simbolica del codice e la diagnostica dei tipi in tempo reale.
2. **Sposta i controlli di qualità negli Hooks (a costo zero di token):** Non scrivere nel prompt "ricordati di formattare con ruff e lanciare pytest", perché i prompt sono probabilistici. Configura `.claude/settings.json` con:
   - Un hook `PostToolUse` (filtrato con `"if": "Edit(*.py)|Write(*.py)"`) che esegue `uv run ruff check --fix` e `ruff format` : quando un hook `PostToolUse` termina con codice `0` , gira interamente nell'harness e **consuma 0 token di contesto**.
   - Un hook `Stop` che lancia `uv run pytest -q` e blocca la chiusura del task (`"decision": "block"`) finché tutti i test non passano.
3. **Mantieni `CLAUDE.md` sotto le 200 righe e usa `.claude/rules/` :** Tutto ciò che metti in `CLAUDE.md` viene caricato a ogni sessione; superate le 200 righe, l'aderenza del modello alle regole cala. Metti in `CLAUDE.md` solo i comandi base (es. usare `uv run`) e sposta le regole specifiche (es. convenzioni FastAPI, SQLAlchemy o test) in file separati dentro `.claude/rules/` usando il frontmatter YAML `paths:` (es. `paths: "src/api/**/*.py"`): verranno caricati in memoria **solo su richiesta** quando Claude tocca quei file.
4. **Usa il flusso *Plan Mode* → *Auto Mode*:**
   - Inizia sempre le nuove feature o i refactoring con `Shift+Tab` (o `--permission-mode plan`) usando il modello `opus` (`/model opus`): fai esplorare il codice a Claude e approva/modifica il piano con `Ctrl+G` prima che tocchi qualsiasi file.
   - Per l'implementazione, passa ad `auto` **mode** (diventata la modalità predefinita per Pro, Max e Team da agosto 2026) con `/sandbox` attivo e il modello `sonnet` : un classificatore di sicurezza approverà il lavoro autonomo bloccando azioni distruttive, mentre puoi forzare la richiesta di conferma umana solo per azioni critiche (come `git push`) tramite `permissions.ask` .
5. **Delega le esplorazioni pesanti ai Subagenti:** Se devi analizzare decine di file o fare un refactoring rischioso, usa subagenti personalizzati (`.claude/agents/*.md`). Ogni subagente lavora in una **finestra di contesto separata** (restituendo alla chat principale solo una sintesi pulita) e può essere isolato in una copia temporanea del repo impostando `isolation: worktree` .

MODELLI ANTHROPIC IN CLAUDE CODE

ottobre 2026

Modello attuale · Scelta per una sessione · Preferenza salvata

Esempi di ID sull'API Anthropic

Haiku

`claude-haiku-5-5`

Rapido ed economico.
Per compiti semplici e modifiche ripetitive.

Sonnet

`claude-sonnet-5-5`

Equilibrio tra capacità, velocità e costo.
Per molte attività quotidiane di coding.

Opus

`claude-opus-5-5`

Ragionamenti complessi, architettura, refactoring e debug impegnativo.

Fable

`claude-fable-5-1`

Ragionamenti impegnativi e attività agentiche prolungate.
Può richiedere crediti a consumo, secondo piano e provider.

Nomi brevi: `haiku` · `sonnet` · `opus` · `fable` | La versione effettiva dipende dal provider e dalla configurazione.

VERIFICARE E SCEGLIERE · SESSIONI INTERATTIVE

1 MODELLO ATTUALE

`/status`

Mostra il modello in uso.

`/model`

Apre il selettore: leggere anche avvisi e voci disattivate.

2 SOLO QUESTA SESSIONE

All'avvio dal terminale:

```
claude --model sonnet
```

Oppure, nel selettore `/model`:
scegliere il modello e premere **S**

La preferenza salvata non cambia.

3 PREFERENZA SALVATA

Per le nuove sessioni.
Nel selettore `/model`:
scegliere il modello e premere **Invio**

Oppure: `/model sonnet`

Salva `model` in `~/.claude/settings.json`.

Al riavvio, configurazioni con priorità maggiore possono prevalere.

CAMBIO DI MODELLO | Conserva la conversazione. La prima richiesta può rielaborare il contesto senza la cache precedente.

Fonti: code.claude.com/docs · Model configuration, Commands, Settings, Prompt caching · Verifica: 21 ottobre 2026

LIVELLI DI EFFORT IN CLAUDE CODE

Comprendere i livelli • Verificare il valore • Scegliere la durata

L'effort regola l'impegno del modello nel compito. Livelli più alti possono favorire ragionamenti più approfonditi e un maggiore uso degli strumenti per raccogliere informazioni e verificare il lavoro.
Non modifica le conoscenze apprese durante l'addestramento.

1 • I CINQUE LIVELLI

low
Piccole modifiche e scambi rapidi. Priorità a velocità e risparmio.

medium
Lavoro quotidiano con obiettivi definiti. Equilibrio tra qualità, tempi e consumi.

high
Debug, verifiche e casi limite. Maggiore approfondimento.

xhigh
Problemi complessi e attività agentiche impegnative.

max
Massimo impegno. Valutare costi, benefici e rischio di ragionamento eccessivo.

Livelli disponibili e default dipendono dal modello. Lo stesso nome non indica lo stesso impegno.

2 • VALORE E DURATA NELLA CLI INTERATTIVA

VALORE EFFETTIVO

```
/effort status
```

Mostra il livello realmente in uso.
Visibile anche nell'intestazione.
Con `/effort` si apre lo slider.

SOLO QUESTA SESSIONE

```
claude --effort high
```

Oppure `s` nello slider `/effort`.

`max`: solo sessione, salvo impostazione tramite variabile d'ambiente.

PREFERENZA PER MODELLO

```
/effort high
```

`Invio` nello slider o comando diretto.

Salvataggio in modelSettings.
Salvabili: low · medium · high · xhigh.

auto • VALORE SPECIALE

```
/effort auto
```

Elimina solo la preferenza salvata per il modello attivo.
Altre impostazioni possono restare applicabili.

OVERRIDE DI AMBIENTE

```
CLAUDE_CODE_EFFORT_LEVEL
```

Prevale su `--effort`, `/effort` e preferenze; restano validi i tetti configurati.
Accetta un livello o `auto`. La durata dipende da come viene impostata.

GUIDA RAPIDA

- Partire dal default del modello. Aumentare l'effort se servono più approfondimenti; ridurlo per compiti semplici.
- Se l'effort non basta, valutare un modello più capace. Più effort non garantisce risultati migliori.
- **Ultracode non è un livello di effort.**

Fonti: Claude Code Docs — Model configuration; Environment variables · Claude Platform Docs — Effort · 21/10/2026

quale differenza tra modello potente con effort medio-basso e modello medio con effort alto?

4 minuti fa

Pianificazione della struttura e del tono della risposta finale. >

Domanda centrale quando si configura un sistema con i modelli attuali (GPT-5, o-series, Claude con extended thinking, gpt-oss con `reasoning_effort`, ecc.). Conviene prima chiarire i due termini, perché agiscono su leve diverse:

- **Capacità del modello** ("potente" vs "medio"): è ciò che il modello ha acquisito in addestramento — conoscenza, intuizione, qualità del giudizio *per singolo token generato*. È fissata una volta per tutte: si paga in costo per token e non si può aumentare a runtime.
- **Effort di ragionamento** (reasoning effort / extended thinking): è la quantità di **calcolo al momento dell'inferenza** (*test-time compute*) che il modello spende prima di rispondere, sotto forma di token di ragionamento interno — una "brutta copia" in cui decompone il problema, prova strade, verifica, si corregge. È regolabile a ogni chiamata.

La differenza di fondo è questa: **l'effort amplifica ciò che il modello sa già, non lo sostituisce.** Da qui discendono i comportamenti pratici.

Dove vince il modello potente con effort medio-basso

- Compiti *knowledge-heavy* e di giudizio: scrittura di qualità, sfumature di contesto, codice idiomatico, conoscenza di dominio, interpretazione di istruzioni ambigue. Qui la risposta giusta richiede "sapere", non "calcolare": ragionare a lungo non fa comparire conoscenza assente.
- Compiti **non verificabili**, cioè senza un criterio interno per capire se un tentativo è giusto (una mail ben scritta, una scelta architetturale sensata). Senza verifica, più thinking porta spesso a *overthinking*: il modello rumina e talvolta peggiora la prima intuizione, che nel modello grande era già buona.
- Flussi agentici lunghi (molti passi con tool): ogni piccola decisione sbagliata si propaga, quindi conta la qualità per-passo più che il ragionamento profondo su un singolo passo.
- Latenza percepita: meno token di thinking significa risposta più rapida, a parità di qualità.

Dove vince il modello medio con effort alto

- Compiti **verificabili**: matematica, logica, puzzle, debugging, codice con test. Qui il modello può decomporre, tentare, controllare il risultato e correggersi — e la letteratura sul *test-time compute scaling* (il filone aperto da Snell et al., 2024) mostra che su questi compiti un modello più piccolo con molto calcolo a inferenza può eguagliare o superare un modello molto più grande con poco.
- Costo per token: il modello medio costa tipicamente una frazione. Attenzione però al conto totale — con effort alto i token di thinking possono essere 5–10 volte quelli della risposta, quindi il risparmio si riduce (e la latenza cresce parecchio).
- Privacy/deployment: un modello medio open (es. gpt-oss) con effort alto può girare in locale su hardware come una RTX 5090, cosa impossibile per i modelli di punta.

Regola pratica

- Problema con risposta verificabile e passaggi espliciti → modello medio + effort alto è spesso il punto cost-effective.
- Problema di conoscenza, giudizio, stile, ambiguità → modello potente, anche con effort basso; l'effort alto lì aggiunge poco.
- In caso di dubbio, misurare: fissare un budget di costo per richiesta e confrontare le due configurazioni *a parità di spesa*, non a parità di impostazioni — è il confronto onesto, perché "medio + effort alto" e "potente + effort basso" possono costare quasi uguale.

Un'analogia utile anche in aula: il modello potente a effort basso è l'esperto senior che risponde d'istinto e di solito azzecca; il modello medio a effort alto è il junior diligente che fa i conti con carta e penna, prova e verifica. Il junior diligente risolve l'integrale; non improvvisa la consulenza che richiede vent'anni di mestiere.

I PERMISSION MODE DI CLAUDE CODE

Sei modalità per gestire le conferme prima delle azioni

`manual`

Conferme manuali

Letture nelle cartelle di lavoro.
Per modifiche e comandi non già consentiti chiede conferma.

`acceptEdits`

Modifiche automatiche

Approva modifiche ai file e comandi filesystem comuni nelle cartelle di lavoro.
Include mkdir, mv, cp e rm.
Restano regole ed eccezioni.

`plan`

Prima il piano

Esplora il progetto e prepara un piano.
Normalmente blocca le modifiche fino all'approvazione.*

`auto` DEFAULT

Auto mode · modalità predefinita

Un classificatore valuta le azioni e blocca quelle giudicate rischiose.
Le ask esplicite richiedono conferma.
Riduce i prompt, non garantisce sicurezza.

Disponibilità: modello, versione, provider e policy.

`dontAsk`

Soprattutto per gli script

Esegue le azioni consentite; nega quelle che richiederebbero conferma.

Ideale per **script e automazioni non interattive**, con autorizzazioni definite.

`bypassPermissions`

Salta le conferme ordinarie

Restano deny, ask esplicite e protezioni sulle rimozioni di percorsi critici.
Da usare in VM o container isolati.

Disponibile solo se abilitato all'avvio.

COME SI SELEZIONA

Durante la sessione
`Shift+Tab`

All'avvio
`claude --permission-mode <valore>`

Nei settings
`permissions.defaultMode`

Priorità: flag → settings → default integrato | **dontAsk** escluso dal ciclo · **auto** e bypass: default non accettato dai settings di progetto o locali

DEFAULT INTEGRATI — salvo configurazioni e policy

Terminale e VS Code
Auto da v2.1.283, se disponibiie; altrimenti **Manual**.

-p e Agent SDK
Manual da v2.1.302, se nessun mode è impostato.

In non interattivo, le azioni da approvare sono negate salvo autorizzazione configurata.

* **Plan:** nella CLI interattiva con bypass disponibile, il blocco delle modifiche non è imposto tecnicamente.

Bypass: `--dangerously-skip-permissions` lo attiva; `--allow-dangerously-skip-permissions` lo rende selezionabile.

MODE = comportamento di base · REGOLE = autorizzazioni e divieti · ISOLAMENTO = risorse raggiungibili

Fonti: code.claude.com/docs/en/permission-modes · GitHub anthropics/claude-code v2.1.302 · Verifica: 21/10/2026, 10:29 UTC

La scelta di rendere `auto` la modalità predefinita (prima per Pro, Max e Team ad agosto 2026, poi estesa come punto di partenza generale) nasce da un problema concreto e da un dato empirico che Anthropic ha portato a sostegno.

Il problema: la modalità manuale interrompeva troppo. Le autorizzazioni predefinite di Claude Code erano volutamente conservative: ogni scrittura su file e ogni comando bash chiedeva approvazione. Un default sicuro, ma che impediva di avviare un compito lungo e allontanarsi, perché Claude richiedeva continue conferme umane lungo il percorso. Con compiti sempre più lunghi e agentici, questo attrito spingeva molti sviluppatori verso la scorciatoia sbagliata: disattivare del tutto i controlli con `--dangerously-skip-permissions`, pratica che può portare a esiti pericolosi e distruttivi e che non andrebbe usata fuori da ambienti isolati. claude claude

La soluzione: una via di mezzo sorvegliata. Auto mode è stato concepito come percorso intermedio: consente compiti lunghi con meno interruzioni, introducendo meno rischio rispetto al bypass totale dei permessi. In auto mode le decisioni di autorizzazione vengono prese da un *classificatore*, cioè un secondo modello che esamina ogni azione prima dell'esecuzione e blocca quelle rischiose. claude

Il dato che ha giustificato il cambio di default. Nei test interni di Anthropic, il classificatore di auto mode ha intercettato l'89% dei comandi pericolosi, contro il 14% dell'approvazione manuale umana. È il punto chiave, e anche il più controintuitivo: l'approvazione manuale *sembra* più sicura, ma nella pratica gli utenti approvano i prompt quasi meccanicamente ("prompt fatigue", l'assuefazione ai popup di conferma), lasciando passare la gran parte delle azioni rischiose. Un controllo automatico sistematico, per quanto imperfetto, si è rivelato empiricamente molto più efficace di un controllo umano distratto. gori arkolith

Le cautele mantenute nel passaggio. Il cambio non è stato imposto alla cieca:

- chi aveva già impostato un proprio `defaultMode` è stato interpellato prima di qualsiasi modifica, e resta possibile cambiare modalità con Shift+Tab o fissarne una nei settings; gori
- `defaultMode: "auto"` nei file di impostazioni di progetto viene ignorato, così un repository non può attivare da solo la modalità automatica (difesa contro repository malevoli); gori
- restano bloccate per default classi di azioni come esfiltrazione di dati sensibili, deploy e migrazioni in produzione, cancellazioni di massa su storage cloud, force push e concessioni di permessi; arkolith
- gli amministratori possono disabilitare auto mode a livello di organizzazione — punto rilevante per i contesti bancari dei corsi.

In sintesi: il default è cambiato perché il vecchio default "sicuro sulla carta" produceva nella pratica o frustrazione o bypass totale, mentre il classificatore ha dimostrato numeri di sicurezza effettiva nettamente migliori dell'approvazione manuale.

Fonti:

- Auto mode for Claude Code — blog Anthropic
- Claude Code Auto Mode Becomes the Default — Arkolith
- Annuncio rollout default (gori.me, con thread X di ClaudeDevs)

Claude Code: perché Auto mode è diventato il default

Meno interruzioni, con un controllo automatico delle azioni

IL PROBLEMA
• Troppe conferme

Le richieste manuali interrompevano i compiti lunghi.

La «prompt fatigue» favoriva approvazioni quasi automatiche o il bypass totale dei permessi.

Rischio: controlli disattivati

LA SOLUZIONE
• Auto mode

Un classificatore esamina le azioni prima dell'esecuzione e blocca quelle giudicate rischiose.

Meno interruzioni rispetto al manuale; meno rischio rispetto al bypass totale.

Una via di mezzo sorvegliata

IL DATO RIPORTATO

Comandi pericolosi intercettati nei test interni Anthropic citati nel testo.

Risultati dei test citati, non garanzia di sicurezza.

LE CAUTELE RESTANO

Preferenze rispettate
Chi aveva già scelto un defaultMode mantiene la propria impostazione.

Modalità modificabile
Si può cambiare con Shift+Tab o impostarla nei settings.

Protezione dai repository
L'impostazione auto nei settings di progetto non attiva da sola la modalità.

Controlli sulle azioni sensibili
Restano blocchi per esfiltrazione, deploy e migrazioni in produzione, cancellazioni di massa su storage cloud, force push e concessioni di permessi.

Gli **amministratori** possono disabilitare Auto mode a livello di organizzazione.

Il motivo del cambiamento: un controllo sistematico può essere più efficace di conferme umane date per abitudine.

Sintesi del testo fornito • Fonti citate: Anthropic, Arkolith, gori.me

DAL PLAN MODE ALL'AUTO MODE

Prima si analizza e si approva il piano; poi si implementa sotto controllo

1 PLAN MODE — ANALIZZARE

Attivare con `Shift+Tab` oppure `/plan`

Esempio (non obbligatorio): `/model opus`

```
Windows PowerShell
PS C:\progetto> claude
> Planning mode enabled
> Using model: opus
> Exploring repository...
> Generating plan...
```

Claude esplora il repository, analizza il codice e prepara un piano di implementazione.
Normalmente non modifica il codice prima dell'approvazione.

2 REVISIONE — DECIDERE

`Ctrl+G` → aprire e modificare il piano nell'editor predefinito.

```
plan.md
# Piano di implementazione
1. Obiettivi
2. Modifiche previste
3. File da creare/modificare
4. Test da eseguire
 ...
```

Rivedere, eventualmente modificare il piano e scegliere come procedere:

- **Continua a pianificare** (chiedere altre analisi o modifiche)
- **Approva manualmente** (eseguire passo per passo)
- **Sì, usa Auto Mode** (Claude implementa il piano)

3 AUTO MODE — IMPLEMENTARE

Approvare il piano con **"Sì, usa Auto Mode"**.

Esempio (non obbligatorio): `/model sonnet`

```
Windows PowerShell
PS C:\progetto> claude
> Auto mode enabled
> Using model: sonnet
> Implementing changes...
> Running tests...
> All tests passed ✓
```

Claude modifica il codice, esegue i test e procede in modo iterativo.
Un **classificatore di sicurezza** valuta le azioni: quelle giudicate rischiose possono essere bloccate o richiedere approvazione.

⚠ Per `git push` e altre operazioni sensibili, configurare regole `ask` esplicite (es. `permissions.ask`).

OBIETTIVO: analisi controllata → approvazione umana → sviluppo iterativo

NOTA IMPORTANTE (Windows nativo)

Auto Mode ≠ sandbox.
Il comando `/sandbox` non isola i comandi su Windows nativo.
Per isolamento usare WSL2 o una VM/container adeguatamente configurati.

CONTROLLI FINALI

Controllare `diff` e test prima di commit/push.
Le regole `deny/ask` continuano a valere.

Fonti: Claude Code Docs · Permission modes · Sandbox · 21/10/2026

Claude Code: a cosa serve ultrathink?

Una richiesta di ragionamento più approfondito per il singolo turno

CHE COS'È

Una parola chiave riconosciuta da Claude Code.
Chiede di approfondire il ragionamento sulla richiesta corrente.

Si scrive **ultrathink**, tutto attaccato e senza /

COME SI USA

```
ultrathink: analizza questo bug,
confronta le possibili cause
e proponi una correzione verificabile.
```

Utile per bug difficili, scelte architetturali e analisi complesse.

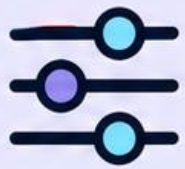
CHE COSA CAMBIA?

Claude Code aggiunge un'istruzione nel contesto per richiedere maggiore approfondimento.

- ✓ Il modello resta lo stesso.
- ✓ Il livello di effort configurato resta invariato.
- ✓ Non imposta automaticamente max.

TRE CONTROLLI DIVERSI

ultrathink
Nel prompt
Richiede maggiore approfondimento per quel turno.



/effort
Impostazione di effort
Modifica il livello di impegno del modello.

Ragionamento on/off
Attiva o disattiva il ragionamento, se il modello lo consente.

Da usare quando serve: per compiti semplici è generalmente superfluo. Un'elaborazione più approfondita può richiedere più tempo e token.

Fonte: documentazione ufficiale Claude Code • 23 ottobre 2026

Claude Code: /init oppure «Crea un README.md»?

Due richieste diverse, due file complementari

/init

COMANDO NATIVO

CLAUDE.md

Come lavorare sul progetto

- **Destinatario principale:** Claude Code
- **Scopo:** fornire istruzioni persistenti per il lavoro sul progetto.
- **Contenuti:** convenzioni, architettura, comandi di build e test, regole operative.
- **Utilizzo:** il `CLAUDE.md` di progetto viene caricato automaticamente nelle sessioni successive.

Esempio di istruzione

```
Prima di completare una modifica, eseguire pytest.
```

Analizza il progetto e genera una prima versione; se il file esiste già, suggerisce miglioramenti.

«Crea un README.md»

PROMPT IN LINGUAGGIO NATURALE

README.md

Che cos'è il progetto e come usarlo

- **Destinatari principali:** utenti e sviluppatori
- **Scopo:** documentare il progetto per chi lo consulta.
- **Contenuti:** descrizione, requisiti, installazione, avvio ed esempi d'uso.
- **Utilizzo:** Claude lo legge quando necessario o quando viene richiamato esplicitamente.

Esempio di documentazione

```
Per avviare l'applicazione, eseguire python main.py.
```

Il contenuto dipende dalla richiesta: si può specificare struttura, pubblico e livello di dettaglio.

COMPLEMENTARI

Il README presenta il progetto alle persone; `CLAUDE.md` guida Claude Code nel lavoro.
Claude può leggere entrambi; `/init` può usare il README come fonte.

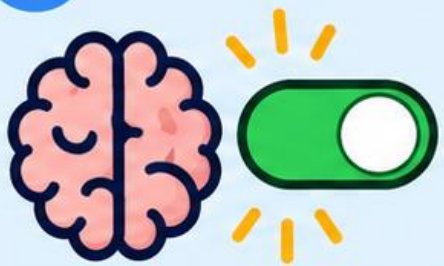
Non occorre eseguire /init a ogni sessione.

Fonte: documentazione ufficiale Claude Code • 22 ottobre 2026

Claude Code e Windows: 4 chiarimenti utili

NEL PROMPT DI CLAUDE CODE

1 Attivare o disattivare il ragionamento



Alt + **T**

Attiva o disattiva il ragionamento esteso, se il modello lo consente.

Nessun effetto sui modelli che ragionano sempre.

2 Cercare nei prompt precedenti

Ctrl + **R**

Avvia la ricerca interattiva nella cronologia dei prompt.

Digitare una parola per ritrovare un prompt precedente.

Il simbolo ~ indica la cartella home dell'utente: per esempio `C:\Users\Utente`

NELLA SHELL DI WINDOWS

3 Cartella utente in cmd

```
echo %USERPROFILE%
```

Mostra il percorso della cartella del profilo utente.

Esempio di output: `C:\Users\Utente`

4 Cartella utente in PowerShell

```
$HOME
```

Mostra il percorso della cartella home.

Esempio di output: `C:\Users\Utente`

Attenzione alla shell: `%USERPROFILE%` è sintassi cmd; in PowerShell usare `$HOME` oppure `$env:USERPROFILE`.

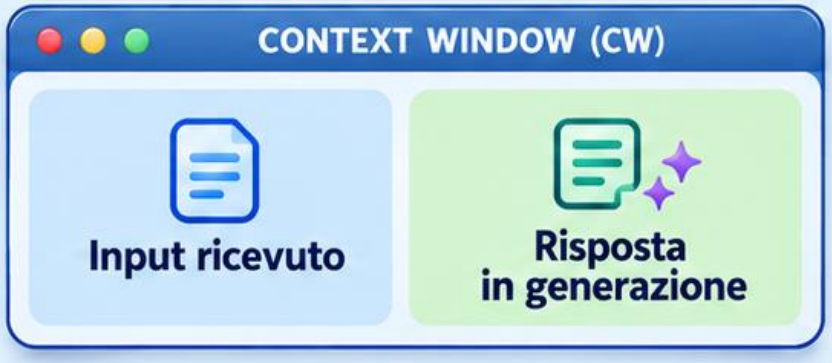
Scorciatoie: documentazione ufficiale Claude Code • Esempi per Windows

CHE COS'È LA CONTEXT WINDOW (CW)

Il contesto di una chiamata e la continuità della conversazione in Claude Code

La CW è lo spazio di lavoro del modello

In una singola chiamata, il modello usa un contesto misurato in token.



L'API Messages è stateless

Il modello non conserva memoria della conversazione tra una chiamata e l'altra.

Claude Code reinvia il contesto corrente:

- definizioni dei tool
- system prompt e CLAUDE.md
- cronologia disponibile + nuovi messaggi

Due chiamate nella stessa sessione

Esempio semplificato: il contesto viene reinviato a ogni chiamata.

CLAUDE CODE — Prepara la richiesta

MODELLO CLAUDE — Genera la risposta

CHIAMATA 1

- Definizioni dei tool
  System prompt • CLAUDE.md
- **Utente:** «Il progetto si chiama Aurora.»

API Messages →

Assistente: «Ricevuto.»

CHIAMATA 2

- Definizioni dei tool
  System prompt • CLAUDE.md
- **Utente:** «Il progetto si chiama Aurora.»
- **Assistente:** «Ricevuto.»
- **Utente:** «Come si chiama il progetto?»

API Messages →

Assistente: «Aurora.»

«Aurora» è presente nel contesto della seconda chiamata.

A ogni chiamata Claude Code reinvia il contesto corrente.
La CW si riempie con nuovi messaggi, risposte e risultati dei tool.

Fonti: Anthropic — Claude Code Docs e Claude API Docs

GUIDA VISIVA · OTTOBRE 2026

La finestra di contesto di Claude

La «memoria di lavoro» dei modelli linguistici: che cos'è, da che cosa dipende la sua dimensione e come la usiamo bene — in chat e in Claude Code.

01 Che cos'è

Tutto ciò che il modello «vede» nel momento in cui genera la risposta: le istruzioni di sistema, i documenti allegati, i messaggi scambiati e la risposta in corso di scrittura. **Ciò che non è nella finestra, per il modello non esiste.**

SI MISURA IN TOKEN — I FRAMMENTI DEL TESTO (␣ = SPAZIO)

`La` `␣fine` `stra` `␣di` `␣conte` `sto`

Un token ≈ 4 caratteri · 1.000 token ≈ 700–750 parole · «500K» e «1M» = 500.000 e 1.000.000 di token.

02 Da che cosa dipende la dimensione

```
finestra = min( modello, tool, piano )
```

MODELLO — il tetto architetturale: il massimo di token che il modello sa gestire.

TOOL — lo stesso modello è servito con finestre diverse a seconda del tool: app Claude (chat), Claude Code, API.

PIANO — Free: limiti variabili con la domanda; a pagamento, alcune finestre da 1M richiedono i crediti di utilizzo (pagamento a consumo).

ESEMPIO — CLAUDE FABLE 5, PIANI A PAGAMENTO

La finestra è una scelta di prodotto, non una costante fisica: servire più contesto costa.

03 Che cosa la occupa

- **Prompt di sistema** — istruzioni di base del prodotto
- **Strumenti e connettori** — caricati anche se non usati
- **Istruzioni di progetto** — rilette a ogni messaggio
- **Documenti e file** — allegati e materiale caricato
- **Conversazione** — messaggi e risposta in corso
- **Spazio libero** — il margine per lavorare

Nei Projects i documenti passano dalla RAG (recupero selettivo): nella finestra entra solo la parte che serve alla domanda, non l'intero archivio.

04 Quando si riempie: la gestione automatica

1. **La chat si avvicina al limite** della finestra di contesto.
2. **Claude riassume i messaggi più vecchi** («sta organizzando i pensieri») per liberare spazio.
3. **La conversazione prosegue** quasi senza limiti, nella maggior parte dei casi.

Piano a pagamento + «Code execution» attiva

La cronologia completa resta consultabile

Il riassunto consuma quota di utilizzo

05 Tre «memorie» da non confondere

Finestra di contesto — il banco di lavoro: ciò che il modello vede adesso. — VIVE IN UNA CHAT

Gestione automatica — gli appunti di sgombero: comprime il passato della stessa chat. — VIVE IN UNA CHAT

Memoria (Memory) — il quaderno nel cassetto: fatti durevoli, a ogni nuova chat. — SEGUE L'ACCOUNT

In Claude Code: **CLAUDE.md** = ciò che scriviamo noi · **auto memory** = ciò che scrive Claude · `/memory` le gestisce entrambe

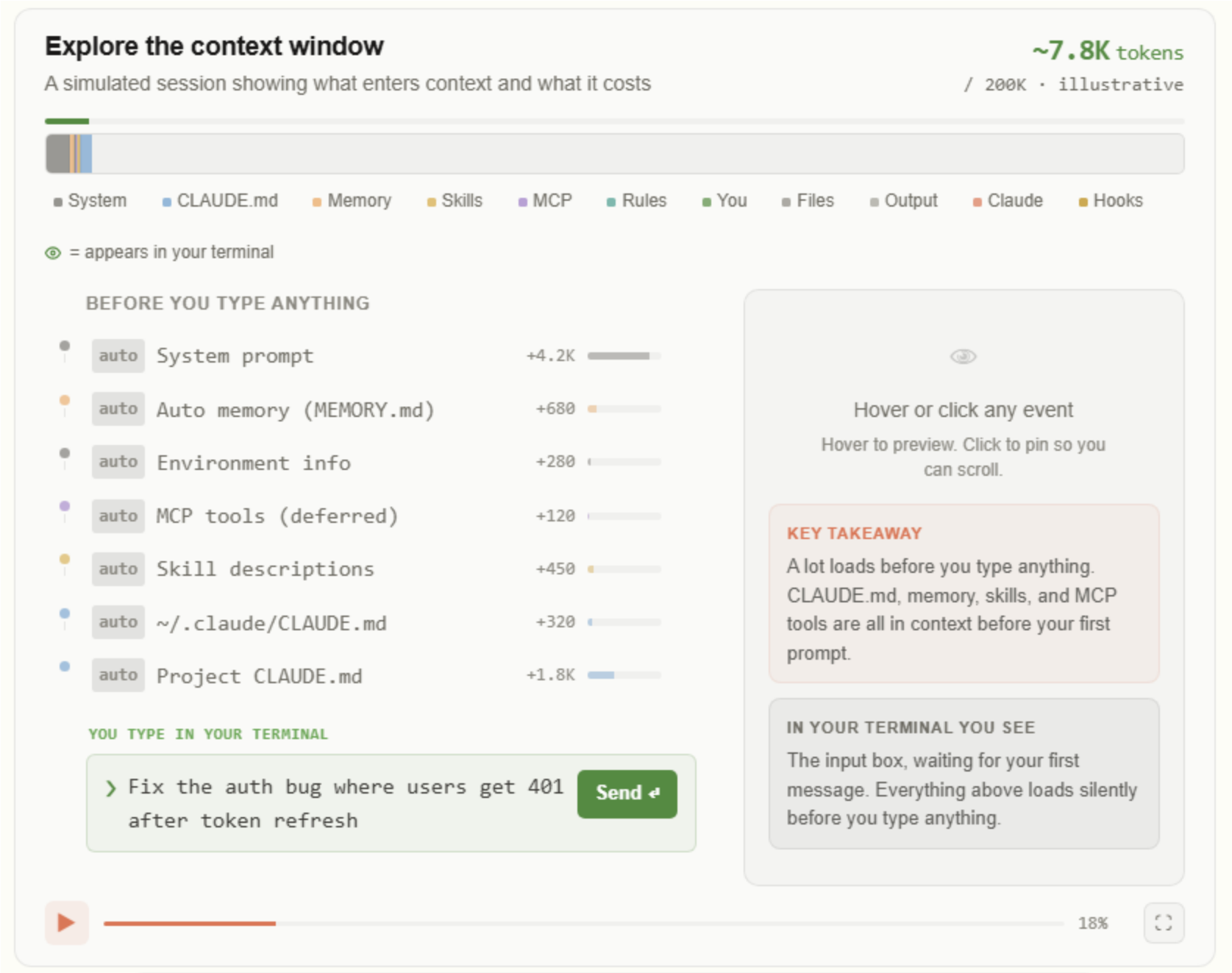
06 Regole pratiche (e il «context rot»)

Il degrado da contesto è reale; le soglie fisse (20%, 40%…) no: **conta che cosa c'è nella finestra, non solo quanto è piena.**

`/clear`	Quando cambiamo compito: il contesto precedente è rumore.
`/compact`	Deliberato, ai punti di svolta: mai aspettare quello automatico vicino al limite.
`Igiene`	Pochi connettori attivi, istruzioni concise, documenti nei Projects.
`Sintomi`	Compattiamo quando il modello rilegge file già letti, reintroduce bug corretti o dimentica decisioni prese.

Fonti: support.claude.com («How large is the context window on paid Claude plans?») · code.claude.com/docs · Ricerca: Lost in the Middle (Liu et al., 2024), Context Rot (Chroma Research). Infografica didattica — ottobre 2026.

link alla timeline interattiva



la dimensione della finestra di contesto dipende dunque non solo dal modello, dal tool ma anche dal piano??

Cercato sul web, letta una pagina ›

Sì, esatto — anche se con una precisazione: tra i vari piani *a pagamento* le dimensioni sono quasi sempre identiche; il piano incide soprattutto ai margini. La dimensione effettiva della finestra di contesto è determinata dall'incrocio di tre fattori:

1. Il modello — il limite architetturale: quanti token il modello è stato addestrato e ottimizzato a gestire (es. 1M per Fable 5.1, 500K per Fable 5 in chat). Oltre questo non si va mai.

2. La superficie d'uso (il "tool") — lo stesso modello viene servito con finestre diverse a seconda del prodotto: le tabelle che hai fotografato mostrano Fable 5 a 500K in chat ma 1M in Claude Code. Anche l'API ha le sue regole (per alcuni modelli la finestra da 1M è un'opzione da richiedere esplicitamente). E persino la piattaforma cloud conta: lo stesso modello su un cloud di terze parti può avere una finestra ridotta rispetto all'API diretta di Anthropic.

3. Il piano — qui gli effetti sono tre:

- **Free vs a pagamento**: la differenza grande. L'articolo di supporto sulla lunghezza massima dei prompt dice esplicitamente che sul piano gratuito i limiti **variano con la domanda** — nei momenti di picco la finestra disponibile può ridursi. Le tabelle da 500K/1M valgono solo per i piani a pagamento.
- **Tra i piani a pagamento** (Pro, Max, Team, Enterprise): le dimensioni nominali sono le stesse, ma cambiano le **condizioni di accesso**. Esempi dalle tue stesse tabelle: in Claude Code, gli utenti Pro devono abilitare i crediti di utilizzo (usage credits, il pagamento a consumo oltre la quota del piano) per avere la finestra da 1M sui modelli Opus; per Sonnet 4.6 i crediti servono su tutti i piani tranne gli Enterprise a consumo. Quindi la finestra "esiste" per tutti, ma su alcuni piani è dietro una condizione di fatturazione.
- **Sull'API**: l'accesso alla finestra da 1M su alcuni modelli dipende dal *usage tier* (il livello di spesa dell'account) — anche qui, stessa logica: il limite tecnico è del modello, ma chi può usarlo per intero lo decide il piano.

Quindi la formula completa per un corso sarebbe: **finestra effettiva = min(limite del modello, configurazione della superficie, ciò che il piano/fatturazione sblocca)** — più la variabile "domanda" sul piano gratuito. È un buon esempio del fatto che la finestra di contesto non è una costante fisica del modello ma una scelta di prodotto: servire 1M di token a ogni richiesta costa (memoria GPU e calcolo crescono con la lunghezza del contesto), quindi chi paga di più, o paga a consumo, ottiene la configurazione più generosa.

link alla pagina della documentazione di Claude

Ottobre 2026

How large is the context window on paid Claude plans?

Updated over a week ago

Copy for LLM

This article explains how large the context window is on paid Claude plans (Pro, Max, Team, Enterprise) when you chat with Claude, or use Claude Code or Claude Cowork.

Chatting with Claude

Model	Context window
Claude Fable 5.1	1M tokens
Claude Fable 5	500K tokens
Claude Opus 5.5	1M tokens
Claude Opus 5	1M tokens
Claude Opus 4.8	500K tokens
Claude Opus 4.7	500K tokens
Claude Opus 4.6	500K tokens
Claude Sonnet 5.5	1M tokens
Claude Sonnet 5	1M tokens
Claude Sonnet 4.6	500K tokens
Claude Haiku 5.5	1M tokens

Claude Code

Claude Code Docs

Ottobre 2026

Model	Context window
Claude Fable 5.1	1M tokens
Claude Fable 5	1M tokens
Claude Opus 5.5	1M tokens
Claude Opus 5	1M tokens
Claude Opus 4.8	1M tokens
Claude Opus 4.7	1M tokens
Claude Opus 4.6	1M tokens **Note:** 1M context window available by selecting `claude-opus-4-6[1m]` with `/model`; on Pro, usage credits must be enabled to access
Claude Sonnet 5.5	1M tokens
Claude Sonnet 5	1M tokens
Claude Sonnet 4.6	1M tokens **Note:** 1M context window available by selecting `claude-sonnet-4-6[1m]` with `/model`; usage credits must be enabled to access (except for usage-based Enterprise plans)
Claude Haiku 5.5	1M tokens

Le due tabelle (la slide precedente e questa) non sono contraddittorie: descrivono due prodotti diversi. La prima tabella vale per le chat nell'app Claude (web/desktop/mobile), la seconda per Claude Code, e **lo stesso modello può avere una finestra di contesto diversa a seconda di dove viene usato**.

In sintesi, dall'articolo ufficiale:

- **Nell'app Claude (chat)**: Opus 5 e Sonnet 5 supportano una finestra di contesto da 1M di token su tutti i piani a pagamento, mentre Opus 4.8, 4.7, 4.6 e Sonnet 4.6 si fermano a 500K — da qui i 500K che vedi anche per Fable 5 nella prima tabella. claude
- **In Claude Code**: con un piano Pro, Max, Team o Enterprise, Sonnet 5, Fable 5, Opus 5, 4.8, 4.7 e 4.6 supportano tutti 1M di token; gli utenti Pro devono però abilitare i crediti di utilizzo per avere la finestra da 1M sui modelli Opus, e Sonnet 4.6 richiede i crediti di utilizzo su tutti i piani tranne gli Enterprise a consumo. claude

COSA CONTIENE LA CONTEXT WINDOW (CW)

Cosa entra all'avvio e cosa si aggiunge durante il lavoro in Claude Code

GIÀ ALL'AVVIO

Istruzioni di base
System prompt e informazioni sull'ambiente.

CLAUDE.md globale e di progetto
`Globale: ~/.claude/CLAUDE.md`
`Progetto: ./CLAUDE.md o ./.claude/CLAUDE.md`

Auto memory
Parte iniziale di MEMORY.md, se presente.

Skill, tool nativi e tool MCP
Descrizioni delle skill, definizioni dei tool nativi e nomi dei tool MCP.
Con Tool Search, le definizioni MCP complete arrivano su richiesta.

7.850 token già prima del primo prompt
Nella simulazione ufficiale; il valore reale varia con la configurazione.

DURANTE IL LAVORO

Le letture dei file spesso dominano
Ogni Read aggiunge alla CW tutto il contenuto restituito, anche se nel terminale si vede solo una riga.
Il file intero, oppure la porzione richiesta.

Conversazione
Prompt, risposte e chiamate ai tool.

Risultati dei tool, anche MCP
Ricerche, test, errori, log e altri output.

Contesto aggiunto quando serve
Istruzioni delle skill usate, regole pertinenti e contesto degli hook.

COME SI RIEMPIE

Schema illustrativo

Ogni nuovo contenuto aggiunge token.

IL TRUCCO DEI SUBAGENT

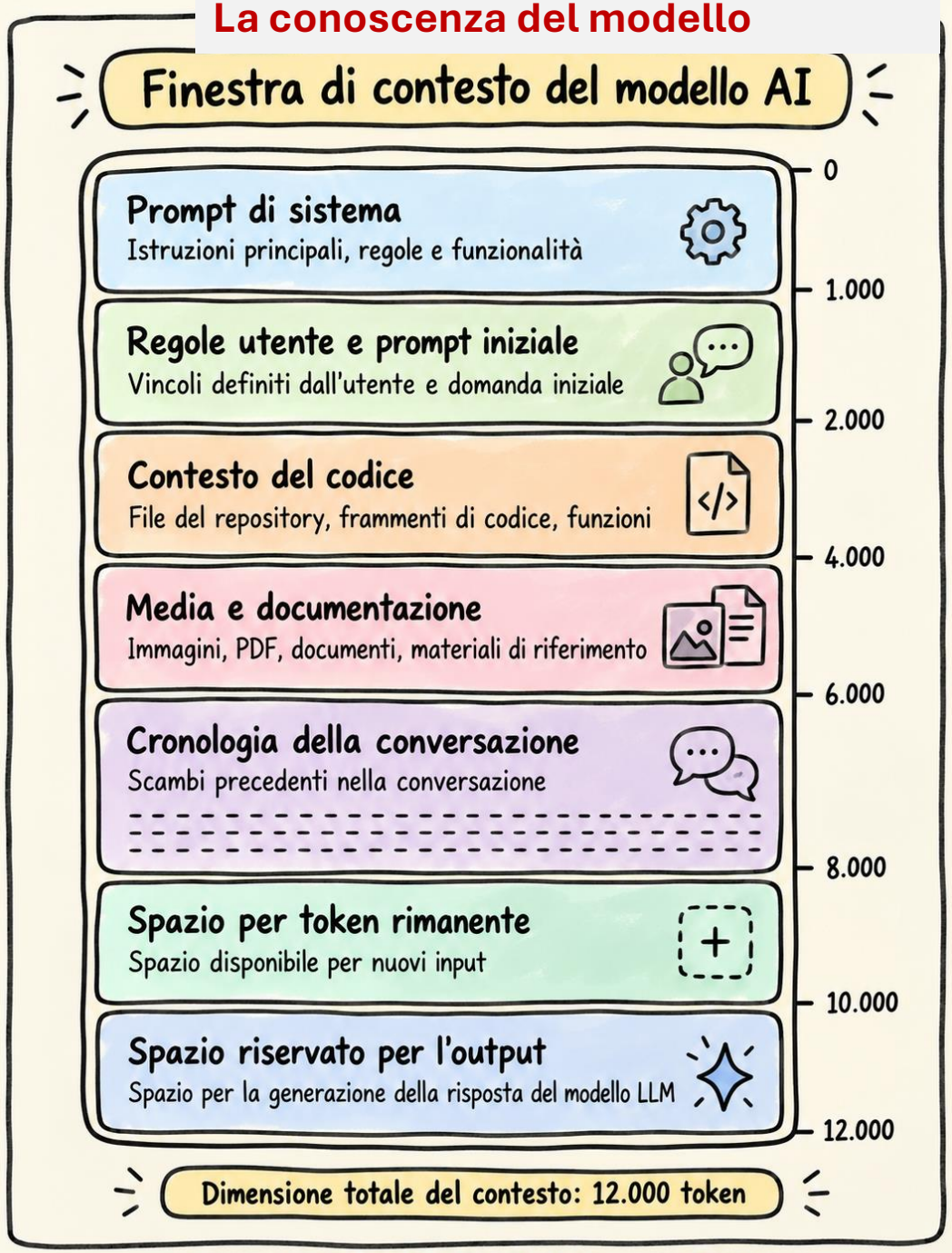
Una CW separata: carica tool MCP e, quando previsti, CLAUDE.md.

Al principale torna solo la risposta finale + un piccolo trailer di metadati.
Esempio dei docs. I CLAUDE.md caricati dipendono dal tipo e dalla configurazione.

`/context` → mostra l'occupazione per categoria

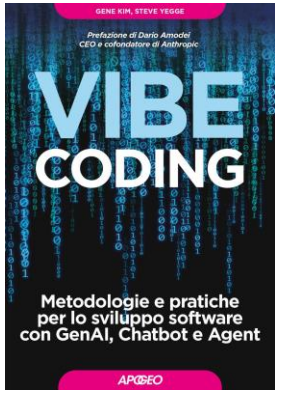
Fonti: Anthropic — Claude Code Docs e simulazione ufficiale

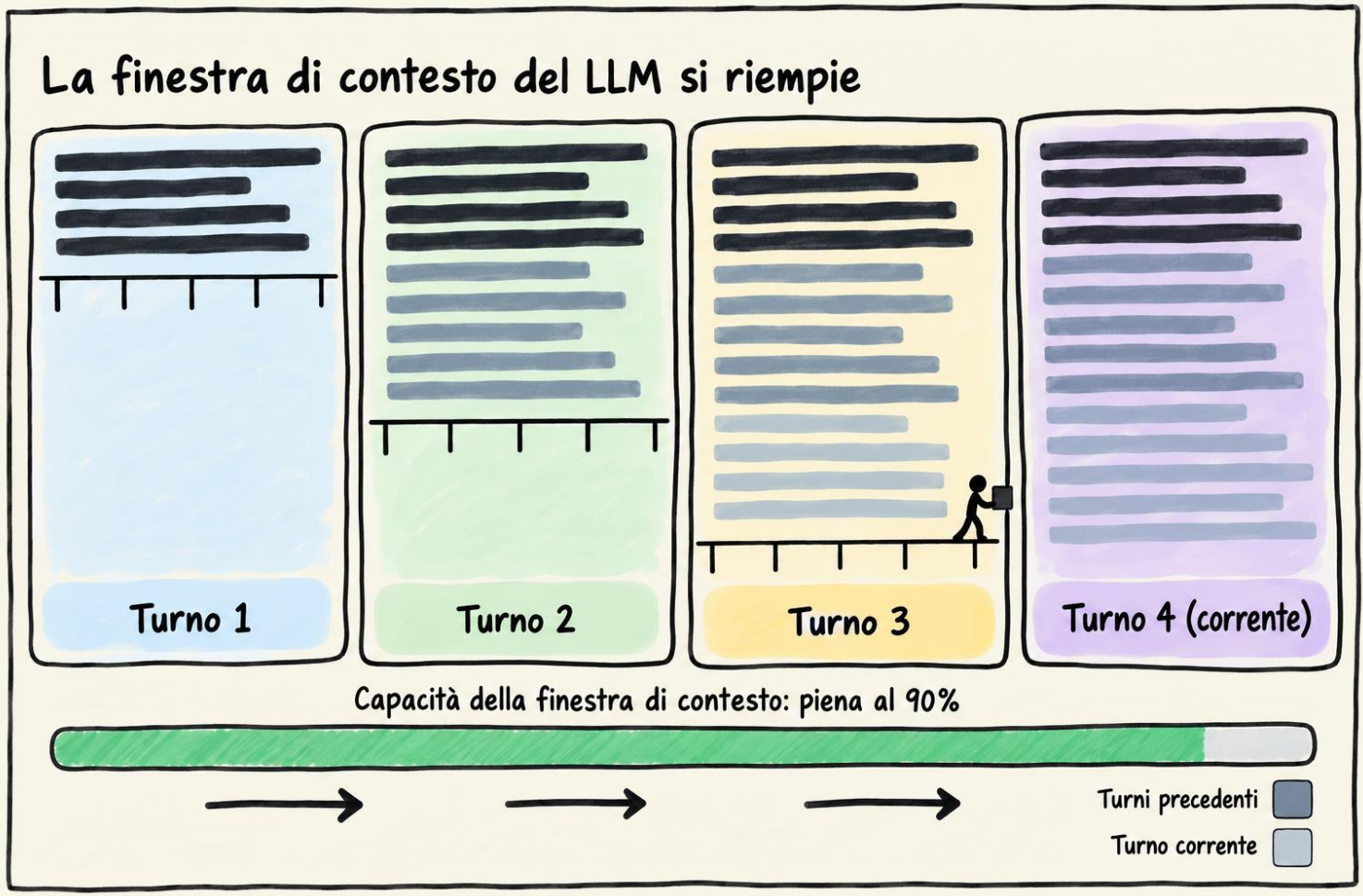
La conoscenza del modello



Schema di una finestra di contesto di un modello AI.

I modelli di AI NON hanno memoria (sono *stateless*): quindi dobbiamo ogni volta rispedirgli tutta la finestra di contesto, che viaggia avanti e indietro





COMPATTAZIONE DELLA CONTEXT WINDOW (CW)

Un riassunto per continuare la sessione: funzionamento, controlli e rischi in Claude Code

CHE COS'È

Claude Code riassume la cronologia e usa la sintesi nel contesto attivo, liberando token per proseguire.

Schema semplificato

La sessione continua con meno cronologia dettagliata nella CW.

AUTOMATICA, MA DISATTIVABILE

Attiva di default; la soglia dipende dal modello e dalla configurazione.

Con OFF: usare /compact manualmente prima del limite.

IL RISCHIO

La CW può raggiungere il limite: le nuove richieste si fermano.

```
>_ Context limit reached
```

Per continuare: **/compact** oppure **/clear**.

CHE COSA RESTA E COSA PUÒ PERDERSI

Nel riassunto
Cerca di conservare obiettivi, decisioni, stato del lavoro e attività aperte.

Istruzioni persistenti
System prompt ancora valido; CLAUDE.md principali e auto memory ricaricati.

Possibili perdite
Dettagli, vecchi output e istruzioni presenti solo nella cronologia possono sfuggire.

Per guidare anche la sintsi automatica: sezione «Compact Instructions» in CLAUDE.md.

COMANDI PER GESTIRLA

`/compact [istruzioni]`	Riassume subito; le istruzioni indicano le priorità. `/compact conserva decisioni, vincoli e attività aperte`
`/autocompact`	Regola la soglia automatica (v2.1.221+). `L'attivazione ON/OFF si gestisce in /config.`
`/rewind → Summarize…`	Riassume una parte della conversazione.
`/clear`	Avvia una nuova conversazione.
`/context`	Mostra l'occupazione della CW.

Fonti: Anthropic — Claude Code Docs

Come leggere l'output di /context in Claude Code

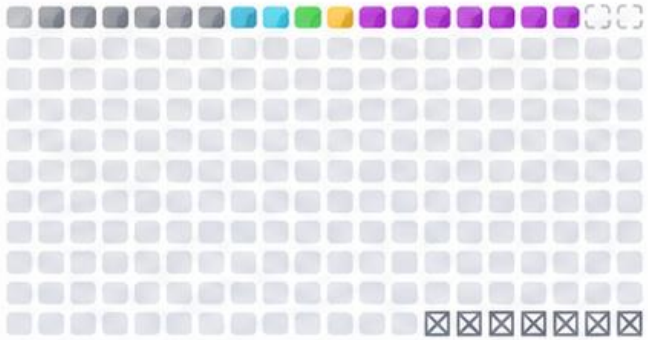
Guida rapida alla lettura del consumo di contesto, dei token e delle componenti della sessione

Più consapevolezza del contesto, conversazioni migliori

Output di esempio: /context

Output reale semplificato e formattato

```
/context
└ Context Usage
```



```
Auto-compact window: 1m tokens

MCP tools · /mcp (loaded on-demand)
54 tools · 644 tokens
```

Opus 5.5
`claude-opus-5-5`
`79.3k/1m tokens (8%)`

Estimated usage by category

- System prompt: 2k tokens (0.2%)
- System tools: 36.9k tokens (3.7%)
- MCP tools: 644 tokens (0.1%)
- MCP server instructions: 826 tokens (0.1%)
- Skills: 2.8k tokens (0.3%)
- Messages: 36.2k tokens (3.6%)
- Free space: 887.7k tokens (88.8%)
- ☒ Autocompact buffer: 33k tokens (3.3%)

Che cosa significa

Model / finestra di contesto	Indica il modello attivo e la capienza massima disponibile.
79.3k / 1m tokens	Quanti token della finestra sono già occupati in questa sessione.
8%	Percentuale di contesto usata finora.
Estimated usage by category	Scomposizione stimata del consumo per tipologia di contenuto.
Free space	Spazio ancora disponibile prima di saturare la finestra.
Autocompact buffer	Margine riservato per permettere il compact automatico prima di arrivare al limite.
Auto-compact window	Soglia/ampiezza usata dal meccanismo di compattazione del contesto.
MCP tools	Strumenti esterni disponibili viaa MCP; qui sono caricati on-demand.

Come interpretare le categorie

System prompt	Istruzioni di base del sistema.
System tools	Descrizioni degli strumenti interni disponibili al modello.
MCP tools	Descrizioni sintetiche dei tool MCP esposti.
MCP server instructions	Istruzioni aggiuntive provenienti dai server MCP.
Skills	Skill caricate o disponibili.
Messages	Cronologia della conversazione e contenuti scambiati.

Lettura pratica dell'esempio

1. La sessione è molto lontana dal limite: è usato solo l'**8%** del contesto.
2. I contributi principali qui sono `System tools` e **Messages**
3. Lo spazio libero è ancora molto ampio (**88.8%**).
4. L'autocompact aiuta a evitare saturazione e perdita improvvisa di contesto.
5. I tool MCP ci sono, ma il loro peso attuale è minimo (**644 token**).

Suggerimento hands-on: controlla `/context` durante attività lunghe o con molti tool per capire quando il contesto sta crescendo troppo.

Meno sorprese, più produttività

LIMITI E DEGRADAZIONE DELLA CONTEXT WINDOW (CW)

Quanto contesto entra e quanto bene Claude riesce a usarlo

CAPIENZA E AFFIDABILITÀ

Capienza nominale

È finita e dipende dal modello e dalla configurazione.

Esempi di capienza

200K token | 1M token

Uso affidabile del contesto

Recuperare dettagli, collegare informazioni lontane e rispettare i vincoli.

La qualità può calare prima che la CW sia piena.

IL DEGRADO: CONTEXT ROT

La qualità del recupero e del ragionamento può diminuire con l'aumento del contesto.

Il rumore può amplificare il problema.

- Testo irrilevante
- Log ridondanti
- Istruzioni contrastanti

Possibili segnali da verificare

- Vincoli trascurati
- Dettagli confusi
- Tentativi falliti ripetuti

L'effetto dipende da modello, compito e contenuti.

LOST IN THE MIDDLE

Schema illustrativo

Anche la posizione dell'informazione può influire.

In alcuni test, i dati al centro vengono recuperati peggio di quelli all'inizio o alla fine.

Effetto osservato in specifici test, non una regola universale.

COME RIDURRE IL RISCHIO

Separare i compiti
Evitare di mescolare attività non correlate.

generiche

Selezionare il contesto
File pertinenti e output essenziali, mantenendo le informazioni necessarie.

Rendere chiari i vincoli
Obiettivo, stato attuale e decisioni valide devono essere espliciti.

Verificare il risultato
Usare test e riscontri sui file correnti.

Un buon contesto contiene tutte le informazioni necessarie, con poco rumore.

Fonti: Anthropic · Liu et al., Lost in the Middle · Chroma, Context Rot

LIMITI E DEGRADAZIONE DELLA CONTEXT WINDOW (CW)

Capienza, qualità e gestione del contesto in Claude Code

LIMITI, RUMORE E POSIZIONE

Capienza nominale:
finita; dipende da modello e configurazione. Esempi: 200K o 1M token.

Context rot:
aumentando il contesto, recupero e ragionamento possono peggiorare. Testi irrilevanti, log ridondanti e istruzioni contrastanti aumentano il rumore.

Possibili segnali:
vincoli trascurati, dettagli confusi, tentativi falliti ripetuti.

Lost in the middle:
in alcuni test, i dati al centro sono recuperati peggio. Non è una regola universale.

Selezionare contenuti pertinenti e verificare sui file correnti.

REGOLARE L'AUTOMATISMO

Claude Code ≥ 2.1.221

`/autocompact 500k` Imposta una finestra di auto-compattazione di 500K token.

- Valori ammessi: 100K–1M token.
- Formati: 500000, 500k, 1M.
- Numeri da 100 a 1000 = migliaia: 500 = 500K.

Il valore effettivo è limitato dalla CW del modello:
500k non amplia una CW da 200K.

`/autocompact auto` Ripristina la finestra prevista per il modello.

ON/OFF: `/config` → **Auto-compact**

COMPATTARE O RIPARTIRE

`/compact focus on <cosa>`
Orienta il riassunto sulle informazioni da conservare.

Esempio: `/compact focus on bug di login e test`
focus on è un'istruzione libera, non un'opzione speciale.

`/clear` Per un nuovo lavoro non correlato: ripartire puliti, senza trascinare un riassunto approssimativo.

COMPATTARE A UN CONFINE PULITO
Quando possibile, a task concluso e non a metà del debug.
Se occorre compattare durante il debug, fissare prima ipotesi, prove, risultati e prossimo passo.

COSA SOPRAVVIVE DOPO /compact

- **Cronologia** → riassunto; dettagli e output integrali possono perdersi.
- **Restano validi** → system prompt e output style.
- **Ricaricati** → CLAUDE.md globale/progetto, regole generali, auto memory e piano. Git aggiornato, se presente.
- **Su richiesta** → regole sui percorsi e CLAUDE.md annidati.
- **File** → fino a 5 riletti, priorità ai modificati più recenti; oltre 5K token, solo riferimento.
- **Skill usate** → max 5K token/skill, 25K totali; le più vecchie possono uscire.
- **Hook** → contesto precedente riassunto; SessionStart(compact) rieseguiti.
- **Background** → comandi e subagent continuano.

Più finestra non garantisce più qualità.
200K token curati possono funzionare meglio di 1M lasciati accumulare.

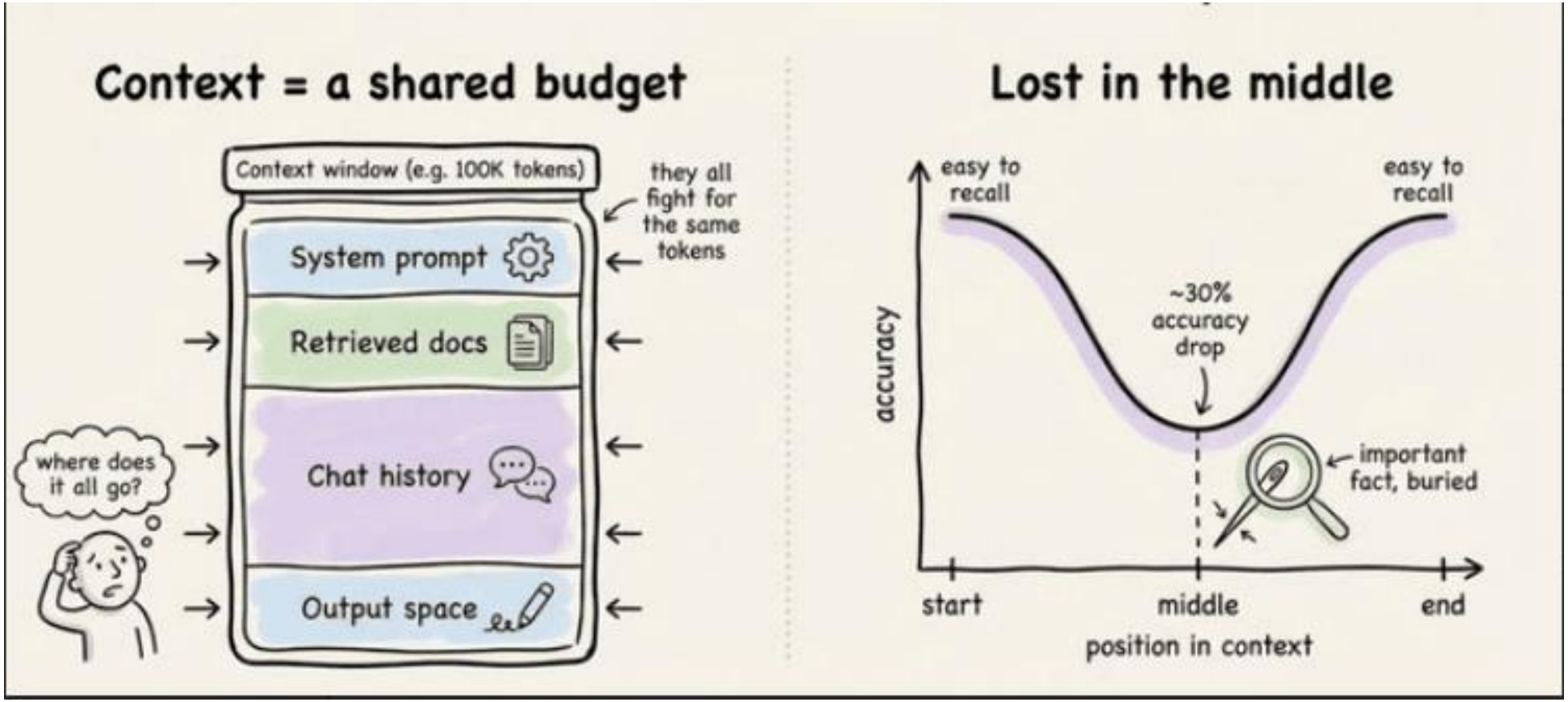
Confronto illustrativo: servono tutte le informazioni pertinenti, con poco rumore.

Fonti: Claude Code Docs — model-config, context-window, commands, prompt-caching · Anthropic · Liu et al., Lost in the Middle · Chroma, Context Rot

Il noto problema «lost in the middle» del contesto

Daily Dose of Data Science

24.04. 2026



A bigger window is not the same as better memory. It just gives the model more room to miss things.

Vedi anche questo articolo

ByteByteGo Newsletter

Automatic context management

For users on paid plans with code execution enabled, Claude automatically manages your conversation context. When your conversation approaches the context window limit, Claude summarizes earlier messages to make room for new content. This allows conversations to continue indefinitely in most cases. Longer conversations that trigger automatic context management use more of your usage limit.

Your full chat history is preserved so Claude can reference it, even after earlier portions have been summarized. You may occasionally notice Claude "organizing its thoughts" during long conversations—this is the automatic context management at work.

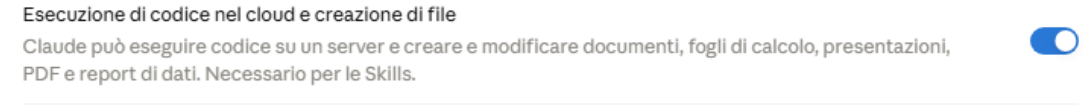
Note: Code execution must be enabled for automatic context management to work. In rare edge cases (such as very large first messages or system errors), you may still encounter context window limits.

Si trova nelle impostazioni dell'app Claude, e il nome esatto dell'interruttore è **"Code execution and file creation"** ("Esecuzione di codice e creazione di file"). Il percorso dipende dal tipo di piano:

- **Piani Free, Pro e Max** (il caso individuale): **Settings > Capabilities** (Impostazioni > Funzionalità), dove si attiva o disattiva l'interruttore **Code execution and file creation**. Dal web il percorso diretto è claude.ai → icona del profilo in basso a sinistra → Settings → Capabilities. claude
- **Piani Team ed Enterprise**: la decisione spetta ai proprietari dell'organizzazione, che la regolano in **Organization settings > Capabilities** con lo stesso interruttore; per i singoli utenti dell'organizzazione la voce può quindi non essere modificabile. claude

Due note utili:

- La funzionalità è disponibile per tutti gli utenti Claude (Free, Pro, Max, Team ed Enterprise) su web, Claude Desktop e Claude Mobile, ed è attiva per impostazione predefinita — quindi è molto probabile che sul tuo account sia già accesa senza che tu debba fare nulla. claude
- Attenzione a non confonderla con Claude Code: nonostante il nome simile, questo interruttore riguarda solo la sandbox (l'ambiente di esecuzione isolato) delle chat nell'app Claude — quella che abilita la creazione di file e, come visto prima, la gestione automatica del contesto. Claude Code, lo strumento da terminale, è un prodotto separato e non è controllato da questa impostazione.



Automatic context management (gestione automatica del contesto)

È il meccanismo con cui l'app Claude evita che una conversazione lunga "sbatta" contro il limite della finestra di contesto (la quantità massima di token — unità di testo — che il modello può tenere in vista in un dato momento, i 500K o 1M delle tabelle precedenti).

Funziona così:

- **Quando si attiva**: solo su piani a pagamento e solo se l'esecuzione di codice (*code execution*, la funzione che permette a Claude di eseguire codice in un ambiente sandbox) è abilitata. Il riquadro "Note" lo ribadisce: senza code execution, la gestione automatica non funziona.
- **Cosa fa**: quando la conversazione si avvicina al limite della finestra, Claude **riassume i messaggi più vecchi** per liberare spazio. I riassunti occupano molti meno token degli originali, quindi la conversazione può proseguire praticamente all'infinito nella maggior parte dei casi.
- **La cronologia non si perde**: l'intera chat resta salvata e Claude può andarla a riconsultare anche dopo che le parti iniziali sono state riassunte. Nel contesto attivo c'è il riassunto, ma l'originale rimane recuperabile.
- **L'indizio visibile**: durante le conversazioni lunghe si può notare Claude che "organizza i pensieri" (*organizing its thoughts*) — quella pausa è proprio il riassunto in corso.
- **Il costo nascosto**: le conversazioni lunghe che attivano questo meccanismo **consumano più limite di utilizzo** (*usage limit*, la quota di messaggi/token del piano), perché l'operazione di riassumere è a sua volta lavoro del modello.
- **I casi limite**: in situazioni rare — un primo messaggio enorme (es. un documento gigantesco incollato subito), o errori di sistema — si può comunque incontrare il limite, perché non c'è "materiale vecchio" da riassumere.

Maximizing your context window

While context is managed automatically for most conversations, you can still optimize how you use your available context space:

- **Utilize projects effectively:** Projects use retrieval-augmented generation (RAG), which allows Claude to work with larger amounts of information by only loading relevant content into the context window.
- **Keep project instructions concise:** Claude performs best when you use project instructions for general context around your project, key guidelines, and Claude's role.
- **Manage tools and connectors:** These features are token-intensive, so being mindful of how many you have active helps maximize your available context.

È la sezione "Maximizing your context window" ("Ottimizzare la finestra di contesto") che avevamo già incrociato nell'immagine precedente — qui la vediamo per intero. Il messaggio di apertura dice: anche se per la maggior parte delle conversazioni il contesto viene gestito automaticamente (il riassunto automatico di cui parlavamo), conviene comunque usare bene lo spazio disponibile. Seguono tre consigli pratici:

1. Utilize projects effectively (usare bene i Projects)

I Projects sono le aree di lavoro di claude.ai in cui si caricano documenti di riferimento (la "knowledge base" del progetto). Il punto chiave è che i Projects usano la RAG, *Retrieval-Augmented Generation*: una tecnica in cui i documenti non vengono caricati per intero nel contesto a ogni messaggio, ma vengono prima indicizzati, e poi a ogni domanda viene recuperata e inserita nel contesto solo la parte rilevante. Il vantaggio: si può lavorare con una mole di materiale molto più grande della finestra di contesto stessa, perché in finestra entra solo ciò che serve in quel momento. (È esattamente il meccanismo che tratti nei tuoi corsi sulla RAG, applicato qui a livello di prodotto.)

2. Keep project instructions concise (istruzioni di progetto concise)

Le *project instructions* sono le indicazioni permanenti che si scrivono in un Project ("sei un revisore di testi, rispondi in italiano, usa questo formato…"). A differenza dei documenti, queste **vengono caricate integralmente nel contesto a ogni messaggio**, senza passare dalla RAG. Quindi ogni parola in più è un costo fisso che si paga sempre. Il consiglio: usarle solo per il contesto generale, le linee guida chiave e il ruolo di Claude — i contenuti lunghi vanno invece nei documenti del Project, dove la RAG li gestisce in modo selettivo.

3. Manage tools and connectors (gestire strumenti e connettori)

I connettori sono le integrazioni con app esterne (Gmail, Google Drive, Slack…) e gli strumenti sono le capacità aggiuntive (ricerca web, ecc.). Ognuno di essi, quando è attivo, inserisce nel contesto le proprie definizioni — la descrizione di cosa fa e di come va chiamato — **anche se nella conversazione non lo si usa mai**. L'articolo li definisce "token-intensive", cioè avidi di token. Il consiglio: tenere attivi solo i connettori che servono davvero, disattivando gli altri, così da liberare spazio per il contenuto vero della conversazione.

In sintesi, i tre punti rispondono alla stessa domanda da tre angolazioni: *cosa occupa la finestra di contesto oltre ai messaggi?* I documenti (→ delegarli alla RAG dei Projects), le istruzioni fisse (→ tenerle brevi) e le definizioni degli strumenti (→ attivarne pochi).

è vera questa regola del pollice: oltre il 20% di occupazione della finestra di contesto conviene fare nuova chat (o /clear in claude code) oppure compattare la chat?

Lette 2 pagine, cercato sul web >

In parte. Il fenomeno sottostante è reale e ben documentato, ma la soglia del 20% come regola universale non è sostenuta né dalla documentazione ufficiale né dalla ricerca: è una euristica di community, e peraltro una delle più aggressive in circolazione. Vediamo cosa dice l'evidenza.

Il fenomeno è vero: il "context rot" (degrado da contesto). Riempire la finestra di contesto non è gratis: il modello non diventa "stanco", ma peggiora nell'*usare* le informazioni che già ha. Diversi studi lo misurano:

- **Lost in the Middle** (Liu et al., 2024): i modelli recuperano bene le informazioni all'inizio e alla fine del contesto, male quelle in mezzo — in un test con 20 documenti, l'accuratezza sulle informazioni centrali scendeva al 25% contro il 42% di quelle finali.
- **Context Rot** (Chroma Research): su 18 modelli testati, le prestazioni calavano di oltre il 30% quando l'informazione rilevante stava in mezzo al contesto, e i "distrattori" semanticamente simili peggioravano le cose.
- Altri lavori (Du et al., 2025) mostrano cali del 14-85% al crescere della lunghezza dell'input, *anche quando il modello recuperava tutte le informazioni rilevanti* — perfino sostituendo il contesto irrilevante con spazi bianchi il calo restava.

Ma le soglie numeriche sono deboli. In giro si trovano il 20%, il 40% (la "regola del 40%", che distingue una "smart zone" da una "dumb zone"), il 70%... Il punto debole: gran parte della ricerca citata usa modelli piccoli (es. Qwen 7B, dove la soglia critica era al 40-50%), dove l'effetto è molto più marcato; sui modelli di frontiera il degrado è più dolce, senza un "precipizio" a una percentuale precisa. Lo ammettono gli stessi autori di queste regole del pollice. Il 20% in particolare viene dall'esperienza personale di singoli sviluppatori, senza benchmark a supporto.

La regola migliore non è una percentuale, ma un criterio di pertinenza. Il consiglio ufficiale di Anthropic per Claude Code è usare `/clear` *tra un compito e l'altro*, non a una soglia fissa: quando cambiamo argomento, tutto il contesto precedente è diventato rumore — distrattori semanticamente simili, che come visto sopra sono i peggiori. Riformulando:

- **Compito nuovo e scollegato** → `/clear` (o nuova chat), anche se siamo al 10% di occupazione
- **Stesso compito, contesto che si allunga** → si può lavorare tranquillamente ben oltre il 20%; `/compact` quando compaiono i sintomi tipici (il modello rilegge file già letti, reintroduce bug già corretti, dimentica decisioni prese nella stessa sessione) o prima di una fase delicata
- **Mai arrivare alla compattazione automatica** sotto pressione (scatta vicino al limite): a quel punto il riassunto butta via dettagli che non scegliamo noi — meglio compattare deliberatamente a un punto di svolta naturale del lavoro, dove sappiamo cosa serve conservare

In sintesi: la direzione della regola è giusta (meno contesto irrilevante = risposte migliori, e molto prima del limite nominale), la soglia fissa del 20% no — è troppo prudente come regola generale e comunque il *quando* dipende più da *cosa* c'è nella finestra che da *quanto* è piena.

Sources: Why I compact my AI coding agent at 20%, not 90% — Level Up Coding, The context 40 percent rule — deknijf.com, Claude Code context management — SitePoint

Incide anche il «workload» del momento? Ufficialmente, no; in realtà, sì.

Come interpretare l'output di /memory in Claude Code

Guida pratica alla schermata di memoria, istruzioni e auto-memory

Claude Code
PIÙ CONTESTO
PER LAVORARE MEGLIO

Output di esempio

Esempio semplificato dell'output di /memory

```
> /memory
  └ Opened ~/.claude/CLAUDE.md
    | To use a different editor, set the $EDITOR or $VISUAL environment variable.

> /memory
  └ Cancelled memory editing

 Memory

> Auto-memory   true

> User instructions      Saved in ~/.claude/CLAUDE.md
  Project instructions   Saved in ./CLAUDE.md
  Open auto-memory folder
```

Lettura corretta

Cosa significa ogni riga e come interpretarla

1. **Auto-memory true**
   "Auto-memory true" = la memoria automatica è attiva.
2. **Opened ~/.claude/CLAUDE.md**
   Indica l'apertura del file delle istruzioni personali.
3. **Cancelled memory editing**
   Significa che è stata annullata un'operazione di editing/ selezione, non che la memoria sia stata cancellata.
4. **User instructions e Project instructions**
   Sono file di istruzioni, distinti dalla auto-memory.
5. **Open auto-memory folder**
   È la voce da usare per vedere i ricordi salvati automaticamente.

Che cosa significa ogni voce

Una guida rapida ai termini dell'output

Voce nell'output	Significato
Auto-memory	Funzione di memoria automatica (attiva o disattiva).
User instructions	Istruzioni personali globali, salvate in ~/.claude/CLAUDE.md.
Project instructions	Istruzioni del progetto corrente, salvate in ./CLAUDE.md.
Open auto-memory folder	Apre la cartella dei file di memoria salvati automaticamente.

Messaggio chiave

3 takeaway da ricordare

1. **Memoria attiva ≠ memoria già popolata.**
   Auto-memory attiva significa solo che la funzione è abilitata, non che ci siano già ricordi salvati.
2. **I file CLAUDE.md non sono la stessa cosa della auto-memory.**
   User instructions e Project instructions sono file di istruzioni, distinti dalla memoria automatica.
3. **Per verificare i ricordi effettivi, apri la cartella della auto-memory.**
   Usa "Open auto-memory folder" per vedere i file di memoria salvati.

Suggerimento hands-on: usa `/memory` per gestire istruzioni e auto-memory, e `/context` per capire quali file sono stati caricati nel contesto.

PICCOLI COMANDI,
GRANDI RISULTATI

Il comando `/memory` di Claude Code apre il pannello di gestione dei **file di memoria** della sessione: è il punto in cui si vede, si modifica e si configura tutto ciò che Claude Code "ricorda" tra una sessione e l'altra. Attenzione: è una memoria diversa da quella dell'app Claude di cui parlavamo prima — stesso concetto (persistenza tra sessioni), ma archivi separati: quella dell'app segue l'account, quella di Claude Code vive sul computer, legata ai progetti.

Nel dettaglio, `/memory` riporta:

1. L'elenco dei file CLAUDE.md — i file di istruzioni scritti *dall'utente*, su più livelli, dal più generale al più specifico:

- **User**: `~/.claude/CLAUDE.md` — vale per tutti i nostri progetti
- **Project**: `./CLAUDE.md` (o `./.claude/CLAUDE.md`) — condiviso col team via git
- **Local**: `./CLAUDE.local.md` — personale, da mettere nel `.gitignore`
- (esiste anche un livello **managed policy** aziendale, es. `/etc/claude-code/CLAUDE.md` su Linux, ma di norma non compare nell'elenco di `/memory`)

L'elenco mostra anche i file che non esistono ancora; selezionandone uno, viene creato e aperto nell'editor (VS Code si apre in una finestra separata, un editor da terminale come Vim prende il controllo del terminale).

2. Il controllo della auto memory — la memoria scritta *da Claude stesso*: appunti su preferenze, correzioni e contesto del progetto che Claude salva automaticamente in `~/.claude/projects/<progetto>/memory/`. Lì c'è un indice `MEMORY.md` (caricato a inizio sessione, fino a 200 righe o 25KB) più file tematici letti al bisogno. Da `/memory` si attiva/disattiva questa funzione (l'impostazione `autoMemoryEnabled` in `~/.claude/settings.json`) e si apre la cartella che la contiene. È attiva per default nelle sessioni locali.

La distinzione utile da ricordare: **CLAUDE.md è ciò che scriviamo noi, la auto memory è ciò che scrive Claude** — e `/memory` è il posto unico per gestire entrambe. Dire a Claude "ricordati che usiamo pnpm" finisce nella auto memory; dire "aggiungilo al CLAUDE.md" (o modificarlo via `/memory`) scrive nel file di istruzioni.

Nota pratica: per vedere quali file di memoria sono stati *effettivamente caricati* nel contesto della sessione corrente, il comando giusto è `/context`, sezione "Memory files" — il che ci riporta al discorso di prima: anche questi file consumano token della finestra di contesto.

Sources: Memory — code.claude.com, Slash commands — code.claude.com

Tabella riassuntiva

	Finestra di contesto	Gestione automatica del contesto	Memoria
Cos'è	Limite fisico del modello	Funzione dell'app (compressione)	Funzione dell'app (archivio)
Ambito	Una conversazione	Una conversazione	Tutte le conversazioni
Persistenza	Nessuna	Nessuna (fuori dalla chat)	Sì
Contenuto	Tutto, letterale	Riassunti del passato della chat	Fatti durevoli selezionati
Scopo	Vedere per rispondere	Non sbattere contro il limite	Continuità tra sessioni

In una frase: la finestra di contesto è *quanto* Claude può vedere ora; la gestione automatica decide *cosa tenere in vista* quando lo spazio finisce; la memoria decide *cosa portarsi dietro* da una conversazione all'altra. Se ti serve per un corso, i tre livelli si mappano bene su concetti classici: memoria di lavoro, compressione/summarization, e storage persistente con retrieval.

link alla docs di Claude Code sul prompt caching

CORE CONCEPTS

How Claude Code uses prompt caching

Claude Code manages prompt caching automatically. See why a model switch triggers a slow uncached turn, what `/compact` costs, why CLAUDE.md edits don't apply mid-session, and how to check your cache hit rate.

Prompt caching makes Claude Code faster and more cost-efficient. Without caching, the API would reprocess your full history on every turn. With caching, it reuses what it already processed, bills the re-read at the **cached token rate**, and fully processes only what changed.

Claude Code handles prompt caching for you, unless you **disable it**. It is still useful to know how prompt caching works, because some actions invalidate the cache and make the next response slower and more expensive while it rebuilds. This page covers which actions those are, why some settings wait for a restart to apply, and how to check cache performance when usage looks high.

/compact E CONTEXT WINDOW (CW)

Si possono perdere informazioni importanti?

SÌ: LA COMPATTAZIONE È LOSSY.

Nella CW, la conversazione integrale viene sostituita da un riassunto che può perdere dettagli.

COSA RESTA, COSA SI RIDUCE

Nel riassunto

Richieste e intento; concetti e decisioni chiave; file e snippet rilevanti; errori risolti; task aperti e lavoro in corso.

Dettagli a rischio

Output integrali dei tool, passaggi intermedi e codice letto parola per parola. Il codice esatto può richiedere una nuova lettura.

RICARICAMENTI MENO OVVI

CLAUDE.md di progetto e regole generali: reiniettati.

Regole con paths: e CLAUDE.md annidati: tornano quando si legge, scrive o modifica un file pertinente.

Skill: nella simulazione ufficiale, l'elenco non viene reiniettato; i corpi delle skill già invocate tornano entro limiti.

QUATTRO MOSSE PER LIMITARE LE PERDITE

1 · DARE UN FOCUS

```
/compact focus on
bug di login, decisioni e test
```

Guida il riassunto; non garantisce zero perdite.

2 · SCEGLIERE IL MOMENTO

Preferire fine task o un confine pulito. Nel debug aperto, fissare prima ipotesi, prove e risultati.

3 · SALVARE E RILEGGERE

```
docs/plan.md: stato, decisioni
e prossimi passi. Chiederne la
rilettura dopo /compact.
```

Regole stabili nel CLAUDE.md di progetto.

4 · CAMBIARE CONVERSAZIONE

```
/clear
```

Per un nuovo lavoro indipendente: ripartire con una conversazione pulita.

SALVARE LO STATO SU FILE E RILEGGERLO DOPO LA COMPATTAZIONE.

Fonti: Claude Code Docs · Context window · Memory · How Claude Code works

Claude Code: configurazioni essenziali

I file e le regole da conoscere per iniziare

ISTRUZIONI · Markdown
Testo che Claude legge come guida

IMPOSTAZIONI · JSON
Regole e opzioni applicate dal programma

4 FILE DA CONOSCERE

`CLAUDE.md`
Progetto · istruzioni condivise
Build, test e convenzioni. Breve e chiaro. In Git.

`~/.claude/settings.json`
Utente · tutti i progetti
Preferenze, modello e permessi personali.

`.claude/settings.json`
Progetto · team
Impostazioni comuni da condividere con Git.

`.claude/settings.local.json`
Progetto · personali
Override e approvazioni persistenti. Da escludere da Git.

~ = cartella home. Su Windows: %USERPROFILE%/.claude

PERMESSI · permissions

`allow` = consenti `ask` = chiedi conferma `deny` = blocca

Tra regole applicabili: `deny` > `ask` > `allow`

Esempio in `~/.claude/settings.json`

```
{
   "permissions": {
     "allow": ["Bash(npm test *)"],
     "deny": ["Read(./.env)", "Read(./.env.*)"]
   }
}
```

Test senza conferma; lettura dei file .env limitata.

Da sole, le regole Read non bloccano ogni lettura indiretta tramite script.

COMANDI UTILI

`/config`	`/status`	`/memory`	`claude doctor`
Opzioni personali	Fonti settings caricate	CLAUDE.md e auto-memory	Diagnostica dal terminale

PRECEDENZA DELLE IMPOSTAZIONI

Dalla più alta alla più bassa

Azienda → **CLI (`--settings`)** → **Locale** → **Condiviso** → **Utente**

I valori singoli si sovrascrivono. Le liste di permessi si uniscono.

VS Code: autocompletamento e validazione con $schema

```
"$schema": "https://json.schemastore.org/claude-code-settings.json"
```

ATTENZIONE

- JSON: niente commenti o virgole finali.
- ~/.claude.json: stato interno dell'app.
- .claudeignore non è supportato.
- Non stampare segreti: le sessioni si salvano in chiaro.

I file settings possono mancare all'inizio: vengono creati quando servono.

Fonti: documentazione ufficiale Claude Code

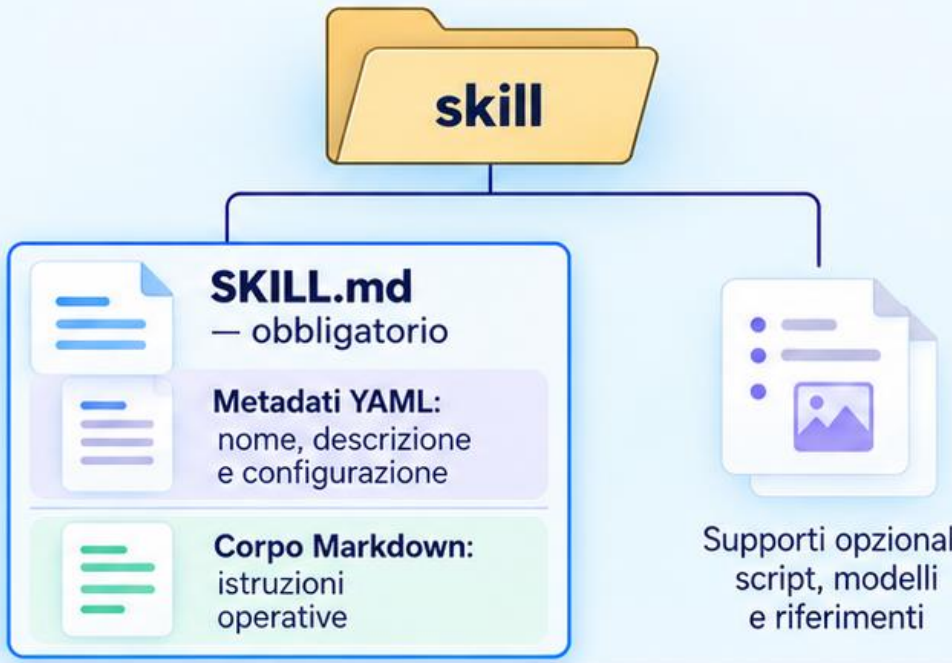
1. **Skill — Come svolgere un compito**
   Struttura di `SKILL.md`, file di supporto, attivazione automatica o tramite comando `/...`, caricamento progressivo ed esempio del report NPL.
2. **Server MCP — Accesso a strumenti e dati**
   Relazione tra Claude Code, server e sistemi esterni; distinzione fra *tools*, *resources* e *prompts*; collegamenti HTTP e stdio, configurazione ed esempio del data warehouse.
3. **Plugin — Riunire e distribuire i componenti**
   Manifest, componenti disponibili, marketplace, installazione ed esempio del pacchetto per l'analisi del rischio di credito.

SKILL IN CLAUDE CODE

Il «come fare» che Claude carica quando serve

UNA PROCEDURA RIUTILIZZABILE

Istruzioni e conoscenza per svolgere un compito.
Insegna a usare meglio gli strumenti già disponibili.



I campi dei metadati sono opzionali.
La *description* aiuta Claude a capire quando usarla.

COME SI ATTIVA

AUTOMATICA
Claude riconosce una richiesta pertinente.

MANUALE
Nella sessione Claude Code:
`/report-npl`

CARICAMENTO PROGRESSIVO

Nome e descrizione nel contesto → Corpo completo solo quando serve

ESEMPIO: REPORT NPL

Report mensile sui crediti deteriorati

1. Leggere `data/npl.csv`
2. Calcolare l'incidenza NPL per filiale
3. Eseguire `scripts/valida_dati.py`
4. Applicare `templates/report.md`

Una procedura standard da riutilizzare

COME SI COLLEGA AGLI ALTRI

Skill → insegna come lavorare — **Server MCP** → collega strumenti e dati esterni — **Plugin** → confeziona e distribuisce

Una skill può spiegare come usare un sistema raggiunto tramite MCP.

Fonte unica: skill-mcp-plugin-claude-code.md

Skill vs Istruzioni in Claude Code

Due meccanismi diversi: regole permanenti vs competenze attivabili

Le **istruzioni** guidano il comportamento generale di Claude. Le **Skill** descrivono procedure e competenze riutilizzabili da attivare quando servono.

Aspetto di confronto	Istruzioni	Skill
Dove si trovano	**CLAUDE.md**	**SKILL.md**
Scopo	Regole, convenzioni, preferenze	Workflow, procedure, compiti specifici
Attivazione	Caricate automaticamente	Invocate manualmente con `/nome-skill` oppure selezionate da Claude quando pertinenti
Ambito	Generale o di progetto	Mirata a un compito
Contenuto tipico	Convenzioni, vincoli, stile	Passi operativi, script, template, file di supporto
Esempio	`Usa Ruff e pytest`	`/verifica-progetto esegue lint, test e report`

In breve: CLAUDE.md dice **come lavorare.** Una Skill spiega come svolgere un lavoro specifico.

Quando usare Istruzioni e quando usare Skill

Esempio pratico in un progetto Python con Claude Code

Usa le Istruzioni (CLAUDE.md) per...

stabilire convenzioni del progetto

```
Usa Python 3.13
```

imporre strumenti e standard

```
Gestisci le dipendenze con uv
```

definire preferenze generali

```
Usa Ruff e pytest
```

ridurre ripetizioni nei prompt

```
Non modificare .env senza
autorizzazione
```

Usa una Skill per...

eseguire workflow ripetibili

```
/scaffold-python
```

crea struttura del progetto

guidare attività multi-step

```
/verifica-progetto
```

lancia lint e test

richiamare procedure specializzate

```
/genera-docs
```

aggiorna documentazione

incapsulare conoscenze operative

Raccoglie best practice, comandi e passaggi per un obiettivo specifico.

Regola pratica

Serve una regola sempre valida?

↓

Istruzioni (CLAUDE.md)

Serve una procedura da attivare quando occorre?

↓

Skill

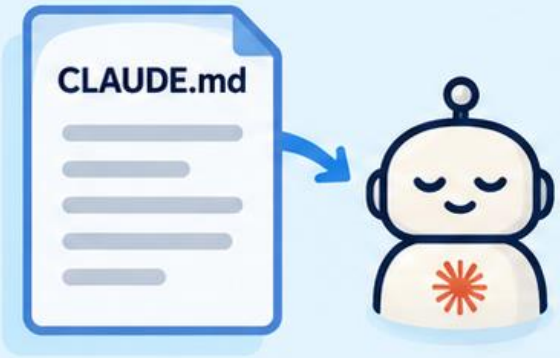
Una Skill contiene istruzioni, ma non tutte le istruzioni sono Skill.

Quando si caricano e come si attivano

La differenza operativa tra istruzioni e Skill in Claude Code

1 Avvio della sessione

Claude legge automaticamente le istruzioni applicabili del progetto (CLAUDE.md).



Le istruzioni vengono caricate all'avvio, se applicabili.

2 Durante il lavoro

Claude conosce l'elenco delle Skill disponibili, soprattutto nome e descrizione.

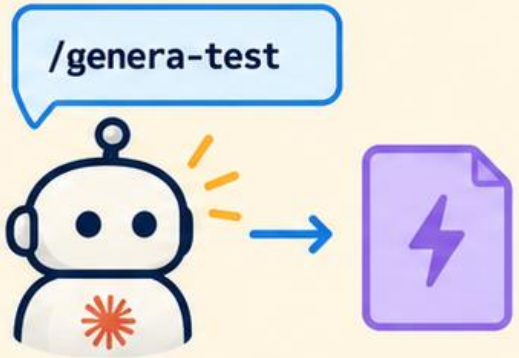
Skill disponibili

- **analizza-codice** — Analizza la qualità del codice...
- **genera-test** — Genera e registra test...
- **deploy** — Prepara il deploy dell'applicazione...

Claude conosce i nomi e le descrizioni, non il contenuto completo.

3 Quando serve un compito specifico

La Skill viene attivata manualmente con /nome-skill oppure scelta automaticamente da Claude se pertinente.

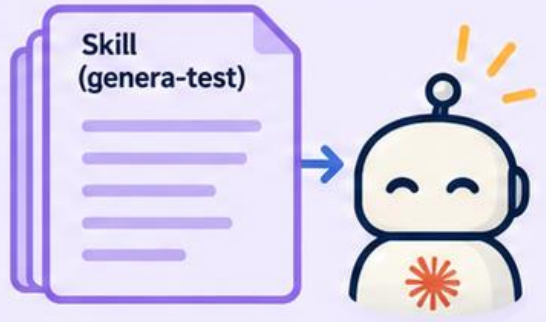


Attivazione manuale con /nome-skill

Oppure automatica se pertinente

4 Dopo l'attivazione

Il contenuto della Skill entra nel contesto e guida il workflow richiesto.



Il contenuto completo della Skill viene caricato nel contesto e Claude segue il workflow.

Nota sul contesto

Istruzioni: occupano contesto fin dall'inizio, se applicabili.

Skill: il contenuto completo viene caricato solo quando usato.

Attenzione

Con `disable-model-invocation: true`, Claude non può attivare autonomamente la Skill.

Quindi:

le istruzioni sono sempre presenti; le Skill sono on demand.

SERVER MCP IN CLAUDE CODE

A che cosa può accedere Claude?

MCP — Model Context Protocol

Un protocollo aperto per collegare l'AI a strumenti e dati esterni.

Il server MCP è un programma locale o remoto.

TRE CAPACITÀ

TOOLS

Azioni da invocare

Query, ricerca email, creazione ticket

`mcp__jira__create_issue`

RESOURCES

Dati da leggere

PROMPTS

Richieste predefinite riutilizzabili

DUE MODI DI COLLEGARSI

HTTP · REMOTO

Via rete, consigliato per i servizi cloud

STDIO · LOCALE

Processo sulla macchina, tramite standard input/output

CONFIGURAZIONE E CONTROLLO

Dalla shell Windows:

```
claude mcp add --transport http nome-server https://esempio.com/mcp
```

`claude mcp list` Elenca i server

Nella sessione Claude Code:

`/mcp` Mostra lo stato delle connessioni

AMBITI DI CONFIGURAZIONE

- **local** — noi, nel progetto corrente
- **project** — team, tramite .mcp.json
- **user** — tutti i nostri progetti

Esempio bancario: MCP collega Claude al data warehouse dei crediti per interrogare le tabelle.

Fonte unica: skill-mcp-plugin-claude-code.md

PLUGIN IN CLAUDE CODE

Un kit di lavoro da distribuire al team

1 UN PACCHETTO INSTALLABILE

Raggruppa componenti in un'unica distribuzione.

Manifest: `.claude-plugin/plugin.json`
Unico campo obbligatorio: `name`

PUÒ CONTENERE

- **Skill** — procedure e conoscenza
- **Server MCP** — accesso a strumenti e dati
- **Subagenti** — assistenti specializzati
- **Hook** — comandi legati agli eventi

Anche: comandi storici, LSP, monitor, workflow, eseguibili, stili e temi.

2 ESEMPIO: RISCHIO-CREDITO

- **MCP** — Interroga il data warehouse dei crediti
- **Skill analisi-npl** — Schema, calcoli e formato dei report
- **Hook PreToolUse** — Blocca query sulle tabelle con dati personali
- **Subagente revisore-report** — Rilegge il report prima della consegna

Un'installazione: accesso, procedura e vincoli

3 MARKETPLACE E INSTALLAZIONE

Un marketplace è un catalogo di plugin, anche aziendale.

Nella sessione Claude Code:

```
/plugin
/plugin marketplace add mia-org/claude-plugins
/plugin install analisi-rischio@mia-org
```

Skill del plugin: `/<plugin>:<nome>`

Ambiti: `user · project · local · managed`

4 DA TENERE PRESENTE

SICUREZZA
Il plugin usa i privilegi dell'utente: esaminare quelli di terze parti.

CONTESTO
Ogni turno include nomi e descrizioni dei componenti attivabili dal modello. Abilitare solo ciò che serve.

Fonte unica: skill-mcp-plugin-claude-code.md

Deployment dei plugin Claude Code: tre modalità

Dal test locale alla distribuzione al team

1 Locale temporaneo
Sviluppo e test

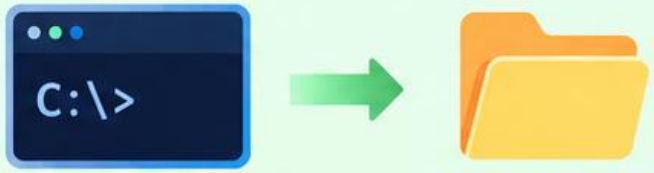
Cartella del plugin (o file .zip) → Avvia Claude Code con il plugin

Comando da CMD

```
claude --plugin-dir C:\claude-plugins\python-tools
```

- Valida solo per la sessione avviata.
- Non installa nulla.
- Accetta una cartella locale o un file `.zip` del plugin.

2 Personale persistente
Uso su questo PC



Crea il plugin personale → `~/.claude/skills/python-tools/`

Comando da CMD

```
claude plugin init python-tools --with skills
```

- Genera il plugin nella directory `~/.claude/skills/python-tools/`.
- Il plugin è caricato automaticamente nelle sessioni successive (se consentito).
- Serve per creare il plugin personale, non per installare una cartella arbitraria esistente.

3 Marketplace GitHub
Team e distribuzione

Repository GitHub con il plugin → `.claude-plugin/marketplace.json` → Installazione del team

Comandi da CMD

```
claude plugin  marketplace add owner/repo
claude plugin install python-tools@formazione-ai
```

- Pubblicazione mediante repository con `.claude-plugin/marketplace.json`.
- Consente l'installazione del plugin su più computer.
- Permette la gestione degli aggiornamenti.

Un plugin è una cartella di componenti (Skill, agent, hook, MCP), non un eseguibile .exe.

Non serve un server cloud per il test locale.

Anatomia di un plugin Claude Code

Manifest, Skill e componenti opzionali

Struttura di un plugin (esempio: python-tools)

```
python-tools/
├── .claude-plugin/
│   └── plugin.json
├── skills/
│   └── verifica-progetto/
│       └── SKILL.md
├── agents/
│   └── reviewer.md   (opzionale)
├── hooks/
│   └── hooks.json    (opzionale)
└── .mcp.json         (opzionale)
```

plugin.json
(.claude-plugin/plugin.json)

- Nome, versione e descrizione del plugin
- Identifica il plugin per l'installazione e la distribuzione
- Solo questo file va nella cartella .claude-plugin/

SKILL.md
(skills/<nome>/SKILL.md)

- Istruzioni operative richiamabili da Claude Code
- Definisce una procedura o competenza riutilizzabile
- Può includere esempi, script e file di supporto

agents/
(agents/<nome>.md)

- Subagent specializzati
- Definiscono comportamento e competenze specifiche
- Utili per attività complesse o di dominio

hooks/
(hooks/hooks.json)

- Automazioni attivate da eventi
- Eseguono comandi o script prima/dopo certe azioni
- Esempi: controlli di sicurezza, linting automatico

.mcp.json
(.mcp.json)

- Configurazione di server MCP e tool esterni
- Permette al plugin di usare strumenti e integrazioni
- Facoltativo, solo se il plugin espone server MCP

Esempio di plugin.json

Manifest minimale del plugin.

```json
{
  "name": "python-tools",
  "version": "1.0.0",
  "description": "Strumenti per progetti Python"
}
```

Esempio di SKILL.md (frontmatter)

Intestazione della Skill.

```yaml
---
name: verifica-progetto
description: Controlla un progetto Python
disable-model-invocation: true
---
...
```

Invocazione della Skill in Claude Code

Una volta installato il plugin, la Skill si richiama così:

```
> /python-tools:verifica-progetto
```

Claude carica il contenuto di SKILL.md ed esegue le istruzioni operative.

Nota importante

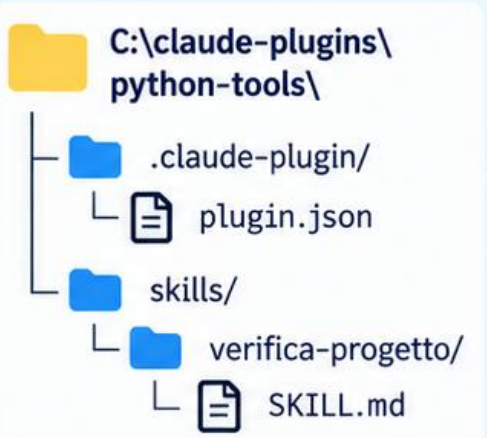
Solo plugin.json va in .claude-plugin/; skills/, agents/, hooks/ stanno nella root del plugin. Nessun .exe da compilare per questo esempio.

Test locale di un plugin su Windows

Dalla cartella alla Skill, senza installazione permanente

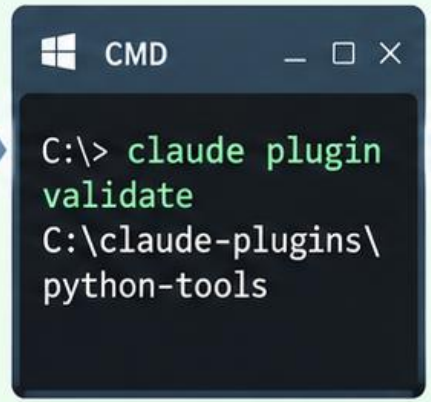
1 Prepara la cartella

Crea la cartella del plugin con la struttura minima richiesta.



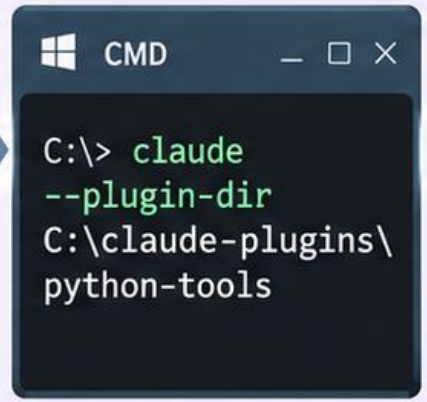
2 Valida da CMD

Verifica che il plugin sia corretto.



3 Avvia da CMD

Avvia Claude Code caricando il plugin.

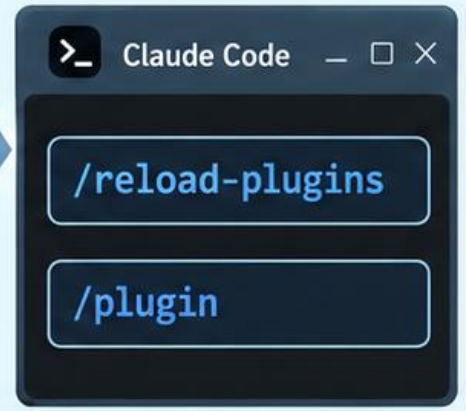


4 Invoca DENTRO Claude Code

Nella sessione di Claude Code usa la Skill del plugin.

5 Dopo le modifiche

Se modifichi i file del plugin, ricaricalo e verifica lo stato.



CMD — comandi di sistema

Si usano nel prompt dei comandi di Windows per validare e avviare Claude Code con il plugin.

Claude Code — comandi slash

Si usano DENTRO la sessione di Claude Code per invocare la Skill, ricaricare i plugin e verificare lo stato.

`--plugin-dir` carica il plugin solo per la sessione avviata: non lo installa.

`disable-model-invocation: true` rende la Skill invocabile solo manualmente.

Distribuire un plugin con GitHub e un marketplace

Pubblicazione, installazione e aggiornamenti per un team

1 Prepara il repository

Crea la struttura del plugin con i file necessari, inclusi plugin.json e marketplace.json.

```
python-tools/
├── .claude-plugin/
│   ├── plugin.json
│   └── marketplace.json
└── skills/
    └── verifica-progetto/
        └── SKILL.md
```

Esempio di marketplace.json:

```json
{
  "name": "formazione-ai",
  "owner": {
    "name": "Team Formazione"
  },
  "plugins": [
    {
      "name": "python-tools",
      "source": "./"
    }
  ]
}
```

2 Pubblica con GitHub Desktop

Esegui commit e push dei file del plugin sul repository GitHub.

1. **Aggiungi i file del plugin** (plugin.json, marketplace.json, skills, ecc.)
2. **Scrivi un messaggio di commit**

   `Initial release of python-tools plugin`
3. **Esegui il push sul repository**

   `organizzazione/python-tools`

   (nome solo esemplificativo)

Il repository può essere pubblico o privato, ad esempio privato per il team.

3 I partecipanti installano

Ogni partecipante aggiunge il marketplace e installa il plugin dal repository pubblicato.

1 Aggiungi il marketplace

```
claude plugin marketplace add organizzazione/python-tools
```

2 Installa il plugin

```
claude plugin install python-tools@formazione-ai
```

3 Verifica l'installazione

Dopo l'installazione, controlla che il plugin sia disponibile:

```
claude plugin list
```

Esegui i comandi in **Windows CMD**.

4 Distribuisci aggiornamenti

Per rilasciare una nuova versione, aggiorna il numero di versione, fai commit e push, e i partecipanti aggiornano il plugin.

1. Aumenta la versione

Modifica il campo version in plugin.json.

```json
{
  "name": "python-tools",
  "version": "1.0.1"
}
```

2. Commit e push

Fai commit e push dei file aggiornati sul repository GitHub.

3. Gli utenti aggiornano

Ogni partecipante esegue:

```
claude plugin update python-tools@formazione-ai
```

Il marketplace è un catalogo di plugin, NON un server applicativo.
Gli aggiornamenti automatici di un marketplace personale sono disattivati per impostazione predefinita.

SUBAGENT IN CLAUDE CODE

Assistenti specializzati: un compito delegato, un contesto dedicato.

CONVERSAZIONE PRINCIPALE
Definisce obiettivo e incarico

delega con Agent

ESEMPI GIÀ INTEGRATI

- **Explore** — Esplora il codice in sola lettura
- **Plan** — Ricerca in sola lettura per preparare un piano
- **general-purpose** — Analisi, attività complesse e modifiche ai file

RISULTATI ALLA PRINCIPALE
Una **sintesi utile**, senza riversare tutti i passaggi intermedi.

Ogni subagent: contesto dedicato · istruzioni del ruolo · strumenti e permessi · modello ereditato o configurato | Le configurazioni restano soggette ai vincoli della sessione.

PERCHÉ USARLI

CONTESTO PIÙ LEGGERO
Ricerche, log e test restano nel contesto del subagent.

SPECIALIZZAZIONE
Ruoli mirati: code reviewer, debugger, ricercatore.

PARALLELISMO
Compiti indipendenti insieme, anche in background.

MODELLO ADATTO
Modelli più rapidi o economici per i compiti semplici.

COME SI USANO E SI CREANO

DELEGA AUTOMATICA
Claude sceglie in base al compito e alla description.

RICHIESTA NEL PROMPT
"Usa Explore per mappare il modulo pagamenti."

PERSONALIZZATI
Chiedere a Claude di crearli o scrivere un file `.md`.
Intestazione YAML + istruzioni del ruolo.Came; name, model, effort...

Progetto: `.claude/agents/`
Utente: `~/.claude/agents/`

/agents: indicazioni per crearli e gestirli · **/tasks:** attività in esecuzione

`/agents` **stampa solo un promemoria** (chiede a Claude di crearli, o di modificare direttamente `.claude/agents/`); il wizard interattivo esisteva fino alla v2.1.197 e non è più disponibile.

Due sessioni Claude Code in due terminali

Che cosa succede davvero se apri Claude due volte

Terminale 1

```
C:\progetto> claude
```

Sessione A

Conversazione indipendente

Due conversazioni distinte

Terminale 2

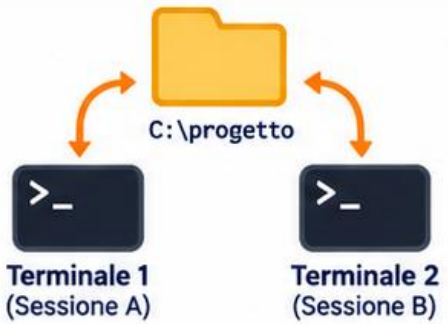
```
C:\progetto> claude
```

Sessione B

Conversazione indipendente

- Ogni terminale avvia una sessione indipendente
- La seconda sessione NON eredita automaticamente tutta la conversazione della prima
- Entrambe possono però leggere gli stessi file del progetto

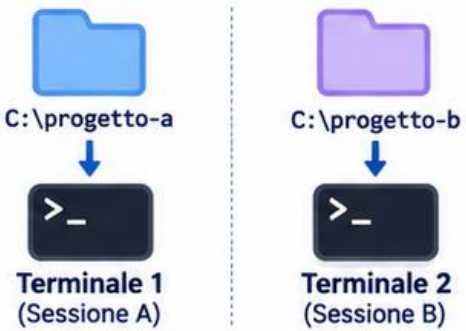
Se entrambe lavorano nella STESSA cartella



- Stessi file sul disco
- Rischio di sovrascrivere modifiche
- Possibili interferenze se entrambe cambiano gli stessi file
- Utile solo se i compiti sono ben separati

Attenzione: stessa cartella ≠ collaborazione coordinata

Se usi cartelle / worktree SEPARATI



- Ambienti di lavoro isolati
- Meno confiitti
- Più adatto al lavoro parallelo
- Ideale per esperimenti o task diversi

Scelta consigliata per lavoro parallelo

Caso più comune consigliato

Terminale 1

```
C:\progetto> claude
```

Claude Code aperto per sviluppare

Terminale 2

```
C:\progetto> uv run python
   scripts/serve_docs.py
C:\progetto> uv run pytest --cov
```

Comando per server docs o test

Meglio una sola sessione Claude + un secondo terminale per comandi o server

Se vuoi riprendere la stessa conversazione

```
C:\progetto> claude --resume
```

1. Chiudi o lascia inattiva la prima sessione
2. Apri la sessione nel nuovo terminale
3. Evita di usare la stessa sessione attivamente in due terminali insieme

Messaggio chiave

- Due claude = due sessioni
- Stessa cartella = stessi file
- Per lavorare bene in parallelo: separa i contesti

Conclusione: aprire Claude Code in due terminali non crea due agenti sincronizzati; crea due sessioni separate che possono interferire se toccano gli stessi file.

Il vibe coding

Evoluzione del coding con l'AI

Dalla programmazione tradizionale al vibe coding

1 Programmazione manuale o tradizionale

pre-AI

Il codice viene scritto a mano dal programmatore.

Ruolo umano
Scrive tutto il codice.

Ruolo dell'AI
Assente.

2 Completamento automatico del codice

dal 2021

L'IDE suggerisce o completa porzioni di codice basandosi su ciò che è stato digitato.

Ruolo umano
Scrive il codice e accetta o modifica i suggerimenti.

Ruolo dell'AI
Suggerisce completamenti locali.

3 Programmazione a chat
chat coding

dal 2023

Si chiede all'AI di generare o modificare codice in una conversazione turno per turno.

Ruolo umano
Formula richieste e integra le risposte.

Ruolo dell'AI
Genera codice e propone modifiche.

4 Programmazione agentica
agentic coding

inizio 2025

L'agente AI modifica file, usa strumenti, esegue test e lavora in modo più autonomo.

Ruolo umano
Assegna obiettivi e supervisiona.

Ruolo dell'AI
Implementa, testa e perfeziona.

5 Vibe coding

oggi / emergente

Il programmatore delega molto della scrittura del codice all'AI, ma resta attivo: supervisiona, valuta e critica i risultati.

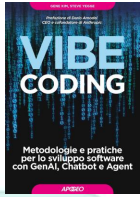
Ruolo umano
Definisce obiettivi, supervisiona e valida.

Ruolo dell'AI
Scrive gran parte del codice e propone soluzioni.

! Il Vibe Coding non è una nuova tecnologia separata, ma un nuovo modo di lavorare con gli strumenti AI, soprattutto quelli agentici

Come cresce la delega all'AI

In sintesi: l'evoluzione non elimina il programmatore; cambia il suo ruolo, da autore diretto del codice a supervisore sempre più strategico.



Origine del termine ‘vibe coding’

Sintesi basata solo sulle informazioni emerse in questa chat

1 Che cosa sappiamo sull’origine

> Il termine viene attribuito ad Andrej Karpathy per descrivere un nuovo modo di programmare.

2 Chi viene indicato come autore del termine

Andrej Karpathy

È indicato come la persona che ha coniato l’espressione ‘vibe coding’.

L’attribuzione del termine viene riportata nelle informazioni considerate qui.

3 Che cosa significa ‘vibe coding’

- L’AI scrive il codice per noi.
- Non si digita più il codice a mano come attività centrale.
- L’AI aiuta lungo tutto il ciclo di vita del software: architettura, soluzioni, implementazione, test, sicurezza.
- La persona dirige, verifica e mantiene la visione; l’AI si fa carico dell’implementazione.

In sintesi: è vibe coding quando si dirige più di quanto si digiti.

4 Che cosa NON sappiamo da queste informazioni

- Non è indicato quando il termine è stato coniato.
- Non è indicato dove è stato introdotto.
- Non è spiegato perché sia stato scelto il nome ‘vibe coding’.
- Non sono disponibili qui i dettagli storici completi del conio.

Importante: questa infografica riporta solo le informazioni emerse nella chat, senza aggiunte esterne.

Messaggio chiave: il termine è attribuito a Andrej Karpathy, ma qui non sono disponibili i dettagli storici completi del conio.

I 5 benefici del vibe coding: FAAFO

Il vibe coding ti permette di realizzare "cose" più velocemente, di essere più ambizioso su ciò che puoi creare, di farlo con maggiore autonomia, di divertirti di più e di esplorare più opzioni. Questi cinque aspetti sono riassunti nell'acronimo FAAFO.

F — Faster

Scrivi codice più velocemente.

- Compiti che prima richiedevano mesi o settimane, ora possono essere svolti in un giorno.
- Compiti che richiedevano giorni, ora possono essere completati in ore.
- L'accelerazione deriva non solo dalla generazione di codice, ma anche dall'aiuto dell'AI con debugging, test e documentazione.
- Progetti rimasti in sospeso per anni possono finalmente vedere la luce.

A — Ambitious

Puoi essere più ambizioso su ciò che puoi realizzare.

- Porta alla tua portata progetti apparentemente impossibili.
- Rende più facili da affrontare anche piccoli compiti con ROI marginale.
- Questo è dovuto alla velocità, alla vasta conoscenza e alle capacità dell'AI.
- Rimodella il tuo approccio allo sviluppo, eliminando molti dei dolorosi compromessi che da sempre limitano ciò che viene realizzato.

A — Autonomously

Puoi lavorare con maggiore autonomia.

- Puoi completare cose che prima richiedevano più persone o team.
- Funzionalità che un tempo richiedevano specialisti di più discipline possono essere gestite da un singolo sviluppatore non specializzato.
- Elimina due costose "tasse": riduce i costi di coordinamento (riunioni, allineare le priorità, aspettare la disponibilità) e le sfide comunicative.
- Lavorare in modo più autonomo con l'AI riduce o rimuove significativamente questi ostacoli.

F — Fun

La programmazione diventa più divertente.

- Ti risparmia le parti meno piacevoli della programmazione, come il debugging degli errori di sintassi, la lotta con librerie sconosciute o il cambio dell'infrastruttura di test.
- Puoi concentrarti sulla risoluzione dei problemi degli utenti, sulla realizzazione di cose interessanti e sul portare a termine i compiti.
- Lavorare con l'AI è anche stranamente avvincente.
- Sta allontanando le persone dal pensionamento, sta attirando non programmatori e sta incoraggiando i leader ad assumersi una quota maggiore del lavoro di programmazione.

O — Options

Puoi esplorare più opzioni.

- Aumenta la tua capacità di esplorare opzioni, sia per trovare una soluzione sia per mitigare i rischi.
- Invece di impegnarti in un unico approccio, puoi prototipare rapidamente molteplici modi per risolvere il problema.
- Puoi valutare i loro compromessi.
- Quando riconosci un problema in cui l'esplorazione aiuterebbe, puoi attivare di riflesso indagini parallele.

FAAFO: un modo di lavorare più veloce, più ambizioso, più autonomo, più divertente e con più opzioni.

Vibe coding per persone adulte

Mentre gli influencer di YouTube catturano i titoli generando simulatori di volo della Seconda guerra mondiale con un singolo prompt, noi ci concentriamo sul portare il vibe coding nell'ingegneria del software professionale. Questo richiede l'applicazione di pratiche ingegneristiche disciplinate, lasciando comunque all'AI il compito di gestire i noiosi dettagli implementativi. In altre parole, **è vibe coding per persone adulte.**

1 Che cosa significa

Fare tutte quelle cose da persone adulte di cui potreste già essere responsabili.

- Revisioni della sicurezza.
- Copertura dei test.
- Gestione del raggio d'esplosione.
- Eccellenza operativa.

2 Velocità e risultati

Lo fate a una velocità che nessuno di noi ha mai sperimentato prima, creando migliaia (potenzialmente anche decine di migliaia) di righe di codice al giorno.

3 Applicazioni reali

Quando lavorate all'autenticazione per un'applicazione rivolta ai clienti, esaminerete ancora ogni riga di codice e realizzerete suite di test complete – ma potete farlo molto più velocemente.

4 Sistemi legacy

Per i sistemi legacy che nessuno ormai comprende più, potreste usare l'AI per analizzare e documentare la base di codice e poi per realizzare test per catturare il comportamento esistente; solo allora potrete iniziare a fare modifiche con una certa fiducia.

5 Esempio concreto

Ecco un ottimo esempio di modifica di codice legacy: il ricercatore Microsoft Jonathan Larson ha dimostrato l'uso di LLM e GraphRAG per modificare il codice sorgente di DOOM di id Software (anno 1993), abilitando la possibilità per il giocatore di saltare. È stata un'impresa non banale, perché. il motore originale non ha un vero modello 3D interno ed era costruito sul presupposto che il giocatore fosse sempre poggiato a terra. La modifica ha coinvolto molti sottosistemi fortemente accoppiati, tra cui la fisica, lo stato del giocatore, la gestione degli input e la logica di livello.[13]

6 Il vostro ruolo

Si tratta di prendere la vostra disciplina ingegneristica così duramente guadagnata e applicarla con maggiore intensità. Voi siete l'head chef, e il vostro ruolo è quello di stabilire gli standard, assaggiare rigorosamente e assicurare che ogni piatto soddisfi i vostri standard, perché, se la cucina accelera, aumenta notevolmente anche la potenziale frequenza e dimensione degli errori.

7 Il momento è adesso

Come sottolinea Karpathy, questi strumenti AI stanno migliorando esponenzialmente. Quelli di oggi sono i meno avanzati che vedrete. Con questo in mente, crediamo sia giunto il momento di andare oltre la scrittura meticolosa di ogni riga di codice per adottare pienamente questo nuovo approccio allo sviluppo software.

Ecco qualcosa in cui crediamo sinceramente: nessuno dovrebbe più scrivere codice a mano se non è necessario.

Quali competenze imparare

Il mondo sta cercando di capire che cosa cambia e non cambia quando uno sviluppatore usa l'AI su tutto ciò su cui sta lavorando, e quali competenze sono le più importanti in questo nuovo mondo. Poiché gli strumenti si evolveranno rapidamente, **i principi di fondo dell'ingegneria del software tradizionale** giocheranno un ruolo almeno altrettanto grande, se non maggiore, di prima. Pertanto, è essenziale:

1 Creare cicli di feedback veloci e frequenti

Creare cicli di feedback veloci e frequenti per la **validazione** e il **controllo.**

2 Curare la modularità

Curare la modularità per **ridurre la complessità**, abilitare il **lavoro parallelo** ed **esplorare opzioni.**

3 Accogliere l'apprendimento

Accogliere l'apprendimento in un mondo in cui **tutto cambia velocemente.**

4 Padroneggiare la professione

Padroneggiare la professione, per prosperare in un ambiente in cui in un breve lasso di tempo tutto il lavoro della conoscenza **cambierà in modo radicale.**

Impairare queste tecniche sarà critico **per tutti coloro che operano nel campo della conoscenza**, non solo per gli sviluppatori e i vibe coder.

Il ciclo del vibe coding

Le fasi del ciclo e il passaggio bonus di automazione

Prima di entrare nell'esempio di Gene sulla scrittura di un generatore di estratti video, parliamo del *ciclo del vibe coding*. Ecco le sue fasi.

1 Inquadrare l'obiettivo

Fornite al vostro assistente AI una panoramica chiara e concisa del risultato che state cercando. Siate specifici sull'aspetto del successo e sul perché lo state costruendo.

2 Scomporre i compiti

Suddividete ciò che state cercando di fare in passaggi chiari e realizzabili. In generale, più piccoli sono i passaggi, maggiori sono le possibilità che l'AI abbia successo. Anche man mano che l'AI diventa più abile, è sempre una buona idea specificare passaggi piccoli. Non esitate a chiederle di suddividere i grandi compiti (per esempio: 'Ecco cosa sto cercando di fare. Proponimi un piano.').

3 Iniziare la conversazione

Chiedete all'AI di generare un piano per raggiungere il vostro obiettivo, o datele istruzioni per iniziare.

4 Revisionare con attenzione

La soluzione che la vostra AI proporrà potrebbe sembrare corretta, ma fino a quando non avrete stabilito una base di fiducia, dovete ricontrollarla.

5 Collaudare e verificare

Siete sempre voi i responsabili della qualità del codice, anche se l'ha scritto l'AI. Tutto funziona meglio se scrivete i vostri test e le vostre aspettative prima di generare il codice, come raccomandano i sostenitori dello sviluppo guidato dai test (TDD). Fallite velocemente, correggete velocemente e chiedete all'AI di aiutarvi a individuare quegli errori subdoli che potrebbero rimanere inosservati.

6 Perfezionare e iterare

Continuate a iterare finché non raggiungerete il vostro obiettivo. Questo ciclo del vibe coding somiglia al ciclo di sviluppo tradizionale. Ma quando si programma con l'AI, ogni passaggio diventa critico. Non potete addormentarvi al volante, o presto vi ritroverete con un frustrante e costoso rifacimento.

BONUS

7 Automatizzare il flusso di lavoro

Iniziate ad automatizzare parti del vostro flusso di lavoro. Qualsiasi attrito crea enormi costi-opportunità. Tutto il tempo che passate a digitare o copiare/incollare/spostare rallenta il ciclo di vibe coding. Se state facendo qualcosa manualmente, ricordatevi che questo è un costo, che pagate ogni volta che provate a fare vibe coding.

Automatizzare questa fatica non solo vi renderà più veloci, ma accelererà la vostra capacità di sperimentare e innovare.

Se, in qualsiasi momento, vi trovate a digitare molto o a cercare manualmente in strutture dati, fermatevi e chiedetevi: 'Potrei chiedere all'AI di aiutarmi?' La risposta di solito è sì, e così sarete più veloci e vi divertirete di più.

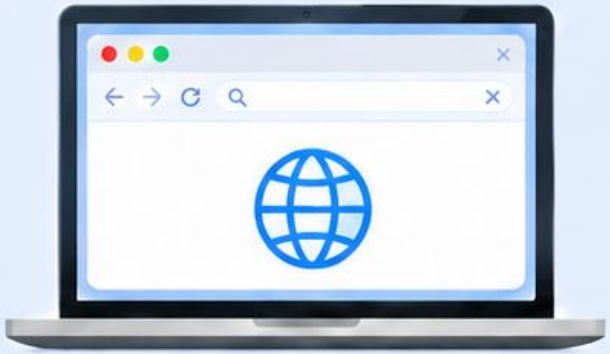
NOTA — Consigliamo vivamente di utilizzare un gestore di Appunti, se già non lo state facendo. Con il vibe coding ci si trova spesso a passare continuamente da uno strumento all'altro. Tra i gestori di Appunti più noti troviamo Alfred o Paste per macOS, Ditto o ClipClip per Windows, CopyQ per Linux (ma è multi-piattaforma) o la cronologia degli Appunti integrata in Windows (Win+V). Attualmente i gestori di Appunti offrono notevoli miglioramenti al flusso di lavoro che impiega l'ingegneria dei prompt e la gestione del contesto.

Le mani sullo strumento

goDeploy: IL LABORATORIO NEL BROWSER

Esercitazioni pratiche anche sui PC che non consentono installazioni

1 · PC DEL PARTECIPANTE



Browser + connessione Internet

Accesso con le credenziali del laboratorio. Nessuna installazione degli strumenti del corso sul PC.

Accesso al laboratorio

lms.godeploy.it

Accesso al desktop remoto

2 · VM goDeploy DEL CORSO

Ambiente predisposto per le esercitazioni

VM OFFERTA PER 6 MESI

Claude Code | CLI / IDE | Miniconda | VS Code | GitHub Desktop

File del progetto | Build e test

File, comandi e test risiedono nella VM.

Richieste e risposte AI

3 · SERVIZIO AI REMOTO

Modello Claude

Inferenza e generazione delle risposte.

ACCESSO SEMPLIFICATO
Esercitazioni anche quando il PC non consente installazioni.

AMBIENTE COERENTE
Strumenti e configurazione predisposti per il corso.

PIÙ PRATICA
Meno tempo dedicato al setup dei singoli PC.

ISOLAMENTO E SICUREZZA
La VM è isolata dal PC del partecipante, per una maggiore protezione.

COME VIENE USATO CLAUDE CODE

Il browser visualizza il laboratorio; Claude Code lavora nella VM e si collega al modello remoto.

Software e accesso a Claude vanno predisposti secondo la configurazione del corso.

Fonti: godeploy.com · code.claude.com/docs · Schema per il corso Claude Code · 21/10/2026

Strumenti da installare per questo corso

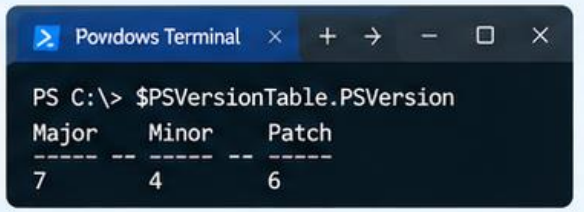
Ordine di installazione e scopo di ogni strumento

Prerequisiti, non installazioni
Windows 10 1809+ (o Server 2019+), 4 GB+ di RAM, connessione a internet, account Claude a pagamento (Pro, Max, Team/Enterprise o Console).
Nessun permesso di amministratore richiesto.
Nota: Node.js non serve — l'installer nativo scarica un binario autonomo.

1 Windows Terminal con PowerShell 7 (pwsh)

Consigliati, non obbligatori

Ambiente da cui si farà tutto il resto.
PowerShell 7 (**pwsh**) è anche la versione che il PowerShell tool di Claude Code preferisce, con ripiego sul 5.1 di sistema.



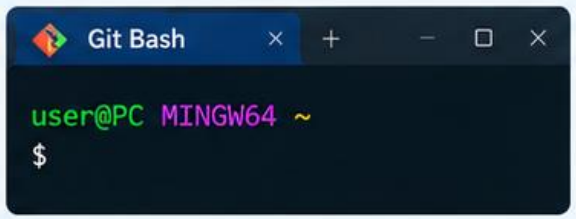
Windows Terminal con PowerShell 7 (pwsh)

2 Git for Windows (include Git Bash)

Un solo installer installa due componenti:

- **Git**, per gestire il versionamento del codice.
- **Git Bash**, che permette a Claude Code di eseguire comandi Bash tramite il **Bash tool** e di utilizzare il Monitor tool.

Sono impostazioni predefinite dell'installer.

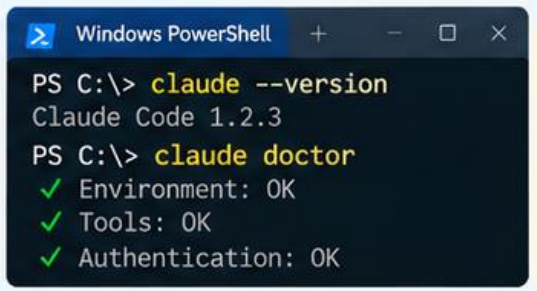


Git for Windows + Git Bash (già configurati)

3 Claude Code (tramite PowerShell 7)

Installa Claude Code e verifica il corretto funzionamento.

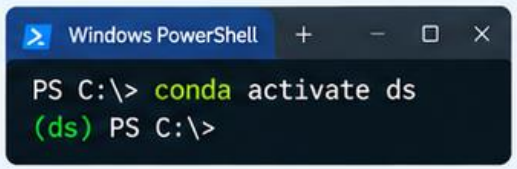
- Aprire un nuovo terminale (PowerShell 7).
- Poi digitare: `claude --version`
- Eseguire: `claude doctor`
- Gli aggionamenti vengono scaricati automaticamente in background.
- **Primo avvio e accesso** (passo, non installazione) — eseguire `claude` e completare il login dal browser (vedi prossima slide).



Claude Code installato e verificato

4 Anaconda (distribuzione Python)

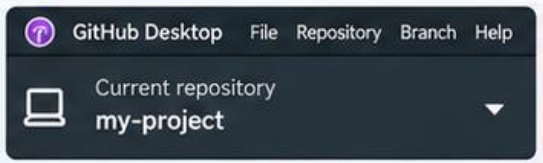
Fornisce Python e gli strumenti per creare e gestire ambienti virtuali con conda, utili nelle esercitazioni.
Attivare l'ambiente conda prima di avviare Claude Code, oppure usare `conda run -n <ambiente> ...` nei comandi, perché il PowerShell tool di Claude Code non carica i profili di PowerShell, cioè gli script di inizializzazione eseguiti automaticamente all'avvio della shell).



Ambiente conda attivato

5 GitHub Desktop

Interfaccia grafica per lavorare con i repository su GitHub, semplice da usare e adatta al corso.
Non è la GitHub CLI e non è obbligatoria, ma è consigliata per gestire il materiale didattico in modo pratico.



GitHub Desktop (gestione repository)

PERCHÉ L'ACCESSO PASSA DAL BROWSER

Claude Code da CLI: primo login, token e accessi successivi

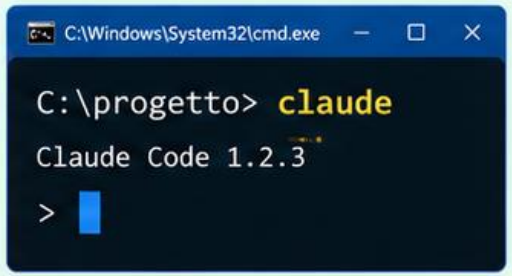
1 · LA PRIMA VOLTA

1. Si esegue il comando nel terminale. La CLI apre automaticamente claude.ai nel browser.
2. Si effettua l'accesso nel browser (con account Anthropic, SSO aziendale, Google, passkey, ecc.).
3. Dopo l'autorizzazione, la CLI memorizza le credenziali OAuth sul PC, non la password.

Credenziali OAuth salvate localmente sul PC

2 · DALLA SECONDA VOLTA IN POI

Si esegue `claude` : la CLI riparte direttamente senza riaprire il browser, finché l'autenticazione resta valida.



Accesso già attivo

- Nessun browser
- Nessuna password
- La CLI utilizza i token salvati localmente

DUE TOKEN, DUE FUNZIONI

La CLI salva due token sul PC, con ruoli diversi.

TOKEN DI ACCESSO

- Usato per le richieste.
- Ha durata breve.

Rinnovo silenzioso: normalmente senza browser.

TOKEN DI RINNOVO (refresh token)

- Conservato in modo protetto sul PC.
- Consente alla CLI di ottenere in automatico un nuovo token di accesso quando serve.

QUANDO IL BROWSER PUÒ RICOMPARIRE

In alcuni casi è necessario autenticarsi di nuovo nel browser.

1. Logout o cambio account

Dopo `/logout` o `claude auth logout` occorre autenticarsi di nuovo.

2. Sessione non più valida

Token revocato o non rinnovabile, credenziali cambiate oppure policy SSO aziendale: è richiesto un nuovo login.

3. Nuova macchina

Ogni PC o VM ha il proprio accesso iniziale. In aula: un login per partecipante e postazione; nei giorni successivi non si ripete se le credenziali restano valide.

Le policy aziendali possono richiedere riautenticazioni periodiche.

CONTROLLO RAPIDO

`claude auth status`	= Stato e metodo di autenticazione, da CMD o PowerShell.
`/status`	= Informazioni su account e sessione, dentro Claude Code.

Se una sessione è revocata, l'interfaccia può richiedere nuovamente il login.

COME L'APP DI UNA BANCA

L'identificazione completa avviene all'inizio; in seguito si accede senza ripeterla, finché la sessione resta valida. Cambiando dispositivo o dopo una revoca, va rifatta.

Il terminale non riceve la password: utilizza una delega revocabile e rinnovabile.

Fonti: Claude Code Docs — Authentication e CLI reference · 21/10/2026

WINGET SU WINDOWS

Che cos'è, a cosa serve nel corso e come si usa

CHE COS'È

WinGet è il package manager di Microsoft per Windows.

Permette di cercare, installare, aggiornare e rimuovere software dal terminale.

È simile ad apt, brew o pip.

LINK UFFICIALE

learn.microsoft.com/windows/package-manager/winget/

Su molti PC Windows 10/11 è già disponibile tramite App Installer.

Verifica rapida:

```
winget --version
```

PERCHÉ È UTILE IN QUESTO CORSO

- velocizza l'installazione dei tool
- rende i passaggi ripetibili su più PC
- evita di cercare manualmente ogni installer
- aiuta a installare rapidamente:
  - PowerShell 7
  - Git for Windows
  - GitHub CLI
  - altri strumenti del corso

Se WinGet non è disponibile, si possono usare gli installer ufficiali.

COME SI USA

Sintassi tipica:

```
winget <azione> [opzioni]
```

1	Cercare un pacchetto	`winget search powershell`
2	Installare PowerShell 7	`winget install --id Microsoft.PowerShell -e`
3	Installare Git for Windows	`winget install --id Git.Git -e --source winget`
4	Vedere i pacchetti installati	`winget list`
5	Aggiornare tutto	`winget upgrade --all`

ELEMENTI IMPORTANTI

- **--id** = identificatore del pacchetto
- **-e** = corrispondenza esatta
- **--source winget** = repository usato

IN SINTESI

Per il corso, WinGet è il modo più rapido e uniforme per preparare l'ambiente Windows da CLI.

CLAUDE CODE DA CLI SU WINDOWS

Ordine consigliato per il corso • CMD per il lancio, PowerShell 7 per i comandi

PRIMA DI INIZIARE

Minimi Claude Code: Windows 10 1809+ / Windows 11 • x64 o ARM64 • RAM 4 GB+ • Internet
Accesso: Pro, Max, Team, Enterprise oppure Console con fatturazione.
Se WinGet non è disponibile, usare gli installer ufficiali.

1 POWERSHELL 7

Dal Windows PowerShell già presente:

```
winget install --id Microsoft.PowerShell -e
```

Aprire PowerShell 7 e verificare:

```
pwsh --version
```

Esito: PowerShell 7.x.

2 GIT FOR WINDOWS

Richiesto per il corso. Include già Git e Git Bash.

Da PowerShell 7:

```
winget install --id Git.Git -e --source winget
```

In una nuova finestra PowerShell 7:

```
git --version
```

Esito: `git version …`

Git Bash resta disponibile per gli script Bash.

3 CLAUDE CODE

Da PowerShell 7, installer nativo:

```
irm https://claude.ai/install.ps1 | iex
```

Poi, in una nuova finestra PowerShell 7:

```
claude --version
claude doctor
```

Node.js non necessario.
Aggiornamenti automatici di default.

4 IMPOSTARE POWERSHELL

File utente: `%USERPROFILE%\.claude\settings.json`

Creare il file o unire alle impostazioni esistenti:

```
{
    "env": {
        "CLAUDE_CODE_USE_POWERSHELL_TOOL": "1"
    },
    "defaultShell": "powershell"
}
```

`%USERPROFILE%`
`\.claude\settings.json`

Il tool usa PowerShell; defaultShell vale per i comandi !.

5 AVVIARE DA CMD E COLLAUDARE

Aprire un nuovo CMD nella cartella del progetto:

```
claude
```

Effettuare l'accesso nel browser.
Poi chiedere nella conversazione:

```
Usa il tool PowerShell ed esegui $PSVersionTable.PSVersion
```

Verificare la chiamata al tool PowerShell e la versione 7.x.

SECONDO LE ATTIVITÀ DEL CORSO

Windows Terminal
contenitore facoltativo per CMD e PowerShell.

Miniconda / Anaconda
per gli esercizi Python.

VS Code + GitHub Desktop
editor e interfaccia grafica per Git.

GitHub CLI (gh)
per operazioni GitHub e pull request.

NOTE PRATICHE

- **Conda:** attivare l'ambiente prima di lanciare claude; il tool PowerShell non carica i profili.
- **WSL2:** percorso Linux alternativo, con installazione di Claude all'interno della distribuzione.
- **Permessi:** l'installer nativo di Claude non richiede admin; altri installer possono richiederlo.

Fonti: code.claude.com/docs • learn.microsoft.com • gitforwindows.org | Verificato il 21/10/2026

Git, GitHub e strumenti Windows: chi fa cosa?

Per un corso su Claude Code da CLI su Windows

1 I cinque termini

Git

Il sistema di version control vero e proprio.

- Gestisce repository locali, commit, branch, merge e tag
- Funziona da riga di comando
- È il motore di base: gli altri strumenti spesso lo usano

Categoria: **tecnologia / motore**

Si collega a repository remoti

GitHub.com

La piattaforma web/cloud per ospitare repository Git e collaborare.

- Repository remoti online
- Pull request, issue, review e condivisione
- Accesso via browser

Categoria: **servizio online**

Usa Git (e si collega a GitHub.com)

GitHub Desktop

Applicazione grafica per usare Git e GitHub senza lavorare sempre da terminale.

- Clone, commit, branch, fetch, pull, push
- Molto comodo per chi preferisce una GUI
- Copre le operazioni più comuni, ma non tutte quelle avanzate

Categoria: **applicazione desktop**

Usa Git (installato con Git for Windows)

Git for Windows

Il pacchetto che porta Git su Windows.

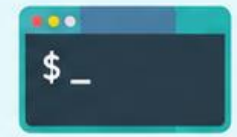
- Installa Git e i comandi Git su Windows
- Permette di usare Git da cmd, PowerShell, VS Code Terminal ecc.
- Include normalmente anche Git Bash

Categoria: **installazione / distribuzione**

Include normalmente Git Bash

Git Bash

Un terminale Bash-like per Windows, fornito con Git for Windows.



- Permette di usare Git in un ambiente simile a Linux/Unix
- Utile a chi preferisce comandi e convenzioni Bash
- È opzionale: su Windows si può usare anche cmd o PowerShell

Categoria: **shell / terminale**

Git ≠ GitHub

Git è il sistema di versionamento (locale, sul tuo computer)

GitHub.com è la piattaforma online (per ospitare e collaborare)

2 Sovrapposizioni e ridondanze

Git vs GitHub.com	**Non sono la stessa cosa:** Git è il sistema di versionamento; GitHub.com è un servizio online che ospita repository Git.
GitHub Desktop vs Git da CLI	**Sovrapposizione parziale:** molte operazioni comuni si possono fare in entrambi. La CLI però resta più flessibile e più adatta all'automazione e ai corsi su Claude Code.
Git for Windows vs Git Bash	**Non sono equivalenti:** Git for Windows è il pacchetto completo; Git Bash è uno degli strumenti inclusi. Installare Git Bash separatamente non serve.
Git Bash vs cmd / PowerShell	**Ridondanza parziale:** tutti possono lanciare Git. Git Bash offre un ambiente più Unix-like; cmd e PowerShell sono più naturali per Windows.

3 Cosa serve davvero per un corso su Claude Code da CLI su Windows?

Necessario	• Git for Windows • Una shell: cmd oppure PowerShell
Utile ma facoltativo	• **GitHub Desktop**, se si vuole anche una GUI
Facoltativo ma spesso consigliato	• **GitHub.com**, se si desiderano repository remoti, backup, condivisione o collaborazione

In sintesi: per lavorare da CLI su Windows, l'elemento chiave è **Git for Windows**. **GitHub Desktop** è un aiuto **grafico**, **GitHub.com** è il servizio online, **Git Bash** è solo una delle shell possibili.

GIT FOR WINDOWS

RICHIESTO PER IL CORSO

Installazione e verifica per Claude Code su Windows nativo

Opzionale per avviare Claude Code; richiesto nell'ambiente del corso.

Git for Windows è un pacchetto software che porta il sistema di controllo di versione Git sul sistema operativo Windows

Include già Git al suo interno, che quindi non deve essere installato separatamente

1 PERCHÉ SERVE

Git
Versionamento: commit, diff, branch e merge.

Git Bash
Shell Bash e strumenti POSIX per Windows.

Bash e Monitor*
Richiedono Git Bash su Windows nativo.

Quando il PowerShell tool è abilitato, PowerShell è la shell primaria.
Senza Git Bash, Claude Code usa PowerShell.

*Monitor: disponibilità soggetta a provider e configurazione.

2 COME INSTALLARLO

1. Scaricare l'installer da:

   `git-scm.com/downloads/win`

   Oppure, se WinGet è disponibile:

   ```
   winget install --id Git.Git -e --source winget
   ```

2. Accettare le impostazioni predefinite.
3. Aprire un nuovo Prompt dei comandi (CMD) per le verifiche.

Il collegamento Git Bash può mancare nel menu Start: non indica un errore.

3 TRE CONTROLLI

A. In CMD: verificare Git

```
git --version
```

Esito: git version ...

B. In CMD: verificare Bash

```
"C:\Program Files\Git\\bin\bash.exe" --version
```

Esito: GNU bash, version ...
Adattare il percorso se Git è installato altrove.

C. In Claude Code: usare Bash

Avviare una nuova sessione locale e chiedere:

```
Usa il tool Bash ed esegui uname -s
```

Verificare la chiamata al tool Bash.
Output tipico: MINGW...

ATTENZIONE AL PATH

`where bash` può fallire anche con Git Bash installato.
Bash non deve essere nel PATH: Claude lo cerca nei percorsi standard e tramite Git.

AMBIENTE CONDA

Attivare l'ambiente prima di avviare Claude Code:

```
conda activate nome_ambiente
claude
```

Il tool PowerShell non carica i profili, anche con Git installato.
L'ambiente deve essere già creato.

SE GIT BASH NON VIENE RILEVATO

In settings. json, nella sezione env:

```
{
  "env": {
    "CLAUDE_CODE_GIT_BASH_PATH": "C:\\Program Files\\Git\\bin\\bash.exe"
  }
}
```

Indicare bin\bash.exe, adattare il percorso e riavviare Claude Code.

Fonti: git-scm.com • gitforwindows.org • code.claude.com/docs | Verificato il 21/10/2026

Da verificare

WSL 2: Linux dentro Windows

Windows Subsystem for Linux • Il sottosistema Windows per Linux

Che cos'è

Permette di eseguire una distribuzione Linux, come Ubuntu, in Windows 10 e 11.

WSL 2 usa un vero kernel Linux in una macchina virtuale leggera gestita da Windows.

Niente dual boot e nessuna VM da configurare manualmente.

Bash nativo

apt su Ubuntu

Python e strumenti Linux

Docker è utilizzabile con un'installazione e una configurazione dedicate.

Git Bash

Una shell Bash con Git e una selezione di strumenti Unix portati su Windows.

Ambiente di comandi: non è un sistema Linux.

WSL 2

Una vera distribuzione Linux, con kernel, pacchetti e strumenti dell'ecosistema Linux.

Un ambiente Linux affiancato a Windows.

Come iniziare

1. Aprire PowerShell come amministratore.
2. `wsl --install`
3. Riavviare, se richiesto, e completare la configurazione di Ubuntu.

Ubuntu si avvia dal menu Start o da Windows Terminal.

Comando rapido: Windows 11 o Windows 10 versione 2004 e successive.

Nel corso su Claude Code

Quando servono strumenti Linux completi, WSL 2 offre un ambiente adatto a workflow e dipendenze Linux.

Git Bash per la shell; WSL 2 per l'ambiente Linux.

Riferimento tecnico: Microsoft Learn • WSL

GitHub CLI: dal pacchetto al comando gh

GitHub.cli è il nome del pacchetto; gh è il comando da usare

INSTALLARE CON WINGET

```
winget install --id GitHub.cli
```

- `winget` — Gestore di pacchetti di Windows
- `install` — Installa il programma
- `--id GitHub.cli` — Identifica il pacchetto GitHub CLI

GitHub.cli

IDENTIFICATIVO DEL PACCHETTO

È l'ID usato nel catalogo WinGet per installare GitHub CLI.

GitHub CLI

IL PROGRAMMA

Lo strumento ufficiale per usare GitHub dal terminale.

CLI = Command Line Interface

gh

IL COMANDO

È il nome da digitare per eseguire GitHub CLI.

```
gh --version
```

Verifica la versione installata

git • Versionamento

Gestisce commit, branch, merge e sincronizzazione del repository.

```
git status
```

gh • Funzioni GitHub

Gestisce autenticazione, repository su GitHub, issue e pull request.

```
gh auth login
```

Git e GitHub CLI sono complementari.

Installazione riuscita, ma gh non riconosciuto?

Chiudere completamente il terminale, riaprirlo dal menu Start e provare `gh --version`: la sessione precedente potrebbe usare ancora il vecchio PATH.

Claude Code senza Git: si può

Se non serve versionare, basta una normale cartella di progetto

IL FLUSSO ESSENZIALE

1. Aprire la cartella del progetto nel terminale
2. Avviare Claude Code con `claude`
3. Descrivere il lavoro da svolgere

```
C:\progetti\second_brain> claude
```

Non occorre inizializzare un repository Git.

CHE COSA SI PUÒ FARE

Creare e modificare file

Installare le dipendenze

Avviare l'applicazione

Eseguire test

Anche senza repository e senza commit

NON NECESSARI PER QUESTO FLUSSO

Git
versionamento del codice

GitHub Desktop
interfaccia grafica per Git

GitHub.com
repository online

gh
GitHub dal terminale

Sono strumenti da aggiungere quando servono le loro funzioni.

SU WINDOWS

Git for Windows è facoltativo.

Senza Git Bash, Claude Code esegue i comandi tramite PowerShell.

Si può avviare Claude Code anche dal Prompt dei comandi (cmd).

Servono comunque gli strumenti richiesti dall'applicazione, per esempio Python per un progetto Python.

Claude Code lavora sui file. Git aggiunge il versionamento: le due cose sono indipendenti.

Fonte: documentazione ufficiale Claude Code • Configurazione locale su Windows

Come avviare un’app con Claude Code

Basta dare il prompt: Claude Code individua ed esegue i comandi per conto dell’utente.

IL PROMPT DA USARE

“Avvia questa appllicazione. Individua il comando corretto dalla struttura del progetto e dal README. Se mancano dipendenze, installale nell’ambiente del progetto. Se è un’app web, indica l’URL da aprire nel browser.”

Può chiedere conferma in base alla modalità di autorizzazione configurata.

CHE COSA FA CLAUDE CODE

1. Esamina i file e le istruzioni del progetto.

2. Individua il punto di ingresso e prepara l’ambiente.

3. Esegue il comando di avvio.

L’applicazione viene eseguita sul computer.

COMANDI CHE ESEGUE CLAUDE CODE

Python con main.py	`python main.py`
CLI Python gestita con uv	`uv run second_brain`
Web app con script dev	`npm run dev`

Non occorre digitarli manualmente: sono esempi di ciò che Claude Code esegue. Il comando esatto dipende dal progetto.

SE È UN’APP WEB

Claude Code avvia il server locale e comunica l’indirizzo. Aprire nel browser l’URL indicato.

`http://localhost:3000` Esempio: la porta può essere diversa.

Il server deve restare attivo durante l’utilizzo.

COME FERMARLA

“Ferma il server dell’applicazione.”

Se il server occupa un terminale dedicato, interromperlo con `Ctrl+C` in quel terminale.

Non esiste un comando Run universale: Claude Code usa gli strumenti e i comandi previsti dall’applicazione.

claude.com/it/product/claude-code

Installazione di Claude Code

Claude

Prodotto | Team di sviluppo | Enterprise | Risorse | Prezzi | Accedi | Contatta il team commerciale | Prova Claude

Claude Code | Esplora qui

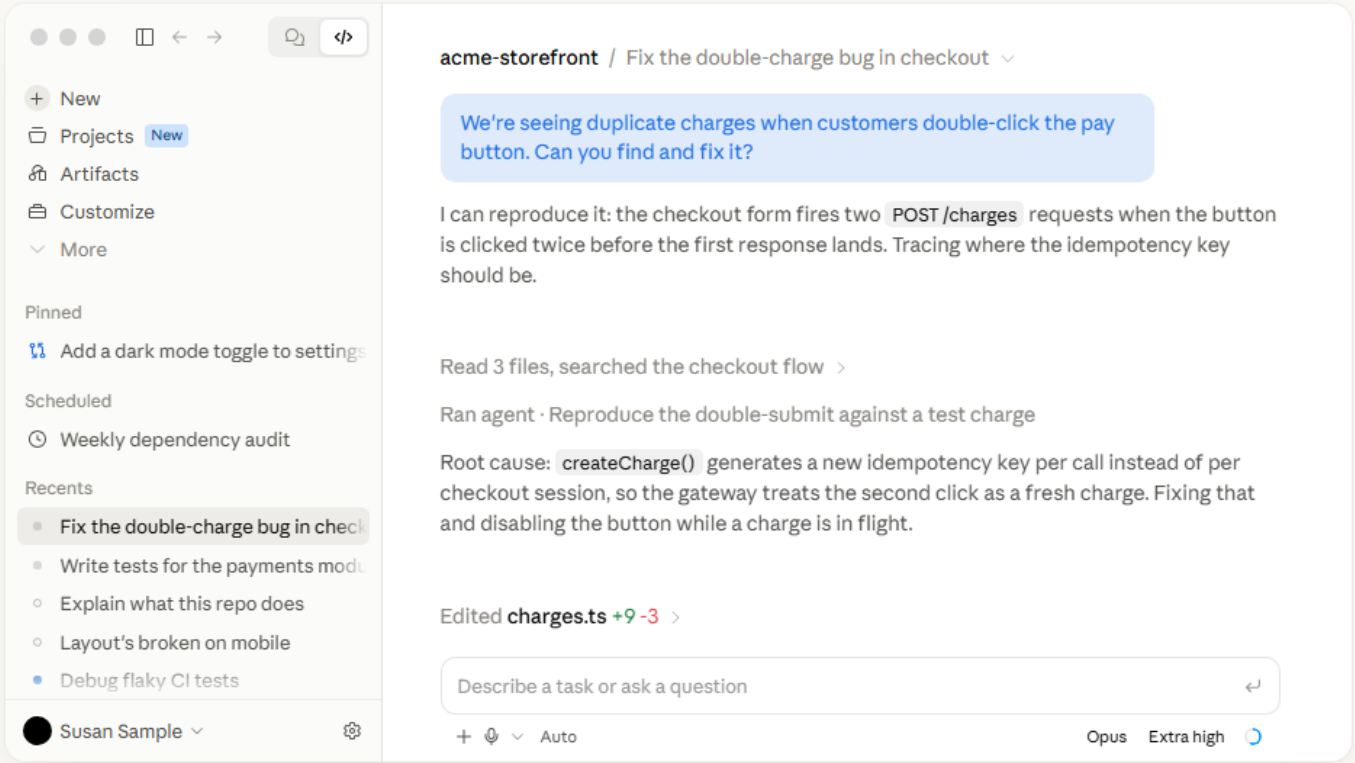
Claude Code

Affida a Claude Code una correzione di bug, un test o una migrazione che richiede giorni. Supervisiona e rivedi dal terminale, dall'IDE, da Slack o dal web.

Scarica per Windows

Consulta la documentazione

Disponibile per macOS, Linux e Windows.

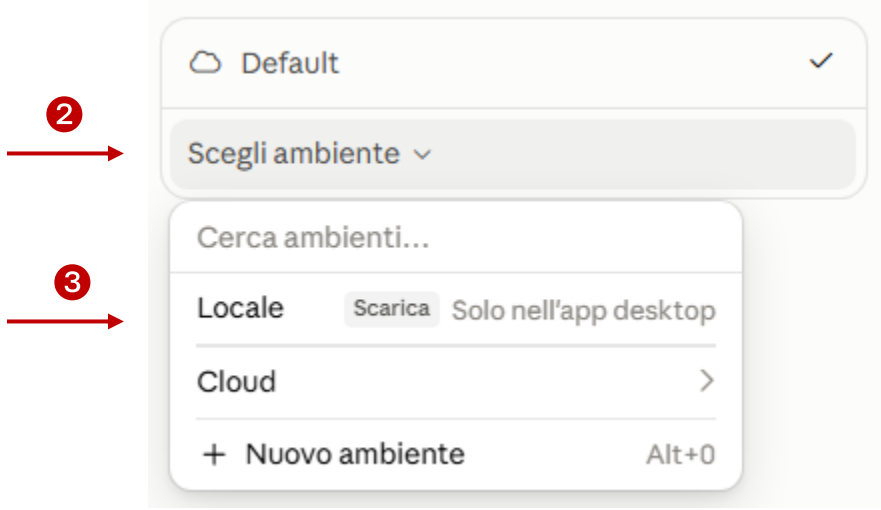


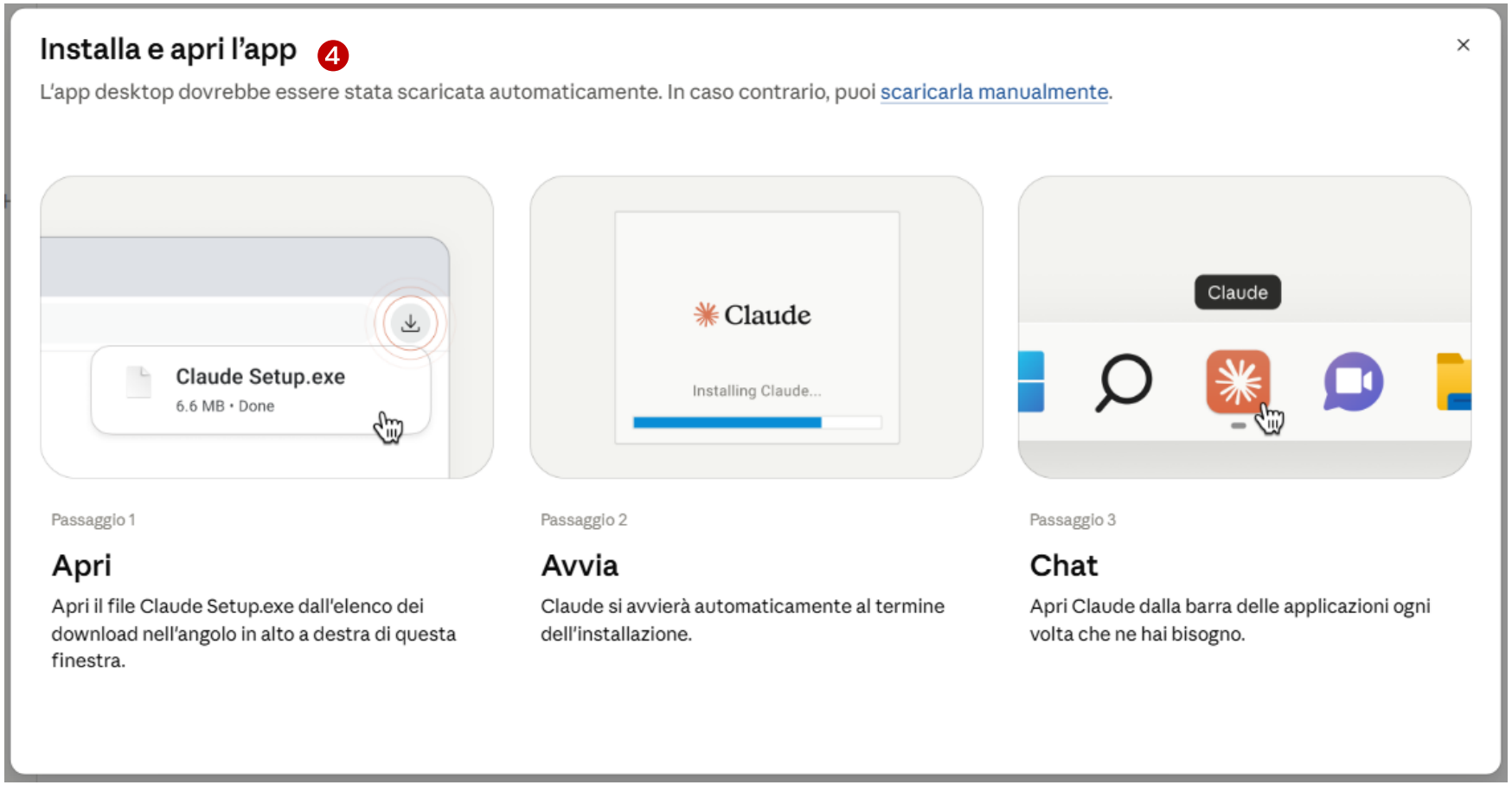
Altri modi per usare Claude Code:

Terminal | Web | iOS | Android | GitHub | VS Code | JetBrains | Slack

attenzione al PATH!, vede Claude?

Installazione di Claude Code da ai





attenzione al PATH!, vede Claude?

- **Via npm**: richiede Node.js (npm arriva in bundle con Node), versione 22 o successiva. Una sfumatura interessante: Node serve *solo per installare* — il pacchetto npm scarica lo stesso binario nativo dell'installer, e il comando `claude` a runtime non usa Node. Su questa macchina il prerequisito era già soddisfatto (Node c'era, come dimostra la directory `Roaming\npm` nel PATH), per questo è filato tutto liscio.
- **Via installer nativo** (`irm https://claude.ai/install.ps1 | iex` su Windows): nessun prerequisito, né Node né altro. È la scelta giusta per chi parte da zero.

Per il corso la regola pratica diventa: *chi ha già Node può usare npm; chi non ce l'ha usa l'installer nativo, senza installare Node solo per questo.* Se in aula qualcuno volesse comunque Node (magari serve anche per gli esercizi), le vie rapide su Windows sono il download da nodejs.org oppure `winget install OpenJS.NodeJS.LTS`.

> *L'installer nativo installa in `~/.local/bin` ma **non** aggiorna il PATH: lo verifica e, se la directory manca, stampa l'avviso con la correzione da eseguire a mano. Su Linux spesso non serve nulla (directory già nel PATH); su Windows tipicamente sì. Con npm il PATH è già a posto, sistemato a suo tempo dall'installazione di Node.*

Via PowerShell (permanente, consigliata)

```powershell
$currentPath = [Environment]::GetEnvironmentVariable('PATH', 'User')
[Environment]::SetEnvironmentVariable('PATH', "$currentPath;$env:USERPROFILE\.local\bi
```

Scrive nel **PATH utente persistente** (quello salvato nel registro di Windows, letto da ogni nuovo processo). Non servono privilegi di amministratore. Poi **aprire un nuovo terminale**: le finestre già aperte conservano il vecchio PATH.

Via interfaccia grafica (per chi preferisce)

1. Tasto Windows → digitare "variabili" → aprire **"Modifica le variabili di ambiente relative all'account"**;
2. selezionare **Path** nella sezione utente → **Modifica** → **Nuovo**;
3. inserire `%USERPROFILE%\.local\bin` → OK su tutto;
4. aprire un nuovo terminale.

Permanente da CMD, in modo sicuro

Due vie:

A. Richiamare PowerShell da dentro CMD (una riga, nessun limite di lunghezza, tocca solo il PATH utente):

```cmd
powershell -NoProfile -Command "[Environment]::SetEnvironmentVariable('PATH', [Environ
```

B. Finestra grafica (la via indicata dalla documentazione): tasto Windows → "variabili" → *Modifica le variabili di ambiente relative all'account* → **Path** → Modifica → Nuovo → `%USERPROFILE%\.local\bin` → OK.

In entrambi i casi, poi, **aprire un nuovo terminale** e verificare con `claude --version`.

Per la slide del corso la sintesi è: *da PowerShell il comando documentato, da CMD la GUI (o il richiamo a PowerShell) — mai `setx` sul PATH*, e il perché del divieto (troncamento a 1024 caratteri) è a sua volta un ottimo aneddoto d'aula.

CLAUDE CODE DA CLI

Due shell, due ruoli diversi

WINDOWS NATIVO

1 SHELL DI AVVIO

Dove si digita claude

CMD | PowerShell | Git Bash

- Avvia Claude Code dalla cartella del progetto.
- Trasmette l'ambiente iniziale: PATH e variabili.
- Poi si interagisce con Claude in linguaggio naturale.

CLAUDE CODE

richiama i tool

2 SHELL DI ESECUZIONE

Dove girano i comandi dell'agente

Bash tool → Git Bash

Disponibile con Git for Windows.

```
Elenco file: ls
```

PowerShell tool → PowerShell

Può coesistere con Bash; se abilitato è primario.

```
Elenco file: Get-ChildItem
```

Senza Git for Windows, i comandi passano al tool PowerShell.

Disponibilità e preferenze dipendono dalla configurazione.

AVVIARE DA CMD NON IMPONE LA SINTASSI CMD

La shell scelta per l'avvio non determina automaticamente quella usata dai tool.

PYTHON E MINICONDA

Attivare l'ambiente desiderato prima di avviare claude.
Il PATH aiuta a trovare interprete e strumenti.

CMD PUÒ COMUNQUE ESSERE RICHIAMATO

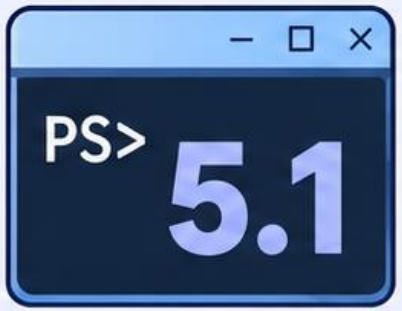
Non è un tool dedicato: un comando può invocare `cmd.exe /c ...`

Fonti: Claude Code Docs — setup, tools-reference, env-vars · Microsoft Learn — cmd · Verifica: 21/10/2026

POWERSHELL E AI AGENTICA

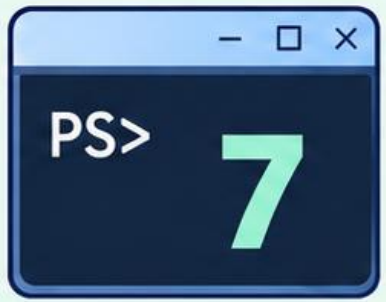
Le differenze che contano per comandi, script e file

WINDOWS POWERSHELL 5.1



- Linea storica, senza nuove funzionalità. Supporto legato al ciclo di vita di Windows.
- Windows • .NET Framework
- Avvio: **powershell.exe**
- **&&** non disponibile
- Codifica predefinita diversa secondo il comando.
- Utile per script e moduli Windows storici.

POWERSHELL 7



- Linea moderna, in evoluzione.
- Windows, Linux e macOS • .NET moderno
- Avvio: **pwsh.exe**
- **&&** disponibile
  Prosegue se il comando precedente riesce.
- Output testuale normalmente UTF-8 senza BOM.
- Scelta consigliata per i nuovi ambienti.

UN ESEMPIO CONCRETO

```
python -m pytest && Write-Output "Test superati"
```

- 5.1: errore di sintassi.
- 7: messaggio mostrato solo se i test riescono.

CON CLAUDE CODE

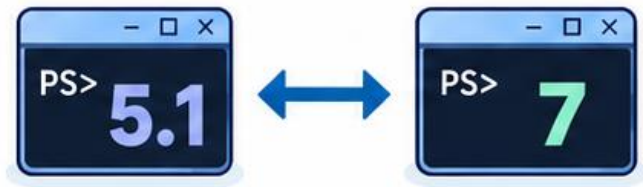
Il tool PowerShell, in preview, quando abilitato preferisce la 7 e ripiega sulla 5.1.

Corregge già alcune differenze di codifica della 5.1.

PER IL CORSO: POWERSHELL 7

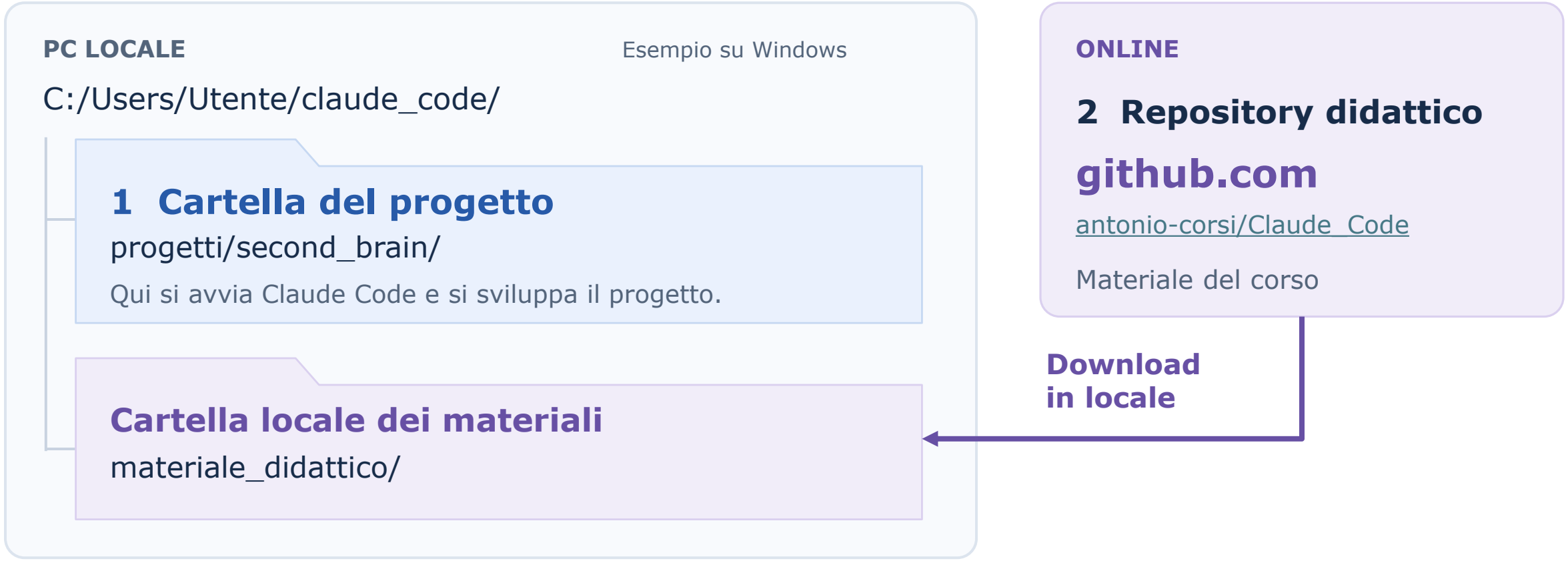
La 5.1 può rimanere affiancata: le due versioni possono coesistere.

La shell che avvia l'agente può essere diversa da quella usata dai suoi strumenti.



I due ambienti del corso

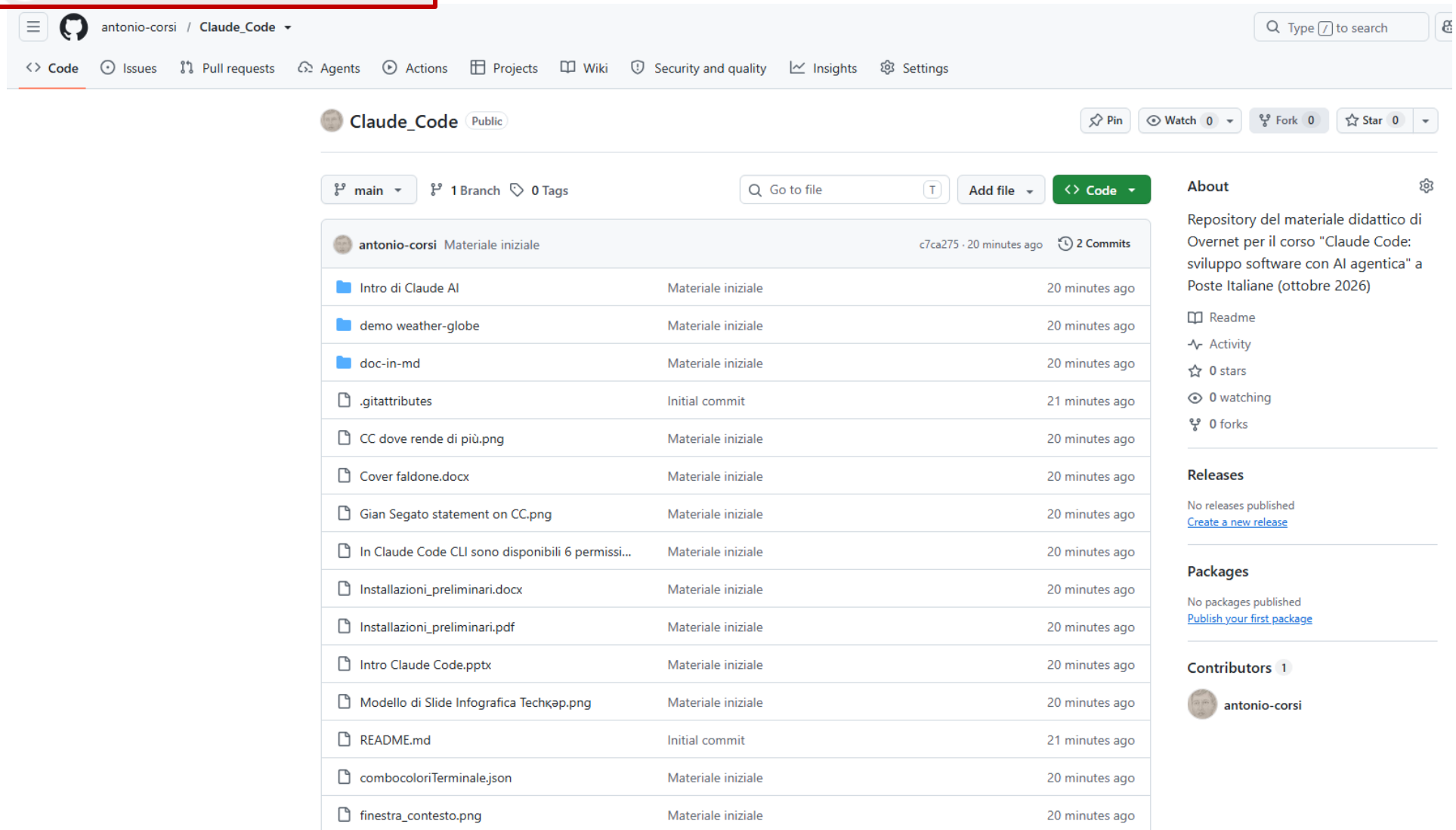
Il progetto si sviluppa in locale. I materiali del corso arrivano da GitHub.



La cartella dei materiali deve restare **fuori dalla cartella del progetto**.

Questo PC › OS (C:) › Utenti › Utente › claude_code › progetti

github.com/antonio-corsi/Claude_Code



Claude Code /init

A cosa serve e quando usarlo

1 Che cos'è

/init inizializza le istruzioni del progetto

- Genera un file **CLAUDE.md** nella cartella di lavoro
- Serve a dare a Claude informazioni persistenti sul progetto

2 Quando è utile

Cartella quasi vuota	Poco utile	Claude ha poco da analizzare.
Scaffolding già creato	Consigliato	Claude riconosce struttura, strumenti e convenzioni.
Progetto Python già esistente	Molto utile	Produce una prima base di CLAUDE.md.
CLAUDE.md già presente	Facoltativo	Può proporre miglioramenti senza essere indispensabile.

3 Cosa NON fa

- **NON** sostituisce git init
- **NON** crea un repository Git
- **NON** crea lo scaffolding applicativo
- **NON** installa dipendenze

/init ≠ git init

4 Esempio pratico su Windows

Prompt dei comandi (cmd)

```
cd C:\progetti\second_brain
claude
```

Sessione Claude

```
/init
```

Poi verificare e rifinire il file CLAUDE.md.

In sintesi

Usare **/init** quando il progetto ha già una struttura minima: aiuta Claude a capire il contesto e a creare una priima memoria di progetto.

Prewarm: cos'è e perché serve

Definizione

Il prewarm (pre-riscaldamento) **prepara in anticipo le risorse** per una richiesta AI, così la risposta successiva parte più velocemente.

Idea principale

Nel prompt caching, il prewarm elabora prima il **prefisso stabile del prompt** e lo mette in cache.

Senza prewarm

Richiesta utente
"... il prompt completo ..."

→

Prefill completo
Elaborazione dell'intero prompt (prefisso + parte nuova).

→

Risposta
"... output ..."

L'utente attende anche l'elaborazione iniziiale del contesto.

Con prewarm

Prompt condiviso
System prompt, istruzioni, tool definitions, ecc.

→

Cache preparata
Il prefisso stabile viene elaborato e salvato in cache.

→

Richiesta utente
"... solo la parte nuova ..."

→

Cache hit
Si riutilizza il prefisso già elaborato.

→

Risposta
"... output ..."

Il prefisso condiviso è già pronto: resta da elaborare solo la parte nuova del prompt.

Beneficio principale: riduzione del TTFT (Time To First Token).

Non accelera necessariamente tutta la generazione successiva.

Utile quando il prompt iniziale è lungo e stabile
(system prompt, tool definitions, istruzioni condivise).

- System prompt
- Tool definitions
- Istruzioni condivise

Prewarm nell'API Anthropic: flusso, costi e limiti

Come si fa

1 Individuare il prefisso stabile

system prompt, istruzioni condivise, definizioni dei tool.

2 Inviare una richiesta di prewarm

con breakpoint cache_control.

3 Usare max_tokens = 0

max_tokens = 0

si esegue il prefill ma non si genera una risposta.

4 Alla richiesta reale: cache hit

se prefisso e configurazione coincidono.

TTL della cache

TTL predefinito: 5 minuti
La cache del prefisso rimane valida per 5 minuti dall'ultimo utilizzo.

Ogni utilizzo rinnova il TTL
Ogni richiesta che usa lo stesso prefisso e configurazione estende la validità della cache.

Può esistere anche un TTL di 1 ora
In alcuni casi può essere disponibile un TTL più lungo, fino a 1 ora.

Costi e benefici

Latenza iniziale	Si riduce in caso di cache hit
Token di output del prewarm	Zero
Scrittura in cache	È fatturata
Beneficio	Migliora il TTFT

Il prewarm conviene soprattutto quando il prompt iniziale è lungo e stabile.

Limiti e attenzione

Se la cache scade, va ricreata
Dopo la scadenza del TTL è necessario eseguire nuovamente il prewarm.

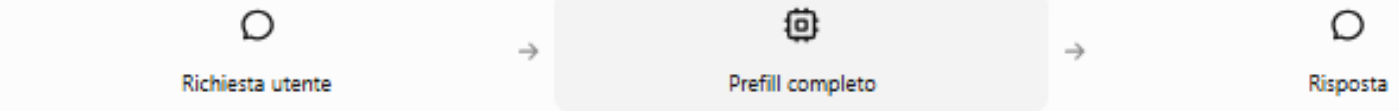
Se il prefisso cambia, può esserci cache miss
Anche piccole modifiche al prefisso o alla configurazione possono invalidare la cache.

Non è un comando slash di Claude Code
Il prewarm si usa tramite l'API, non è un comando speciale di Claude Code.

In Claude Code il caching è gestito internamente
Il prewarm esplicito riguarda l'uso diretto dell'API Anthropic; in Claude Code il caching è gestito automaticamente.

Come funziona

Senza prewarm



L'utente attende anche l'elaborazione iniziale del contesto.

Con prewarm

L'elaborazione del prefisso condiviso è già avvenuta. Resta da elaborare la parte nuova del prompt e generare la risposta.

Il beneficio principale è la riduzione del **TTFT (Time To First Token)**, cioè il tempo necessario a ricevere il primo token della risposta. Non significa necessariamente accelerare la generazione di tutti i token successivi. Claude Platfo... +1

COSA FA statusline_prompt.md

Una specifica per creare una status line minimale e informativa in Claude Code

In breve
Il file descrive il risultato da generare; non contiene ancora lo script Bash finale.

DAL FILE ALLA STATUS LINE

RISULTATO RICHIESTO

```
main+*  ·  Modello (high)  ·  67%
```

Tre segmenti, separati da un punto centrale “·” in grigio attenuato.

1. BRANCH GIT

Mostra il nome del branch corrente in ciano. Il segmento compare solo quando ci si trova in un repository Git.

`main`	branch pulito
`main*`	* arancione = modifiche non in stage
`main+`	+ giallo = modifiche in stage

Fuori da Git: il segmento non appare

2. MODELLO ED EFFORT

Mostra il nome visualizzato del modello in blu, seguito dal livello di effort tra parentesi in grigio attenuato.

```
Modello (high)
```

Origine dell’effort:

`~/.claude/settings.json`

Campo letto `effortLevel`

3. UTILIZZO DEL CONTESTO

Mostra soltanto il numero e il simbolo %. Il colore cambia in base alla percentuale di contesto utilizzata.

32%	normale: verde
58%	dal 50%: giallo

Nessuna etichetta “ctx”

ASPETTO VISIVO

- Stile minimale
- Separatori “·” in grigio dim
- Colori ANSI discreti
- Nessuno sfondo colorato

COSA NON MOSTRA

- ✕ nome utente o host
- ✕ directory corrente
- ✕ etichetta “ctx”
- ✕ informazioni non richieste

IN PRATICA

Il file è una specifica operativa: dice a Claude Code come configurare una status line tramite uno script Bash.

Obiettivo: informazioni essenziali a colpo d’occhio

Fonte: statusline_prompt.md allegato

Formato 16:9 · sintesi fedele

non è idempotente, se si vuole rieseguirlo cambiare cartella

DAL PROMPT AL PROGETTO PYTHON

rispetto all'originale, inserita una clausola di salvaguardia dopo le tre domande (❶)

Che cosa chiede a Claude Code il prompt di scaffolding

letter. «impalcatura, ponteggio»: nella IT «struttura di base»

1 • CHIEDE

Navigazione con le frecce
- left-right → sceglie il tab
- up-down → sceglie il valore

Nome del progetto

second_brain

Non mettere MAI spazi nei nomi di cartelle e variabili dati a Claude Code; usare *underscore* «_»

Versione iniziale: 0.0.1

0.0.1

Cartella di destinazione

./progetti/second_brain

Non mettere MAI spazi nei nomi di cartelle e variabili dati a Claude Code; usare *underscore* «_»

Adatta i nomi per Python

2 • CREA E CONFIGURA

STRUTTURA

src/ • tests/ • docs/ • scripts/

Package installabile e avvio da terminale

AMBIENTE

Python ≥ 3.13 • uv • uv_build

pyproject.toml: dipendenze e versione

LOGGING

Loguru: console + file

Configurazione con variabili d'ambiente

TEST

pytest: verifica il saluto

Log dei test in una cartella temporanea

QUALITÀ

Ruff: controllo e formattazione

Copertura: soglia configurata 80%

DOCUMENTAZIONE

MkDocs + Material • README

Guida d'uso e riferimento API

3 • INSTALLA E VERIFICA

Windows PowerShell

```
PS C:\progetti\second_brain> uv sync
```

Installa progetto e dipendenze

Windows PowerShell

```
PS C:\progetti\second_brain> uv run pytest
```

Esegue il test iniziale

DA AVVIARE SEPARATAMENTE

Ruff • copertura • anteprima e build della documentazione

APP INIZIALE

Configura il logging e scrive un saluto.

Windows PowerShell

```
PS C:\progetti\second_brain> uv run second_brain
2025-03-08 10:15:27 | INFO | second_brain | Hello from second_brain!
```

RISULTATO

Una base organizzata per sviluppare le funzionalità dell'applicazione.

uv.lock da versionare • .env e .env.test esclusi da Git

I contenuti del prompt di scaffolding - 1

Due versioni del prompt di scaffoldding (nella cartella Claude Code › doc-in-md)

python_project_scaffold.md ← *la versione originale, da dare in input a Claude Code (**in plan mode**)*

python_project_scaffold_render.md ← *la versione modificata, **ben renderizzabile in VS Code***

La parte centrale del prompt (dopo le domande):

```
Requirements:

- **Package manager:** uv
- **Config:** pyproject.toml only (no setup.py, no setup.
- **Layout:** src/ layout, installable package
- **Type hints:** none
- **Versioning:** single source of truth in pyproject.tom
```

personalizzabili

il package manager

*Il file pyproject.toml **semplifica la configurazione dei progetti Python**, unificando la configurazione dei pacchetti, la gestione delle dipendenze e l'ottimizzazione dei processi di build.*

il layout (sarà più chiaro più avanti)

Che cosa sono i type hints?
*Sono annotazioni che specificano il **tipo di dato atteso** per i parametri di una funzione, per il risultato restituito o per una variabile. Aiutano a comprendere il codice e possono essere utilizzate dagli editor e dagli strumenti di analisi statica per segnalare possibili errori.*
Generano un pò di confusione visuale??
Valori alternativi: optional, public API only, all function signatures, full, ecc)

Seguono i contenuti di certi file.

I contenuti del prompt di scaffolding - 2

```
### src/<packagename>/app.py
- Define a `configure_logging()` function that:
  1. Imports `os`
  2. Reads `LOG_LEVEL` from `os.environ.get("LOG_LEVEL",
  3. Reads `LOG_FILE` from `os.environ.get("LOG_FILE", "a
  4. Removes the default loguru handler
  5. Adds a stderr handler at the `LOG_LEVEL` level (defa
  6. Adds a file handler at DEBUG level using the `LOG_FI
```

```
```python
def configure_logging():
    """Configure loguru for console and file logging.

    Removes the default handler and sets up:
    - stderr handler at LOG_LEVEL (default: INFO, confi
    - File handler at DEBUG level writing to LOG_FILE (
    """
    import os
    log_level = os.environ.get("LOG_LEVEL", "INFO")
    log_file = os.environ.get("LOG_FILE", "app.log")
    logger.remove()
    logger.add(sys.stderr, level=log_level)
    logger.add(log_file, level="DEBUG", rotation="50 KB
```
```

Note sul prompt:

- Doppia descrizione delle istruzioni: **umane** e **codificate**
  → migliore comprensione sia per Claude Code che per il manutentore umano di questo prompt (documentazione
- Il prompt evolverà (cambiano framework e specifiche, ecc)
- Diversi ambienti (sviluppo, test,ecc)
- Readme e standard .gitignore
- After scaffolding →

After scaffolding: 1. Run `uv sync` to set up the environment 2. Run `uv run pytest` to verify the starter test passes 3. Remind the user to: - Commit `uv.lock` to version control for reproducible installs - Try `uv run python scripts/serve_docs.py` to preview the documentation locally ```

Il piano

Plan: Scaffold second-brain Python Project

Context

Scaffold a best-practice Python application in the current directory /Users/rp/projects/cclc/sb.
- **Project name** (pyproject.toml, docs): second-brain
- **Package name** (Python imports/dirs): second_brain
- **Version:** 0.0.1
- **Location:** current directory (scaffolded directly into /Users/rp/projects/cclc/sb)

File Structure to Create

```
/Users/rp/projects/cclc/sb/
  src/
    second_brain/
      __init__.py          (empty)
      __main__.py          (calls main() from app.py)
      app.py               (configure_logging + main)
  tests/
    __init__.py            (empty)
    conftest.py            (autouse log-file fixture)
    test_app.py            (test_main_logs_greeting)
  scripts/
    serve_docs.py          (mkdocs serve + tee to
```

Key Decisions

Names

- Package dir: src/second_brain/
- CLI entrypoint: second_brain = "second_brain.app:main"
- Docs reference :::second_brain

pyproject.toml highlights

- Build backend: uv_build>=0.10.9,<0.11.0
- requires-python = ">=3.13"
- Dependencies: loguru
- Dev group: ruff, pytest, pytest-cov, pytest-env, mkdocs, mkdocs-material, mkdocstrings[python]
- Ruff: line-length=88, target-version="py313", select=["E","F","B","I","UP"], ignore=["E501"]
- pytest: --import-mode=importlib, --strict-markers, --strict-config, -ra, testpaths=["tests"]
- pytest-env: loads .env.test
- Coverage: branch=true, source=["src"], fail_under=80

app.py

- configure_logging(): reads LOG_LEVEL/LOG_FILE from env, removes default handler, adds stderr + rotating file handlers
- main(): decorated with @logger.catch, calls

Post-Scaffold Steps

1. Run uv sync to install dependencies and generate uv.lock
2. Run uv run pytest to verify the starter test passes
3. Remind user to:
   - Commit uv.lock for reproducible installs
   - Try uv run python scripts/serve_docs.py to preview docs

Verification

- uv run pytest → 1 test passes (test_main_logs_greeting)
- uv run second_brain → prints "Hello from second_brain!" to stderr + writes app.log
- uv run pytest --cov → coverage ≥ 80%

Claude has written up a plan and is ready to execute. Would you like to proceed?

❯ 1. Yes, auto-accept edits
  2. Yes, manually approve edits
  3. Type here to tell Claude what to change

ctrl-g to ediVS Cod · ~/.claude/plans/hazy-weaving-u
in mbrella.md

Come rieseguire il prompt di scaffolding - 1

Procedura consigliata per il corso

1. **Creare una nuova cartella vuota**, esterna al progetto della sessione precedente.
2. **Aprire un terminale nella nuova cartella e avviare una nuova sessione di Claude Code** con `claude`. Avviare Claude dalla directory del progetto è il flusso documentato da Anthropic.
3. **Fornire il prompt completo.**
4. Alla terza domanda del prompt, **scegliere "current directory"**: il progetto verrà creato direttamente nella cartella appena predisposta.

Il file del prompt può rimanere nella cartella dei materiali del corso, così la nuova destinazione resta vuota.

Se viene scelta invece l'opzione "**new subdirectory**", la root effettiva sarà la sottocartella creata da Claude: **eseguire anche uv sync, i test e gli altri comandi da quella posizione**. Questa distinzione è già prevista nelle istruzioni del file.

Che cosa significa, esattamente, "ripartire da zero"

Una nuova root permette di **ricreare da zero i file del progetto**. Alcuni elementi della macchina rimangono però disponibili:

- **La cache di uv**, che può riutilizzare pacchetti già scaricati e rendere la seconda esecuzione più rapida.
- **Le installazioni Python già presenti**. uv può riutilizzare un interprete compatibile e, quando necessario, scaricarne uno.
- **Le impostazioni utente di Claude Code**, che si applicano anche ai nuovi progetti.

Per ripetere l'esercitazione di scaffolding, questo riuso è normalmente appropriato. La cartella autonoma presuppone inoltre che il nuovo progetto non venga inserito in un *workspace uv* già configurato per gestire più package insieme.

Come rieseguire il prompt di scaffolding - 2

Nuova esecuzione e riproducibilità non sono la stessa cosa

Cambiare root non rende il prompt idempotente: consente di eseguirlo su una nuova destinazione. Il prompt attuale non specifica una politica completa per gestire file già esistenti, quindi non andrebbe trattato come un'operazione sicura da ripetere indiscriminatamente nella stessa cartella.

Inoltre, per ottenere **le stesse identiche versioni delle dipendenze in lezioni successive**, una nuova root non basta: nel prompt ci sono requisiti aperti, come `ruff>=0.11` e `pytest>=8.0`. Occorre conservare una versione di riferimento del progetto con `pyproject.toml`, `uv.lock` e una versione Python fissata. Il lockfile registra le versioni risolte delle dipendenze.

Per l'uso durante il corso si consiglia di aggiungere al prompt un controllo iniziale della destinazione: mostrare il percorso assoluto, verificare che la cartella sia vuota — inclusi i file nascosti — e fermarsi se contiene già qualcosa. Questa regola renderebbe esplicito il comportamento atteso prima di creare file o avviare `uv sync`.

uv

Il tool moderno per lo sviluppo Python

Un unico strumento, tutto quello che ti serve

Cos'è uv?

uv è un **programma da riga di comando (CLI)** open source di Astral, scritto in **Rust**, che rende più semplice, veloce e affidabile lo sviluppo Python.
Gestisce versioni di Python, ambienti virtuali, dipendenze, esecuzione di strumenti e progetti, con un'unica interfaccia.

- **Veloce** — Scritto in Rust, estremamente rapido
- **Affidabile** — Risoluzione delle dipendenze consistente
- **Completo** — Ambiente, dipendenze, Python e strumenti
- **Moderno** — Progettato per i workflow attuali
- **Open source** — Sviluppato da Astral

Cosa sostituisce?

uv riunisce le funzionalità di diversi strumenti tradizionali in un'unica soluzione.

Strumento tradizionale		Funzionalità equivalente in uv
pip	→	Installazione dei package Python
venv	→	Creazione di ambienti virtuali
pip-tools	→	Risoluzione e blocco delle dipendenze
pyenv	→	Installazione e gestione delle versioni Python
pipx	→	Esecuzione di strumenti Python isolati
poetry	→	Gestione di progetti e dipendenze

Cosa puoi fare con uv?

I comandi principali per le operazioni più comuni.

Gestire Python
Installare e gestire diverse versioni di Python.

```
uv python install 3.13
uv python list
```

Creare un progetto
Inizializzare un nuovo progetto Python.

```
uv init myproject
cd myproject
```

Gestire dipendenze
Aggiungere, rimuovere e sincronizzare i package.

```
uv add pandas
uv remove pandas
uv sync
```

Eseguire comandi
Lanciare script o strumenti nell'ambiente del progetto.

```
uv run python main.py
uv run pytest
uv run ruff check .
```

Strumenti isolati
Eseguire strumenti senza installarli globalmente.

```
uv tool run ruff
uv tool install mkdocs
```

Esempio pratico: ciclo di vita di un progetto

Dalla creazione all'esecuzione, in pochi comandi.

1	**Crea il progetto**	`uv init analisi-clienti`
2	**Entra nella cartella**	`cd analisi-clienti`
3	**Aggiungi dipendenze**	`uv add loguru pandas`
4	**Aggiungi dipendenze di sviluppo**	`uv add --group dev pytest ruff`
5	**Prepara l'ambiente**	`uv sync`
6	**Esegui l'applicazione**	`uv run python main.py`
7	**Esegui i test**	`uv run pytest`
8	**Esegui strumenti**	`uv run ruff check .` `uv run mkdocs serve`

Come si installa?

Puoi installare uv in diversi modi.

Installer ufficiale (consigliato)

```
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Installa l'eseguibile e lo aggiunge al PATH %USERPROFILE%\.local\bin).

Tramite pip

```
pip install uv
```

Installa l'eseguibile nella cartella Scripts di Python.

Tramite Scoop (Windows) `scoop install uv`
Usa il gestore di package Scoop.

Dove si trova su Windows?

uv è un eseguibile da riga di comando, non un'app grafica. Può non comparire nell'elenco App installate.

Verifica se è installato:

```
uv --version
# es. uv 0.x.x
```

Trova il percorso dell'eseguibile:

```
where  uv
# es. C:\Users\<utente>\.local\bin\uv.exe
```

Se non viene trovato, potrebbe non essere installato o non essere nel PATH.

Struttura tipica di un progetto con uv

uv crea progetti moderni con layout src/.

```
analisi-clienti/
├── pyproject.toml
├── uv.lock
├── src/
│   └── analisi_clienti/
│       ├── __init__.py
│       ├── __main__.py
│       └── app.py
├── tests/
├── docs/
├── scripts/
├── README.md
├── .gitignore
├── .env.example
├── .env  (non committato)
└── .env.test (non committato)
```

Come si disinstalla?

Dipende dal metodo di installazione utilizzato.

Installer ufficiale `uv self uninstall`
Rimuove l'eseguibile installato dallo script ufficiale.

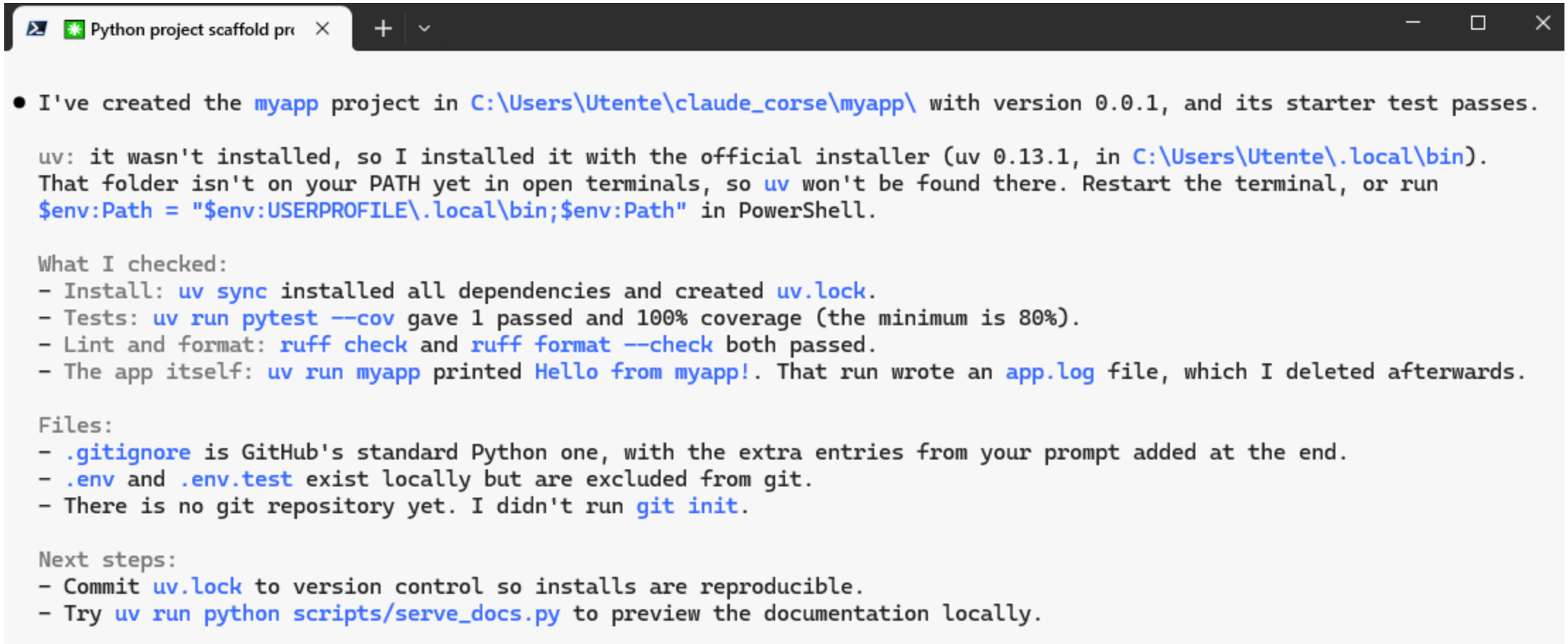
Tramite pip `pip uninstall uv`
Rimuove uv dall'ambiente Python.

Tramite Scoop `scoop uninstall uv`
Rimuove uv installato con Scoop.

Vantaggi principali

- Molto più veloce di pip
- Un unico strumento
- Dipendenze affidabili
- Gestione integrata di ambienti e versioni Python
- Ideale per progetti moderni e team
- Funziona su Windows, macOS e Linux

L’esecuzione del prompt di scaffolding termina così:



A cosa serve pyproject.toml?

Il file centrale di configurazione di un progetto Python moderno.

pyproject.toml è uno standard moderno che permette di descrivere, configurare, costruire e gestire un progetto Python in **un unico file**.

1 Identità del progetto

- Contiene i metadati del progetto:
  - nome del progetto
  - versione
  - descrizione
  - autori / mantenitori
  - versione minima di Python (`requires-python`)

2 Dipendenze

- Permette di dichiarare:
  - **dipendenze runtime** (librerie necessarie per far funzionare il progetto)
  - **dipendenze di sviluppo** (es. per test, linting, ecc.)
- Tutte le dipendenze sono in un unico posto, in modo chiaro e versionato.

3 Build e packaging

- Definisce come il pacchetto viene costruito e quale **build backend** utilizzare (es. `setuptools`, `hatchling`, `poetry`, ecc.).
- Sostituisce i vecchi meccanismi sparsi (`setup.py`, `setup.cfg`, ecc.).
- Rende il processo di build standardizzato e più semplice.

4 Configurazione strumenti

- Molti strumenti possono leggere la loro configurazione direttamente da `pyproject.toml`, ad esempio:

Ruff	Pytest	Coverage
Black	Mypy	e molti altri

- Una configurazione centralizzata per tutto il progetto.

5 Script ed entry point

- Permette di definire comandi da riga di comando (entry point).
- Utile per creare una CLI del tuo progetto e rendere disponibile un comando, ad esempio `my_app.`
- Il comando verrà installato automaticamente quando il pacchetto è installato.

6 Perché è utile

- Un unico file per le informazioni principali del progetto.
- **Più ordine:** meno file di configurazione sparsi.
- Manutenzione più semplice.
- **Maggiore riproducibilità** tra ambienti e team di sviluppo.
- Standard moderno e ampiamente supportato dall'ecosistema Python.

In pratica...

Prima:
configurazioni sparse in molti file

`setup.py` `requirements.txt` `.flake8` `pytest.ini` `mypy.ini` `...`

Con pyproject.toml:
configurazione centralizzata

Risultato:
progetto più chiaro e gestibile

Esempio di pyproject.toml (semplificato)

```toml
[project]
name = "my-app"
version = "0.0.1"
requires-python = ">=3.13"
dependencies = ["loguru"]

[tool.ruff]
line-length = 88

[project.scripts]
my_app = "my_app.app:main"
```

Dopo il prompt: quali verifiche fare

Checklist finale dopo l'esecuzione del prompt di scaffolding Python in Claude Code

da altro terminale: il server della doc (❻) rimane in attesa!

🎯 **Obiettivo:** verificare che uv, progetto, test, qualità e documentazione funzionino davvero sul proprio PC.

1 Apri un nuovo terminale

- Meglio PowerShell o CMD aperti dopo l'installazione di uv
- Se uv non è riconosciuto, chiudi e riapri il terminale

```
uv --version
```

✅ **OK:** compare la versione

⚠️ Se non funziona: PATH non aggiornato

2 Entra nel progetto

- Vai nella cartella creata da Claude
- Verifica che ci siano pyproject.toml, uv.lock, src/, tests/

```
cd C:\Users\Utente\\
claude_corse\myapp
```

Adatti il percorso al proprio progetto.

3 Avvia l'app iniziale

- Controlla che il package sia installabile e avviabile
- L'output atteso contiene il saluto iniziale

```
uv run myapp
```

✅ **Atteso:**
`Hello from myapp!`

4 Riesegui i test

- Conferma che il test automatico passi anche sul suo terminale
- La copertura dovrebbe risultare almeno 80%

```
uv run pytest --cov
```

✅ **Atteso:**
`1 passed · coverage >= 80%`

link a pytest

5 Verifica qualità del codice

- Controllo statico con Ruff
- Controllo della formattazione

```
uv run ruff check .
```

```
uv run ruff format
--check .
```

✅ **Atteso:**
nessun errore

link a ruff

6 Prova la documentazione

- Avvia l'anteprima locale MkDocs
- Apri nel browser l'indirizzo locale mostrato dal terminale

```
uv run python
scripts/serve_docs.py
```

Per fermare il server: Ctrl + C

link a MkDocs

⚠️ Se vede un messaggio rosso su Claude

Esempio: `Auto-update failed: claude.exe in use`

Riguarda Claude Code, non il progetto Python.
Chiuda altre sessioni Claude Code e, se serve, esegua:

```
claude doctor
```

⚙️ Controlli extra utili

A **uv.lock presente**

Da versionare in Git.

B **.env e .env.test presenti**

Locali, esclusi da Git.

C **app.log**

Può essere creato durante l'esecuzione.

D **git init**

Non è automatico, se Claude non lo ha eseguito.

🏆 **Esito finale** | Se tutti i controlli passano, il progetto base è stato creato correttamente ed è pronto per essere sviluppato o messo sotto versionamento.

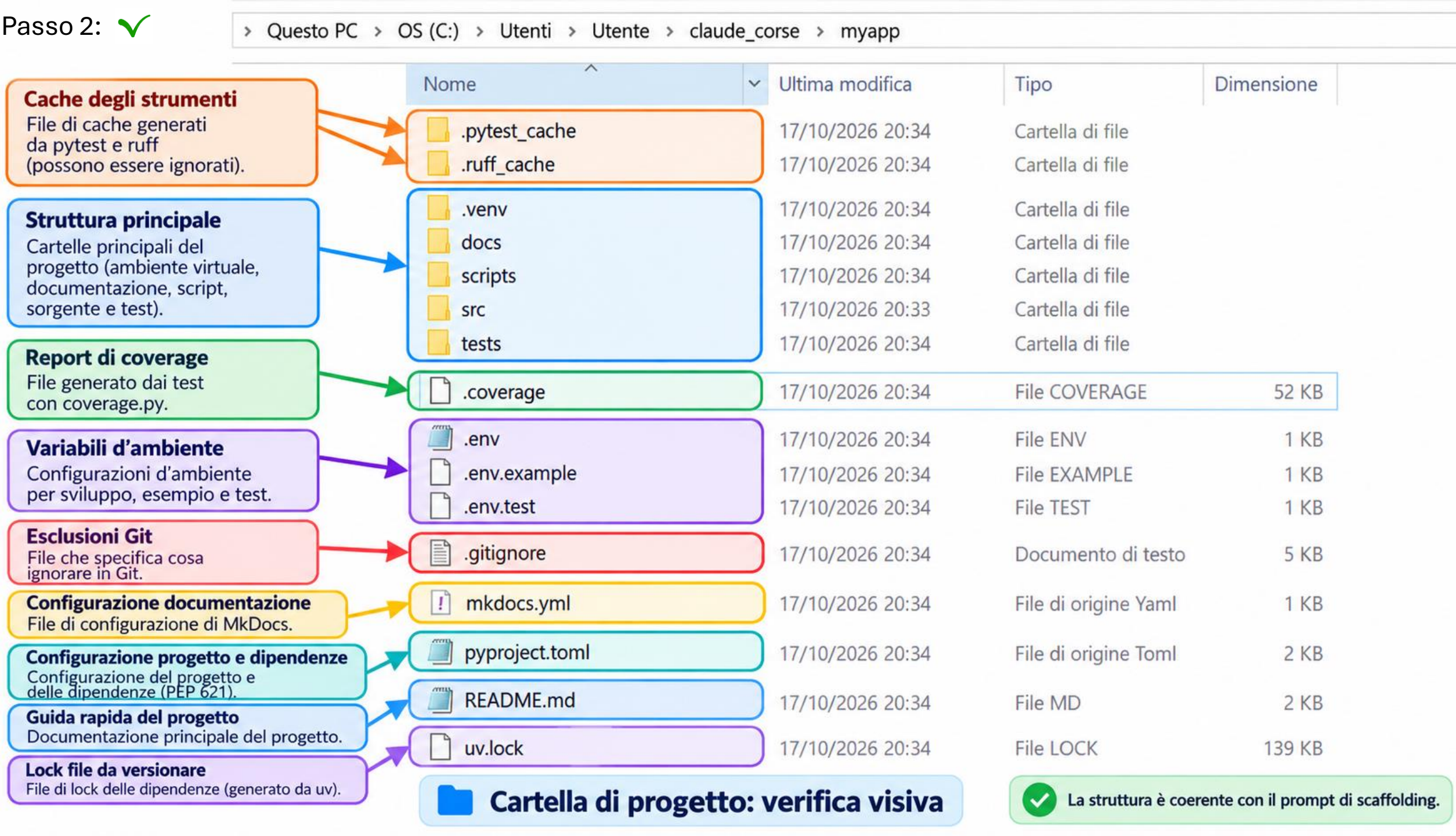
pytest: helps you write better programs
The pytest framework makes it easy to write small, readable tests, and can scale to support complex functional testing for applications and libraries.

Ruff - Astral Docs
Ruff aims to be orders of magnitude faster than alternative tools while integrating more functionality behind a single, common interface.

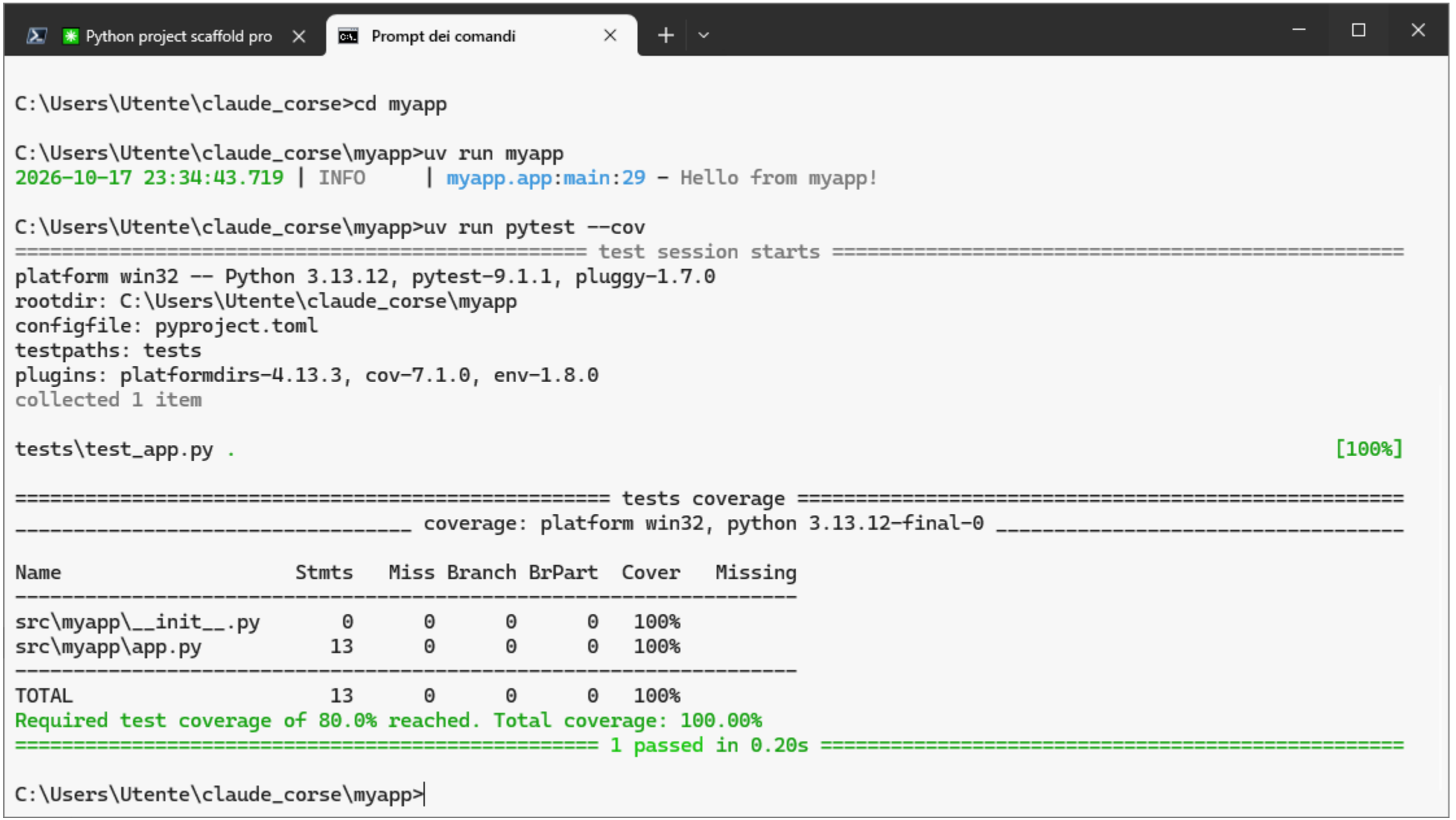
MkDocs is a **fast**, **simple** and **downright gorgeous** static site generator that's geared towards building project documentation. Documentation source files are written in Markdown, and configured with a single YAML configuration file. Start by reading the introductory tutorial, then check the User Guide for more information.

MkDocs
Project documentation with Markdown.

Passo 2: ✓


Questo PC › OS (C:) › Utenti › Utente › claude_corse › myapp
Nome
Ultima modifica
Tipo
Dimensione
.pytest_cache 17/10/2026 20:34 Cartella di file
.ruff_cache 17/10/2026 20:34 Cartella di file
.venv 17/10/2026 20:34 Cartella di file
docs 17/10/2026 20:34 Cartella di file
scripts 17/10/2026 20:34 Cartella di file
src 17/10/2026 20:33 Cartella di file
tests 17/10/2026 20:34 Cartella di file
.coverage 17/10/2026 20:34 File COVERAGE 52 KB
.env 17/10/2026 20:34 File ENV 1 KB
.env.example 17/10/2026 20:34 File EXAMPLE 1 KB
.env.test 17/10/2026 20:34 File TEST 1 KB
.gitignore 17/10/2026 20:34 Documento di testo 5 KB
mkdocs.yml 17/10/2026 20:34 File di origine Yaml 1 KB
pyproject.toml 17/10/2026 20:34 File di origine Toml 2 KB
README.md 17/10/2026 20:34 File MD 2 KB
uv.lock 17/10/2026 20:34 File LOCK 139 KB
Cache degli strumenti
File di cache generati da pytest e ruff (possono essere ignorati).
Struttura principale
Cartelle principali del progetto (ambiente virtuale, documentazione, script, sorgente e test).
Report di coverage
File generato dai test con coverage.py.
Variabili d'ambiente
Configurazioni d'ambiente per sviluppo, esempio e test.
Esclusioni Git
File che specifica cosa ignorare in Git.
Configurazione documentazione
File di configurazione di MkDocs.
Configurazione progetto e dipendenze
Configurazione del progetto e delle dipendenze (PEP 621).
Guida rapida del progetto
Documentazione principale del progetto.
Lock file da versionare
File di lock delle dipendenze (generato da uv).
Cartella di progetto: verifica visiva
La struttura è coerente con il prompt di scaffolding.

Passo 3-4: ✓


Python project scaffold pro
Prompt dei comandi
C:\Users\Utente\claude_corse>cd myapp
C:\Users\Utente\claude_corse\myapp>uv run myapp
2026-10-17 23:34:43.719 | INFO | myapp.app:main:29 - Hello from myapp!
C:\Users\Utente\claude_corse\myapp>uv run pytest --cov
test session starts
platform win32 -- Python 3.13.12, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\Utente\claude_corse\myapp
configfile: pyproject.toml
testpaths: tests
plugins: platformdirs-4.13.3, cov-7.1.0, env-1.8.0
collected 1 item
tests\test_app.py . [100%]
tests coverage
coverage: platform win32, python 3.13.12-final-0
Name Stmts Miss Branch BrPart Cover Missing
src\myapp__init__.py 0 0 0 0 100%
src\myapp\app.py 13 0 0 0 100%
TOTAL 13 0 0 0 100%
Required test coverage of 80.0% reached. Total coverage: 100.00%
1 passed in 0.20s
C:\Users\Utente\claude_corse\myapp>

Passo 5: ✅

Controllo	Risultato	Significato
Analisi statica — `ruff check .`	`All checks passed!`	Nessuna violazione rilevata dalle regole configurate.
Formattazione — `ruff format --check .`	`11 files already formatted`	Tutti gli **11 file controllati** sono già formattati correttamente.

Non sono richieste correzioni da Ruff. In particolare, "11 files already formatted" è un esito positivo, non un avviso di errore.

Passo 6a: ✓

Il server di anteprima di MkDocs, si chiude con ctrl-c

```
C:\Users\Utente\claude_corse\myapp>uv run python scripts/serve_docs.py

 \u2502  \u26a0  Warning from the Material for MkDocs team
 \u2502
 \u2502  MkDocs 2.0, the underlying framework of Material for MkDocs,
 \u2502  will introduce backward-incompatible changes, including:
 \u2502
 \u2502  × All plugins will stop working – the plugin system has been removed
 \u2502  × All theme overrides will break – the theming system has been rewritten
 \u2502  × No migration path exists – existing projects cannot be upgraded
 \u2502  × Closed contribution model – community members can't report bugs
 \u2502  × Currently unlicensed – unsuitable for production use
 \u2502
 \u2502  Our full analysis:
 \u2502
 \u2502  https://squidfunk.github.io/mkdocs-material/blog/2026/02/18/mkdocs-2.0/

INFO    -  Building documentation...
INFO    -  Cleaning site directory
INFO    -  Documentation built in 0.95 seconds
INFO    -  [23:52:36] Watching paths for changes: 'docs', 'mkdocs.yml'
INFO    -  [23:52:36] Serving on http://127.0.0.1:8000/
```

deve rimanere attivo?

Elaborato per 1m 25s

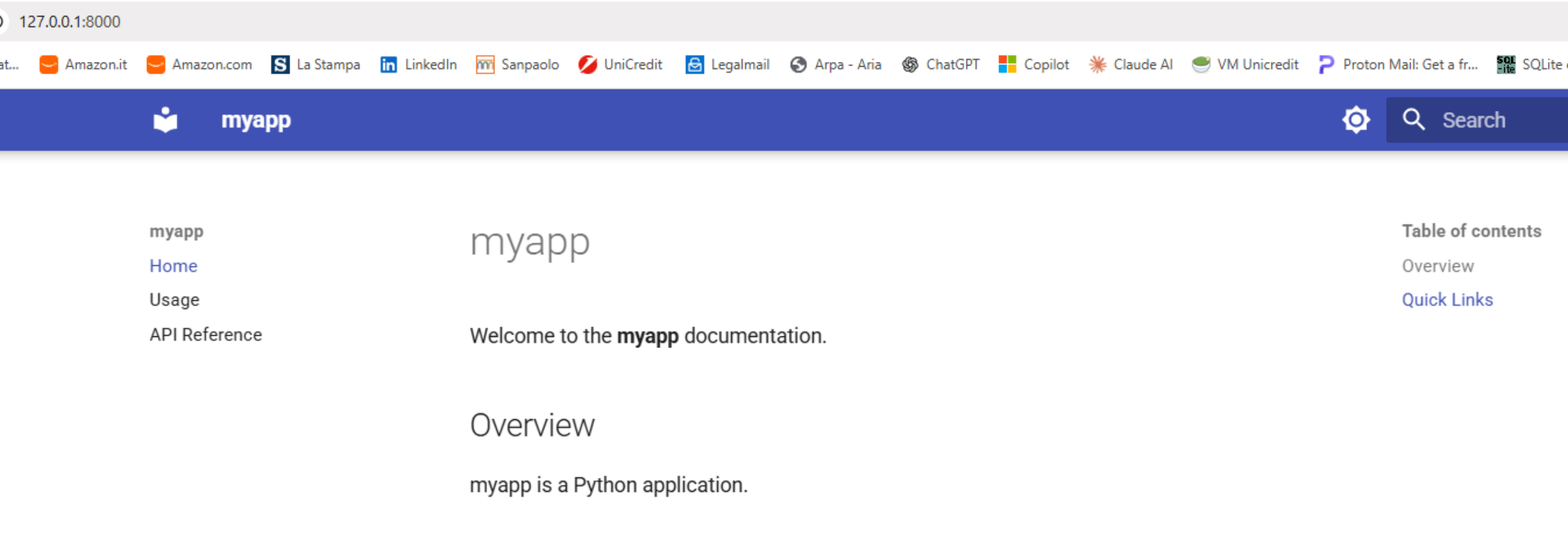
No, non deve rimanere sempre attivo. Serve soltanto mentre consulta nel browser l'**anteprima della documentazione** o vuole vederla aggiornarsi durante le modifiche. Non è un servizio necessario al funzionamento dell'applicazione Python. mkdocs.org

Terminata la consultazione, può fermarlo con Ctrl+C nel terminale. Può continuare a lavorare con Claude Code, modificare il progetto ed eseguire l'applicazione e i test anche con questo server spento.

Quando vorrà consultare nuovamente l'anteprima, dalla cartella `myapp` esegua di nuovo:

```
uv run python scripts/serve_docs.py
```

Passi 6b-6c: ✓



6c

Verifica	Esito
uv e ambiente del progetto	✅ Funzionano: i comandi vengono eseguiti correttamente.
Avvio dell'applicazione	✅ `uv run myapp` produce il saluto previsto.
Test automatico	✅ **1 test superato**, con copertura misurata del **100%** sul codice iniziale.
Analisi statica con Ruff	✅ `All checks passed!`
Formattazione	✅ Tutti gli **11 file controllati** sono già formattati correttamente.
Documentazione	✅ Generata; server avviato e pagina **Home** correttamente visibile nel browser.

L'unico controllo visivo non ancora confermato dalle immagini è l'apertura delle pagine "Usage" e "API Reference": se anche quelle si aprono correttamente, la verifica della documentazione è completa.

L'avviso sull'aggiornamento di **Claude Code** resta una questione separata: non segnala un fallimento del progetto Python.

Ora ha una base di sviluppo funzionante, non ancora un'applicazione completa: per il momento la sua funzionalità è scrivere il saluto iniziale. Può fermare il server della documentazione con **Ctrl+C** e iniziare a sviluppare le funzionalità desiderate, senza rilanciare il prompt di scaffolding.

Varie

Claude Code Skills, Clearly Explained

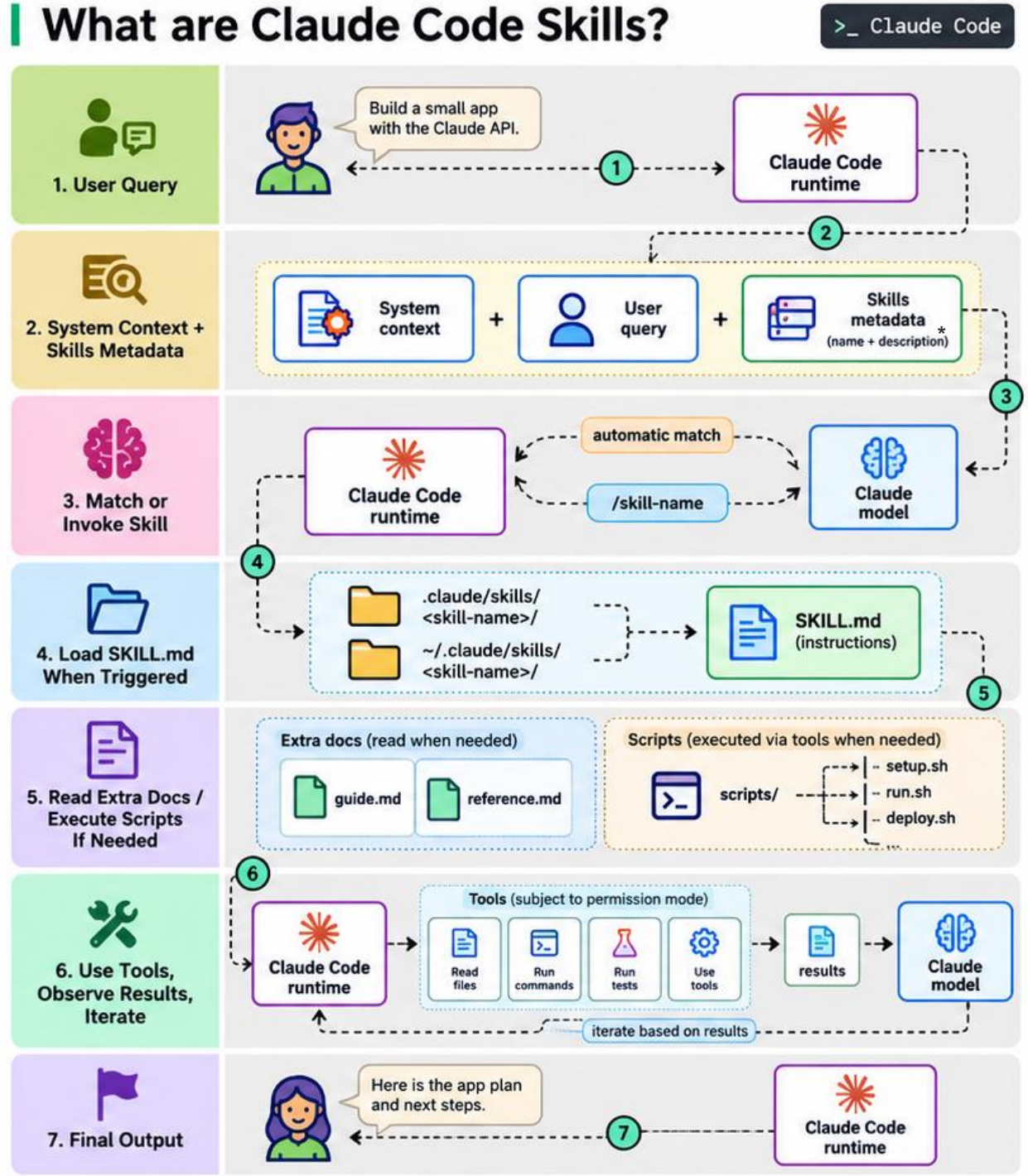
Why do we need Skills in Claude Code? Instead of repeating the same specialized guidance in every prompt, Claude Code can discover reusable Skills. Each Skill packages lightweight metadata, instructions, and optional resources, and is used only when relevant.

Here is what the Claude Code Skills workflow looks like:

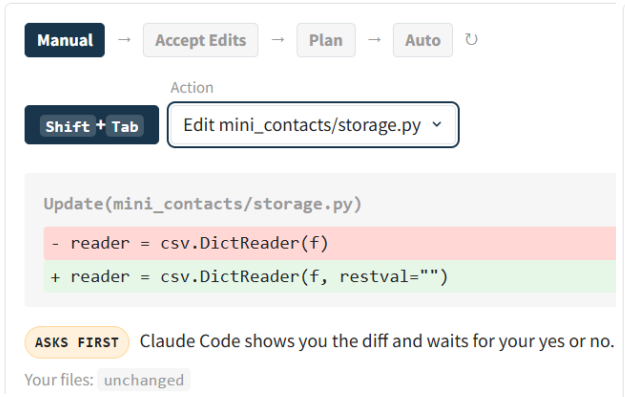
1. **User Query:** A user asks for help, for example: "Build a small app with the Claude API."
2. **System Context + Skills Metadata:** Claude Code works with system context plus a lightweight catalog of available Skills, including each Skill's name and description*.
3. **Match or Invoke Skill:** Claude may match a relevant Skill from its description, or the user can explicitly invoke a user-invocable Skill with /skill-name, for example /claude-api or /my-skill.
4. **Load SKILL.md When Triggered:** When a Skill is triggered, Claude reads that Skill's SKILL.md from the filesystem. Custom Skills commonly live in .claude/skills/<skill-name>/SKILL.md or ~/.claude/skills/<skill-name>/SKILL.md.
5. **Read Extra Docs / Execute Scripts If Needed:** Additional markdown guides can be read into context when needed. Helper scripts can be executed through tools when needed.
6. **Use Tools, Observe Results, Iterate:** While following the Skill guidance, Claude Code may read files, run commands, execute tests, and use other tools—subject to the current permission mode. Tool results feed back into the next decision step.
7. **Final Output:** Claude returns the result, such as code changes, an explanation, or an implementation plan.

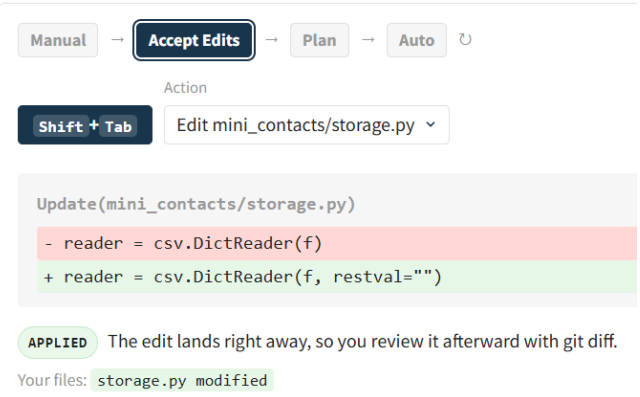
Skills use progressive disclosure: lightweight metadata supports discovery, SKILL.md is read when the Skill is triggered, and extra docs or scripts are used only when needed. This keeps context efficient and workflows reusable.

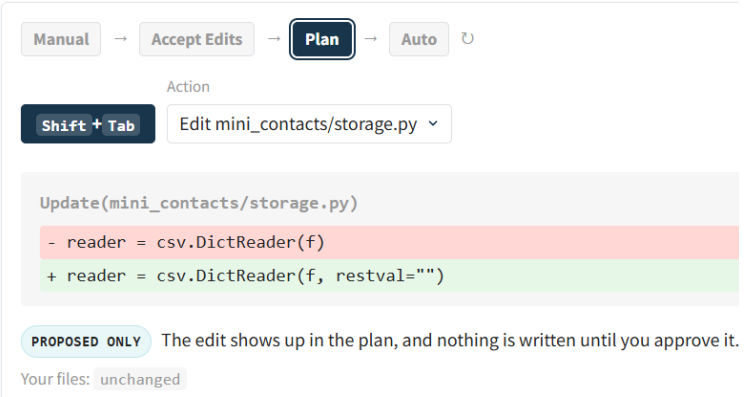
Custom Skills in Claude Code are filesystem-based. Automatic invocation and slash-command behavior depend on Skill configuration.

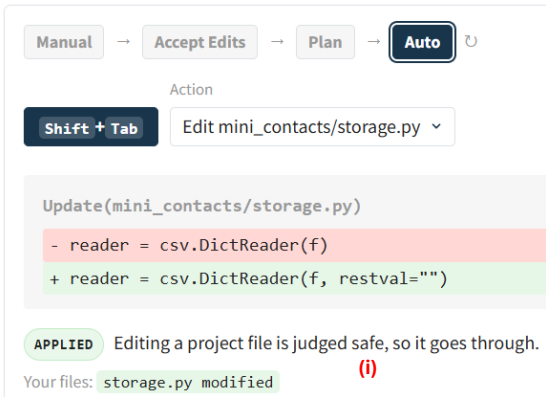


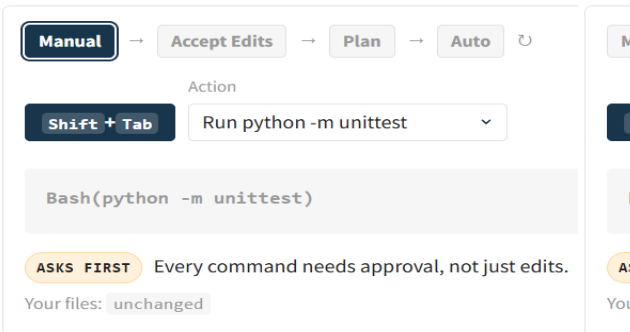
Griglia delle *permission*: incrocia due cose: le **4 modalità** che si scorrono con Shift+Tab (colonne) e **3 azioni tipiche** (righe), scelte apposta perché appartengono a **categorie diverse**: una modifica a un file del progetto, un comando shell innocuo, un comando distruttivo su un file fuori dal progetto. Ogni cella dice cosa succede *prima* che l'azione avvenga (chiede, esegue, propone, blocca) e lo stato dei file dopo.

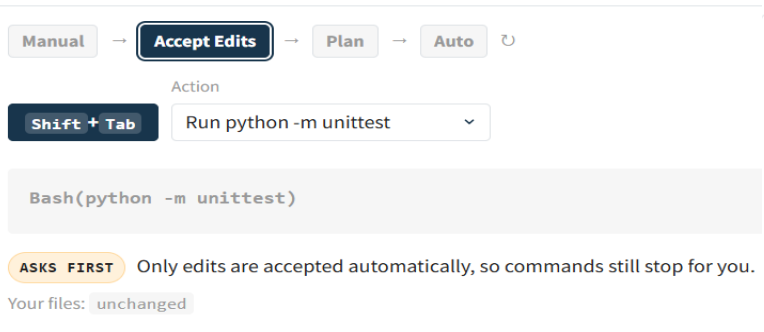


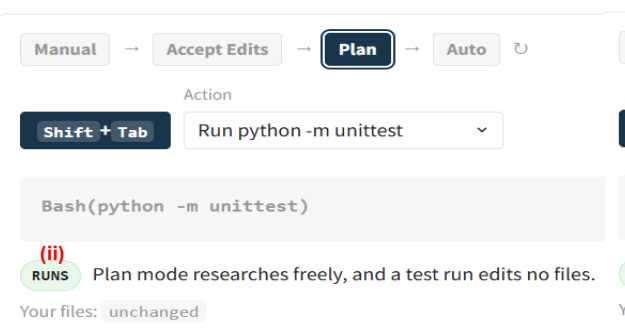


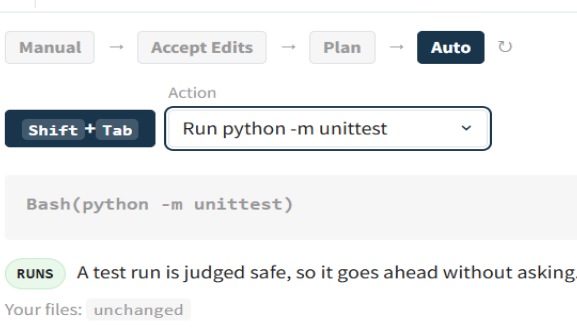


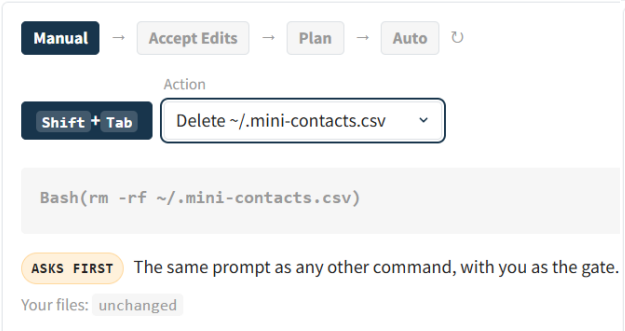


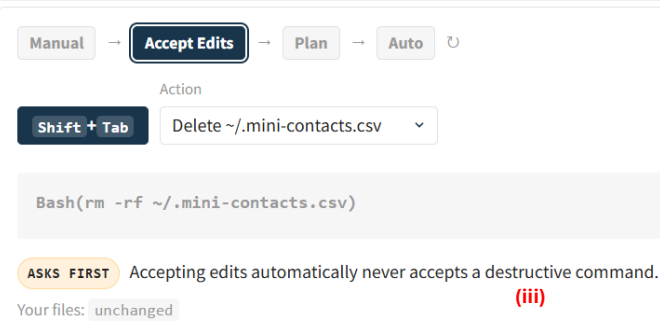


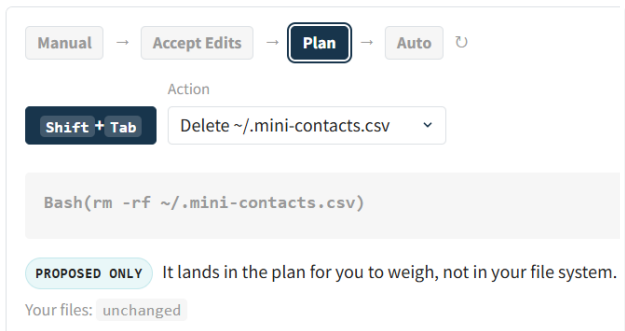


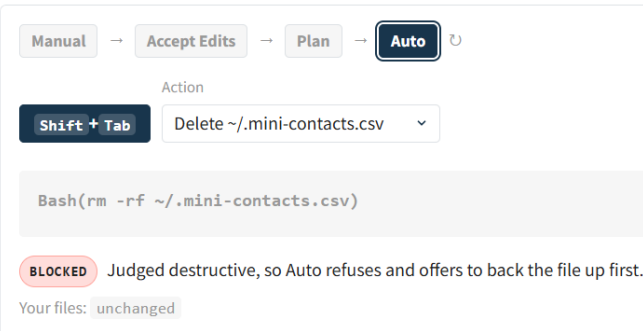












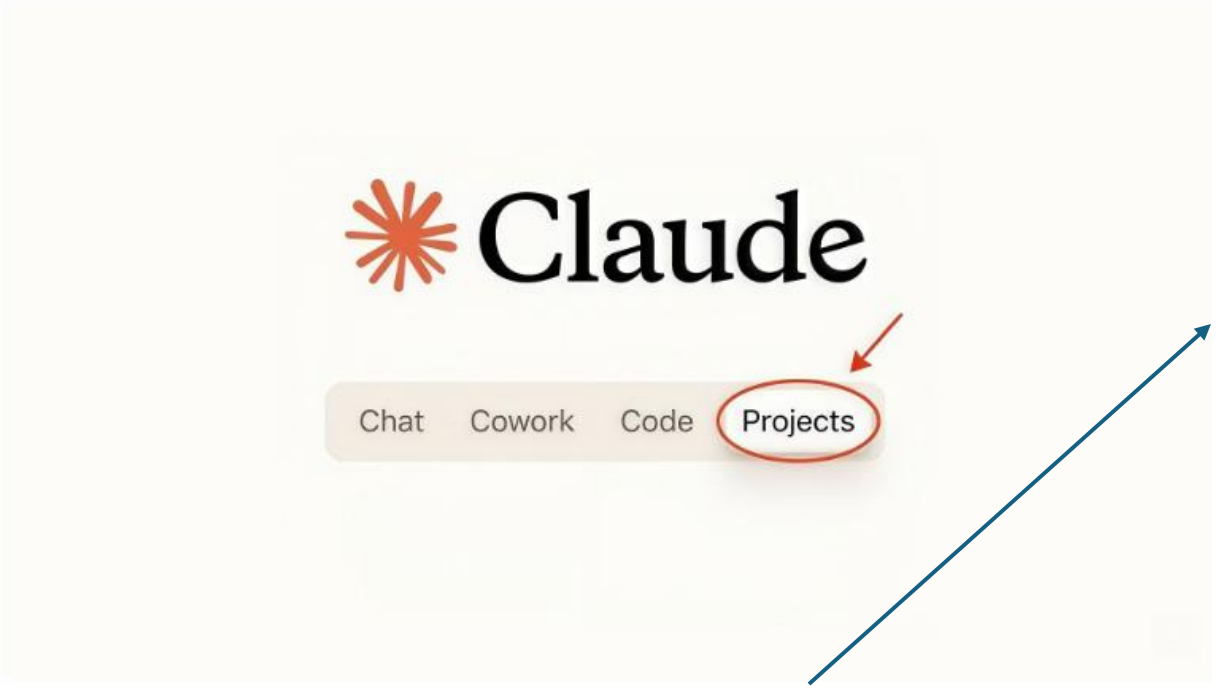
(i) "judged safe" è un modo di dire. Letture e modifiche ai file nella working directory sono approvate direttamente senza passare dal classifier, tranne i protected paths (.git, .claude, .vscode...).
(ii) "RUNS" presuppone che Auto sia disponibile. Con auto mode disponibile, il classifier valuta i comandi durante la pianificazione; altrimenti i comandi fuori dal set read-only integrato chiedono conferma, e python -m unittest non è in quel set.
(iii) la didascalia "never accepts a destructive command" è imprecisa. Accept Edits auto-approva anche rm, ma solo dentro la working directory: qui chiede perché il file sta nella home. Lo stesso comando su un file del progetto passerebbe senza prompt. Il criterio è "dove", non "distruttivo o no".

Tabella riassuntiva

	Finestra di contesto	Gestione automatica del contesto	Memoria
Cos'è	Limite fisico del modello	Funzione dell'app (compressione)	Funzione dell'app (archivio)
Ambito	Una conversazione	Una conversazione	Tutte le conversazioni
Persistenza	Nessuna	Nessuna (fuori dalla chat)	Sì
Contenuto	Tutto, letterale	Riassunti del passato della chat	Fatti durevoli selezionati
Scopo	Vedere per rispondere	Non sbattere contro il limite	Continuità tra sessioni

In una frase: la finestra di contesto è *quanto* Claude può vedere ora; la gestione automatica decide *cosa tenere in vista* quando lo spazio finisce; la memoria decide *cosa portarsi dietro* da una conversazione all'altra. Se ti serve per un corso, i tre livelli si mappano bene su concetti classici: memoria di lavoro, compressione/summarization, e storage persistente con retrieval.

Claude can now run a whole project for you



Anthropic has [let every Pro and Max user on the waitlist](#) into Claude Code Projects.

Generative AI
Daily Newsletter
12.10.2026

You give Claude one ongoing goal, and it splits the work into separate tasks that each run on their own in the background.

- The tasks keep going after you close your laptop, and you can check in from your phone.
- Each task can propose code changes on GitHub and fix them when tests fail.
- It works for non-code jobs too, like digging through a pile of support tickets for the most common problems.
- It remembers what you tell it across tasks so you don’t need to constantly remind it.

It runs on the web, desktop and mobile apps, and it burns through your usage limits faster than regular Claude Code.

Epic ha vietato il codice scritto a mano nel suo gioco Marvel e Star Wars

Epic Games ha detto al team che sta realizzando il suo sparatutto Marvel e Star Wars di adottare un approccio AI-first con Claude di Anthropic, riporta Bloomberg.

"Niente più codice scritto a mano."

È quello che ha scritto uno dei direttori del gioco in un messaggio interno. Lo stesso direttore ha detto che anche tutta la grafica del gioco sarebbe stata realizzata con l'AI.

- Il gioco era previsto per marzo 2027 e ora è rinviato, senza una nuova data.
- Bloomberg dice che Epic ha sottovalutato quante persone e quanto tempo le servissero.
- Gli sviluppatori sono rimasti sbalorditi quando hanno sentito la notizia in una videochiamata all'inizio di ottobre.
- Disney ha investito 1,5 miliardi di dollari in Epic nel 2024, e questo dovrebbe essere il primo gioco nato da quell'accordo.
- Peccato per Disney, perché la notizia ha fatto arrabbiare molti videogiocatori su X. La gente non sarà più così entusiasta del gioco.

Il CEO Tim Sweeney la definisce una svolta enorme pensata per permettere a tutti in Epic di costruire di più e meglio.

Quando un critico gli ha detto che non sapeva "leggere la stanza" (cioè capire l'umore del pubblico), Sweeney ha risposto che gran parte della "stanza" vorrebbe che fosse di nuovo il 2016, e che lui sta cercando di stare al passo con i progressi tecnologici. Tu che ne pensi?

CLAUDE CODE · DENTRO IL TERMINALE

Gli «spinner verbs» di Claude Code

I verbi sempre diversi che girano nel terminale mentre il modello lavora: ecco cosa significano — e perché, di proposito, non significano niente.

puro folclore, estratto a caso · l'informazione vera: tempo, token, strumenti

Il pensiero (sempre più pomposo)

Pondering…	*ponderando*
Ruminating…	*ruminando*
Cerebrating…	*«cerebrando»*

La cucina a fuoco lento

Brewing…	*in infusione*
Simmering…	*sobbollendo*
Percolating…	*filtrando (la moka)*

L'officina e la fatica

Forging…	*forgiando*
Crunching…	*macinando numeri*
Schlepping…	*sgobbando (yiddish)*

Il bighellonare creativo

Noodling…	*giocherellando*
Moseying…	*andando a zonzo*
Vibing…	*a sensazione*

Magia e new age

Conjuring…	*evocando*
Transmuting…	*trasmutando*
Manifesting…	*«manifestando»*

L'aziendalese parodico

Actioning…	*«azionando»*
Ideating…	*ideando*
Actualizing…	*«attualizzando»*

Nonsense e citazioni

Honking…	*starnazzando (l'oca)*
Reticulating…	*reticolando (SimCity)*
Clauding…	*«claudeggiando»*

E sono personalizzabili

```
// ~/.claude/settings.json
{ "spinnerVerbs": {
"mode": "append",
"verbs": ["Rimuginando", "Sobbollendo"]
} }
```

«append» aggiunge ai verbi di serie, «replace» li sostituisce.

01 · ESTRATTI A CASO

Vengono pescati da una lista fissa cablata nel programma, cresciuta dalle ~50 voci iniziali a diverse centinaia.

02 · ZERO SIGNIFICATO TECNICO

Il verbo non descrive l'operazione in corso: «Honking…» e «Synthesizing…» indicano la stessa cosa — il modello sta elaborando.

03 · UNA TRADIZIONE DEL SOFTWARE

Omaggio a «Reticulating splines…» di SimCity 2000 (1994), la frase di caricamento nonsense per antonomasia.